AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI, INFORMATYKI I INŻYNIERII BIOMEDYCZNEJ

Projekt dyplomowy

Praktyczne wprowadzenie do tworzenia sterowników urządzeń w systemie Linux na przykładzie urządzeń z magistralami SPI i I2C.
A practical introduction to creating device drivers in Linux using the example of devices with SPI and I2C buses.

Autor: *Paweł Piotr Zięba*
Kierunek studiów: *Mikroelektronika w Technice i Medycynie*
Opiekun pracy: *dr hab. inż. Mirosław Żołądź*

Kraków, 2025

Serdeczne podziękowania kieruję do mojego promotora
dr hab. inż. Mirosława Żołędzia, za wsparcie, cenne wskazówki
i merytoryczną pomoc podczas realizacji pracy, a także mojej
rodzinie i bliskim za wyrozumiałość, motywację oraz
nieocenione wsparcie w całym okresie studiów.
Dziękuję również wszystkim, którzy swoją wiedzą i radą
przyczynili się do powstania tego projektu, a w szczególności
Panu dr. inż. Pawłowi Skrzypcowi za merytoryczne wsparcie i
wkład w pracę.

Skrócony Spis treści

Wstęp....7
Zestaw narzędzi niezbędnych do wykonania instrukcji....8
Moduł jądra Linux....10
Bufor diagnostyczny....12
Klasy urządzeń....14
Urządzenia, atrybuty i wirtualny system plików....16
Obsługa błędów....24
Urządzenie znakowe....29
Urządzenie miscellaneous device....42
Drzewo urządzeń....46
Urządzenie platformowe....49
Urządzenie platformowe na przykładzie sterownika diod LED....60
Urządzenie platformowe na przykładzie sterownika serwomechanizmu....74
Urządzenia SPI/I2C na przykładzie sterowników układów scalonych....86
- Wprowadzenie....86
- Komunikacja sterownika ze scalonym przetwornikiem C\A z magistralą SPI....88
- Komunikacja "użytkownika" ze sterownikiem scalonego przetwornika C\A....99
- Komunikacja "sterownika" ze scalonym ekspanderem portów I/O z magistralą SPI z użyciem mechanizmu REGMAP....114
- Komunikacja "sterownika" ze scalonym ekspanderem portów I/O z magistralą I2C....125

Podsumowanie....133

Spis treści

Wstęp....7
Zestaw narzędzi niezbędnych do wykonania instrukcji....8
Sprzęt....8
Oprogramowanie....9
Moduł jądra Linux....10
Wprowadzenie....10
Kod minimalnego modułu....10
Kompilacja modułu....11
Ładowanie i usuwanie modułu z jądra....11
Zadanie – minimalny kod modułu....11
Bufor diagnostyczny....12
Wprowadzenie....12
Poziomy logów....12
Zadanie – obsługa bufora diagnostycznego....12
Klasy urządzeń....14
Wprowadzenie....14
Zadanie – dodawanie klasy urządzenia....14
Urządzenia, atrybuty i wirtualny system plików....16
Wprowadzenie....16
Zadanie – dodawanie urządzenia....16
Atrybuty urządzeń i komunikacja z modułem za pomocą sysfs....19
Wprowadzenie....19
Zadanie - tworzenie atrybutu urządzenia tylko do odczytu....21
Zadanie - obsługa odczytu i zapisu atrybutu urządzenia....22
Obsługa błędów....24
Wprowadzenie....24
Kody błędów....24
Funkcje PTR_ERR, ERR_PTR i IS_ERR....25
Obsługa błędów podczas inicjalizacji modułu....25
Zadanie – dodanie obsługi błędów....26
Urządzenie znakowe....29
Wprowadzenie....29
Inicjalizacja i usuwanie urządzenia znakowego....29
Numery major i minor....29
Zadanie – tworzenie urządzenia znakowego....30
Operacje na pliku urządzenia znakowego....33
Zadanie – obsługa otwierania i zamykania pliku urządzenia....34
Zadanie – Obsługa zapisu i odczytu danych z pliku urządzenia....35
Wymiana danych z urządzeniem znakowym....36
Zadanie – komunikacja z sterownika z przestrzenią użytkownika....37
Urządzenie miscellaneous device....42
Wprowadzenie....42
Zadanie – komunikacja za pomocą miscdevice....43
Drzewo urządzeń....46
Wprowadzenie....46
Zadanie – odczyt nazwy platformy sprzętowej z drzewa urządzeń....46
Nakładki drzewa urządzeń i kompilacja....46
Zadanie – dodanie nakładki drzewa urządzeń....48
Urządzenie platformowe....49
Wprowadzenie....49

Tworzenie instancji urządzenia platformowego na podstawie drzewa urządzeń.....49
Dopasowanie sterownika do urządzenia.....49
Rejestracja sterownika platformowego.....50
Zadanie – dodawanie i usuwanie urządzenia platformowego.....51
Odczyt właściwości węzła z drzewa urządzeń.....52
Zadanie - *dodanie obsługi odczytu właściwości węzła w sterowniku*.....53
Problem dostępu do globalnej zmiennej modułu przez wiele instancji urządzenia.....54
Zadanie - przedstawienie problemu z wieloma instancjami urządzenia.....54
Rozwiązanie problemu dostępu do globalnej zmiennej modułu przez wiele instancji urządzenia – prywatne dane urządzenia.....56
Zadanie – rozwiązanie problemu wielu instancji urządzenia.....57
Urządzenie platformowe na przykładzie sterownika diod LED.....60
Wprowadzenie.....60
Dostęp do GPIO w drzewie urządzeń.....61
Zadanie – sterowanie diodą LED z użyciem drzewa urządzeń.....63
Zadanie – odczyt stanu przycisku.....65
Zadanie - współpraca sterownika z aplikacją użytkownika.....67
Urządzenie platformowe na przykładzie sterownika serwomechanizmu.....74
Cykliczne sterowani pinami GPIO - Timer.....74
Dostęp do prywatnych danych urządzenia w funkcjach.....75
Makro container_of.....75
Zadanie - dodanie obsługi timera i GPIO.....76
Obsługa serwomechanizmu.....80
Zadanie – sterownik serwomechanizmu.....81
Urządzenia SPI/I2C na przykładzie sterowników układów scalonych.....86
Wprowadzenie.....86
Magistrala SPI.....86
Magistrala I2C.....87
Komunikacja sterownika ze scalonym przetwornikiem C\A z magistralą SPI.....88
Zadanie – podłączenie układu MCP4921 do kontrolera Raspberyy Pi Zero W.....90
Nakładka drzewa urządzeń dla sterownika SPI.....90
Zadanie – przygotowanie nakładki dla urządzenia MCP4921.....92
Przygotowanie minimalnego kodu sterownika SPI.....93
Zadanie - przygotowanie minimalnego kodu sterownika SPI.....93
Komunikacja z przetwornikiem C/A.....95
Zadanie - komunikacja z przetwornikiem C/A.....95
Komunikacja "użytkownika" ze sterownikiem scalonego przetwornika C\A.....99
Za pomocą atrybutu sysfs na przykładzie ustawiania napięcia przetwornika 99
Za pomocą urządzenia znakowego – łańcuchy znakowe.....102
Za pomocą urządzenia znakowego – mechanizm ioctl.....106
Zadanie – dodanie obsługi ioctl.....107
Komunikacja "sterownika" ze scalonym ekspanderem portów I/O z magistralą SPI z użyciem mechanizmu REGMAP.....114
Komunikacja "sterownika" z układem, na przykładzie z magistralą SPI i układem scalonym o architekturze rejestrowej.....116
Zadanie – obsługa komunikacji SPI za pomocą mechanizmu regmap.....117
Komunikacja "sterownika" ze scalonym ekspanderem portów I/O z magistralą I2C.....125
Zadanie – obsługa komunikacji I2C za pomocą mechanizmu regmap.....126
Podsumowanie.....133

Wstęp

Sterowniki urządzeń odgrywają kluczową rolę w systemach operacyjnych takich jak Linux, pełniąc funkcję pomostu między sprzętem a oprogramowaniem. Ich zadaniem jest ukrycie szczegółów technicznych urządzenia i udostępnienie jednolitego interfejsu programistycznego, który umożliwia użytkownikom korzystanie z funkcji sprzętu za pomocą standardowych wywołań systemowych. Dzięki temu sterowniki urządzeń są modułami umożliwiającymi dynamiczne rozszerzanie funkcji jądra systemu Linux [1].

System operacyjny Linux różni się od systemów typu bare-metal tym, że oferuje zaawansowany model ochrony i izolacji zasobów. Przestrzeń jądra (kernel space) zapewnia pełny dostęp do sprzętu i jest odpowiedzialna za obsługę systemowych wywołań procesów użytkownika (user space), które działają z ograniczonymi uprawnieniami. Przestrzeń jądra działa w trybie uprzywilejowanym procesora, co pozwala na bezpośredni dostęp do pamięci i zasobów, podczas gdy przestrzeń użytkownika jest projektowana w celu ochrony systemu przed potencjalnymi błędami aplikacji [1].

W odróżnieniu od systemów bare-metal, które często są projektowane pod kątem określonego zastosowania bez pośrednictwa systemu operacyjnego, Linux oferuje warstwę abstrakcji nad sprzętem. Ułatwia to implementację złożonych sterowników urządzeń, które mogą być używane na różnych platformach sprzętowych. Dzięki modułowości i elastyczności Linux stał się popularnym wyborem w systemach wbudowanych, które wymagają zarówno niskopoziomowego dostępu do sprzętu, jak i możliwości korzystania z rozbudowanych funkcji systemu operacyjnego [1].

Inspiracją do napisania tej pracy była chęć zgłębienia tematyki systemów wbudowanych oraz programowania w systemie Linux. Praca ta opisuje proces tworzenia sterowników urządzeń na przykładzie magistral SPI i I2C, począwszy od prostego modułu, aż po kompletny sterownik.

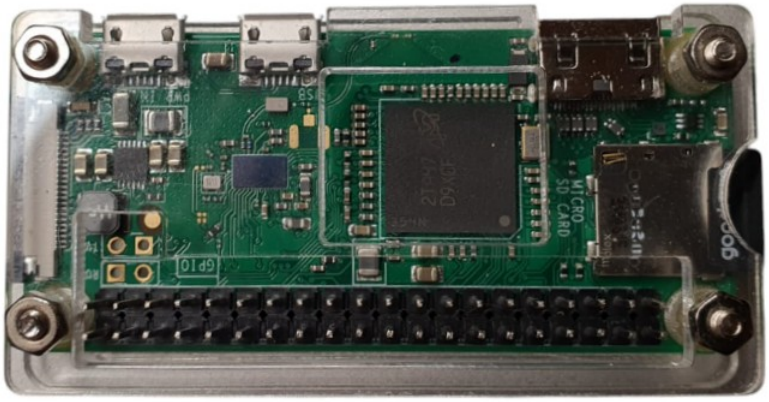
Zestaw narzędzi niezbędnych do wykonania instrukcji

W niniejszym rozdziale przedstawiono zestaw narzędzi sprzętowych i programowych niezbędnych do realizacji instrukcji implementacji sterowników w systemie Linux. Poniżej opisano wymagany sprzęt oraz oprogramowanie, które zostały wykorzystane podczas realizacji pracy.

Sprzęt

Do wykonania zadań przedstawionych w instrukcji zastosowano następujące komponenty:

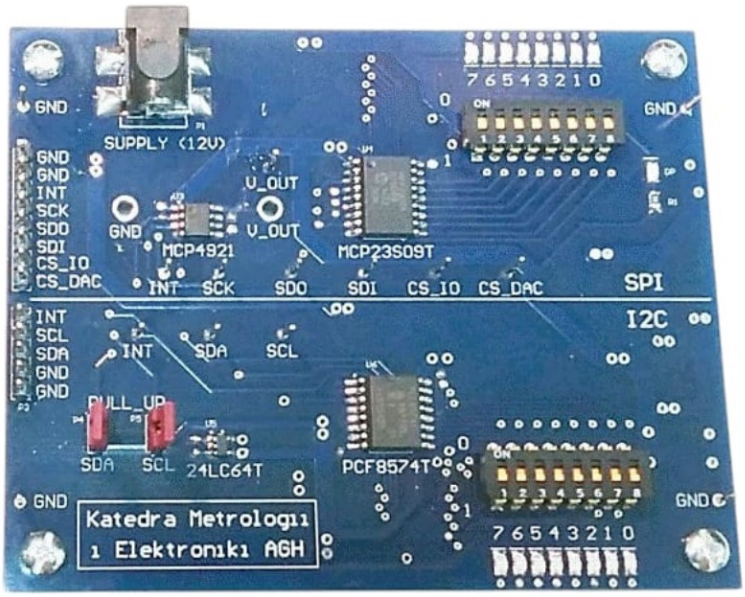
- Raspberry Pi Zero W – kompaktowy komputer jednopłytkowy, który pełni rolę głównego urządzenia obliczeniowego. Jego niewielkie wymiary oraz wsparcie dla różnych interfejsów komunikacyjnych, takich jak SPI, I2C i GPIO, czynią go odpowiednim wyborem do nauki tworzenia sterowników.



- Nakładka HAT z diodami LED i przyciskami – moduł rozszerzający funkcjonalność Raspberry Pi, pozwalający na wizualizację oraz interakcję z urządzeniem za pomocą diod i przycisków, co umożliwia praktyczne testowanie sterowników.

- Zestaw do ćwiczeń z układami komunikującymi się przez magistrale SPI i I2C – zestaw zawierający układy scalone, takie jak:

- MCP4921 – cyfrowo-analogowy przetwornik (DAC) komunikujący się za pomocą magistrali SPI,
- MCP23S09 – układ ekspandera portów GPIO, również korzystający z magistrali SPI,
- PCF8574 – układ ekspandera portów GPIO, wykorzystujący magistralę I2C. Zestaw ten umożliwia praktyczne ćwiczenia związane z implementacją sterowników obsługujących popularne protokoły komunikacyjne.



- Multimetr cyfrowy – urządzenie pomiarowe używane do weryfikacji poprawności działania sterowników poprzez sprawdzanie napięć, sygnałów oraz stanu linii komunikacyjnych.

Oprogramowanie

Aby zrealizować instrukcję, niezbędne było przygotowanie odpowiedniego środowiska programistycznego. Zastosowano następujące oprogramowanie:

- System operacyjny Linux z wersją jądra 6.1.21 – system Linux stanowi podstawę środowiska pracy, a wybrana wersja jądra zapewnia wsparcie dla omawianych funkcji i mechanizmów implementacji sterowników.
- Python 3.9.2 – język programowania wykorzystywany do skryptów wspomagających pracę oraz testowanie sterowników.
- Biblioteka Curses w wersji 6.2 – biblioteka używana w Pythonie do tworzenia interfejsów tekstowych, co pozwala na wygodne prezentowanie wyników testów w terminalu.

Moduł jądra Linux

Wprowadzenie

Moduł jądra Linux to niewielki fragment kodu, który można dynamicznie włączyć do działającego jądra systemu operacyjnego. Dzięki temu jądro zyskuje nowe możliwości, na przykład obsługę dodatkowych urządzeń (USB, kart sieciowych i wielu innych), systemów plików lub innych funkcji, bez konieczności jego ponownej kompilacji. Moduły jądra są jednym ze sposobów implementacji sterowników urządzeń.

Kod minimalnego modułu

Poniżej przedstawiono przykładowy kod minimalnego modułu. Składa się on z:

- dołączenia wymaganych plików nagłówkowych,
- definicji funkcji, które mają być wykonane podczas ładowania i usuwania modułu z jądra (tak zwane *callbacks*),
- makr służących do zarejestrowania utworzonych funkcji w jądrze,
- makra określającego licencję udostępniania modułu

Licencja *GPL v2 [2]* to licencja wolnego oprogramowania stworzona przez Free Software Foundation (FSF). Jej głównym celem jest zapewnienie użytkownikom swobody używania, modyfikowania i dystrybucji oprogramowania, przy jednoczesnym zachowaniu tych samych praw dla innych użytkowników.

Dodanie informacji o licencji do modułu jądra jest konieczne, w przeciwnym wypadku proces kompilacji się nie powiedzie.

```
#include <linux/init.h>
#include <linux/module.h>

static int __init on_init(void)
{
        // module initialisation code
        return 0
}
static void __exit on_exit(void)
{
        // module exit code
}

module_init(on_init);
module_exit(on_exit);

MODULE_LICENSE("GPL v2");
```

Kompilacja modułu

Kompilacja modułów dla jądra Linux odbywa się przy pomocy narzędzia *Make.* W pierwszym wierszu pliku *Makefile* przedstawionego poniżej, podano nazwę pliku obiektowego, który będzie wynikiem kompilacji. Nazwa pliku obiektowego musi zgadzać się z nazwą pliku źródłowego (z pominięciem rozszerzeń).

```
obj-m += minimod.o

all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Ładowanie i usuwanie modułu z jądra

Moduły jądra to pliki binarne, które zawierają kod rozszerzający jądro o nowe funkcje (np. sterowniki urządzeń). Aby załadować moduł, używa się polecenia insmod (np. insmod minimod.ko) lub modprobe (np. modprobe minimod), które automatycznie obsługuje zależności.

Zadanie – minimalny kod modułu

1. Stworzyć plik o nazwie *minimod.c* zawierający kod źródłowy przedstawiony we wstępie.
2. Stworzyć plik *Makefile* na podstawie przykładu zamieszczonego we wstępie.
3. Wykonać kompilację modułu poleceniem: *make*.
4. Załadować moduł do jądra poleceniem: *sudo insmod [nazwa pliku modułu].ko*.
5. Sprawdzić czy moduł został poprawnie załadowany: *lsmod* | *grep [nazwa pliku modułu]*.
   (polecenie *grep* pozwala na wyświetlenie tylko linii zawierającej nazwę podanego pliku)
   Umieścić wynik z konsoli poniżej.

```
pi@pi:~ $ lsmod | grep minimod
minimod                 1128  0
```

6. Usunąć moduł z jądra poleceniem: sudo rmmod [nazwa pliku modułu]
7. Sprawdzić, czy moduł został usunięty z jądra i umieścić wynik poniżej.

```
pi@pi:~ $ lsmod | grep minimod
```

Bufor diagnostyczny

Wprowadzenie

Bufor diagnostyczny to bufor do przechowywania komunikatów systemowych generowanych przez jądro i niektóre procesy użytkownika. Jest to mechanizm umożliwiający monitorowanie działania systemu oraz diagnostykę problemów.
Dane z bufora wyświetla się za pomocą polecenia *dmesg*.

Poziomy logów

Ważność komunikatów znajdujących się w buforze diagnostycznym, określana jest za pomocą tak zwanych poziomów logów. Ułatwiają one filtrowanie i analizę logów diagnostycznych w zależności od ich priorytetu.

Poniżej przedstawiono wszystkie poziomy logów dostępne w systemie Linux.

- 0 - Emergency: Krytyczne błędy powodujące zatrzymanie systemu. (makro: *pr_emerg(fmt, ...)*)
- 1 - Alert: Bardzo poważne błędy wymagające natychmiastowej interwencji. (Funkcja: *pr_alert(fmt, ...)*)
- 2 - Critical: Poważne błędy wpływające na działanie systemu. (makro: *pr_crit(fmt, ...)*)
- 3 - Error: Ogólne błędy, system nadal działa poprawnie. (makro: *pr_err(fmt, ...)*)
- 4 - Warning: Ostrzeżenia o potencjalnych problemach. (makro: *pr_warn(fmt, ...)*)
- 5 - Notice: Ważne zdarzenia wymagające uwagi administratora. (makro: *pr_notice(fmt, ...)*)
- 6 - Informational: Informacje o stanie systemu. (makro: *pr_info(fmt, ...)*)
- 7 - Debug: Szczegółowe dane do debugowania. (makro: *pr_debug(fmt, ...)*)

Makra służące do wyświetlania komunikatów, jako argumenty przyjmują sformatowany ciąg znaków, który ma zostać umieszczony w buforze diagnostycznym.

Zadanie – obsługa bufora diagnostycznego

Dodać do utworzonego modułu zapisywanie komunikatów do bufora diagnostycznego. W tym celu:

1. Zapisać kod modułu *minimod* pod nazwą *dmes.c.*
2. Zmodyfikować kod modułu *dmes*, tak aby wpisał do bufora diagnostycznego komunikaty:
   - „Module init” przy ładowaniu modułu (w funkcji *on_init*),
   - „Module exit” przy jego usuwaniu (w funkcji *on_exit*).

3. Rozwiązanie zamieścić poniżej.

```
#include <linux/init.h>
#include <linux/module.h>

static int __init on_init(void)
{
        pr_info("Module init\n");
        return 0;
}

static void __exit on_exit(void)
{
        pr_info("Module exit\n");
}

module_init(on_init);
module_exit(on_exit);

MODULE_LICENSE("GPL v2");
```

4. Sprawdzić funkcjonalność dodaną w punkcje 2. i zamieścić dane wpisane do bufora poniżej.

```
pi@pi:~ $ dmesg
[...]
[  203.977630] dmes: loading out-of-tree module taints kernel.
[  203.978146] Module init
[  278.299880] Module exit
```

Klasy urządzeń

Wprowadzenie

Klasy urządzeń w Linuxie to logiczne grupy urządzeń o podobnych funkcjach lub właściwościach. Podobnie jak w przypadku plików reprezentujących dane na dysku, które są grupowane w katalogach, klasy pozwalają na organizację urządzeń w kategorie, co ułatwia zarządzanie nimi oraz dostęp do nich przez system operacyjny. Przykładowo urządzenia reprezentujące piny GPIO należą do klasy GPIO.

Linux udostępnia informacje o klasach urządzeń w systemie plików */sys*. Każda klasa ma swój katalog w */sys/class*, który zawiera symboliczne dowiązania do rzeczywistych urządzeń w */sys/devices*.

W systemie Linux nie każde urządzenie musi należeć do konkretnej klasy. Niektóre „specyficzne" urządzenia nie wymagają prezentowania plików w przestrzeni użytkownika i działają tylko w obrębie jądra, są to różne kontrolery np.. zegarów, DMA itd.

Bez przydzielonej klasy urządzenie nie jest reprezentowane w katalogu /*sys/class,* oraz nie zadziała mechanizm *udev*, który automatycznie tworzy plik urządzenia znakowego w katalogu */dev* (w przypadku urządzenia znakowego).

Zadanie – dodawanie klasy urządzenia

1. Zapisać kod modułu *dmes* pod nazwą *class.c*
2. Dołączyć plik nagłówkowy *linux/cdev.h,* który pozwoli na użycie funkcji obsługujących tworzenie klas.
3. Utworzyć globalny wskaźnik na zmienną typu *struct class* o nazwie *my_class*.
4. W trakcie inicjalizacji modułu dodać tworzenie klasy. W tym celu użyć makra *class_create*, które przyjmuje dwa argumenty:
    1. wskaźnik na bieżący moduł (użyj definicji *THIS_MODULE*),
    2. nazwę klasy (ciąg znakowy).

   Makro zwraca wskaźnik na utworzoną klasę, który należy przypisać do wskaźnika *my_class* z punktu 2.
5. W trakcie usuwania modułu z jądra, usunąć klasę urządzenia za pomocą funkcji *class_destroy*, która jako argument przyjmuje wskaźnik na utworzoną klasę.
6. Zamieścić kod modułu poniżej.

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/cdev.h>

struct class *my_class;

static int __init on_init(void)
```

```
{
        pr_info("Module init\n");
        my_class = class_create(THIS_MODULE, „my_class”);
        return 0;
}

static void __exit on_exit(void)
{
        pr_info("Module exit\n");
        class_destroy(my_class);
}

module_init(on_init);
module_exit(on_exit);

MODULE_LICENSE("GPL v2");
```

7. Skompilować i załadować moduł.
8. Sprawdzić czy w katalogu */sys/class* znajduje się katalog reprezentujący stworzoną klasę. Użyć polecenia *ls -al /sys/class/my_class | grep [nazwa klasy]*. Wynik zamieścić poniżej.

```
pi@pi:~ $ ls -al /sys/class | grep my_class
drwxr-xr-x  2 root root 0 Dec 31 15:13 my_class
```

9. Wyświetlić zawartość katalogu utworzonej klasy i umieścić ją poniżej.

```
pi@pi:~ $ ls -a /sys/class/my_class
.  ..
```

Katalog jest pusty, ponieważ nie zostały jeszcze utworzone żadne urządzenia skojarzone z nową klasą.

10. Usunąć moduł z jądra.
11. Ponownie wyświetlić zawartość katalogu klasy i umieścić wynik poniżej.

```
pi@pi:~  $ ls -a /sys/class/my_class
ls: cannot access '/sys/class/my_class': No such file or directory
```

Urządzenia, atrybuty i wirtualny system plików

Wprowadzenie

Urządzenie w systemie Linux to dowolny zasób komputerowy, który jest zarządzany i wykorzystywany przez system operacyjny. Może to być element wewnętrzny, taki jak procesor, karta sieciowa, czy pamięć, lub zewnętrzny, jak drukarka czy kamera.

W celu dodania urządzenia do jądra podczas inicjalizacji modułu, należy użyć funkcji *device_create*, której deklarację przedstawiono poniżej. [3]

```
struct device *device_create(struct class *class,
                             struct device *parent,
                             dev_t devt, void *drvdata,
                             const char *fmt, ...);
```

Argumenty funkcji:

- **class* – wskaźnik na klasę, do której należeć będzie urządzenie,
- **parent* – wskaźnik na urządzenie nadrzędne, jeśli istnieje (podać *Null),*
- *devt* – numer identyfikujący urządzenia znakowego (podać 0),
- **drvdata* – wskaźnik na prywatne dane urządzenia (podać *Null*),
- **fmt* – nazwa urządzenia widoczna w systemie (podać „*my_device”*).

Funkcja device_create zwraca wskaźnik na utworzone urządzenie jeśli nie wystąpi błąd, w przeciwnym wypadku zwraca wskaźnik ERR_PTR, który zostanie omówiony w dalszej części instrukcji.

Urządzenie dodane do jądra trzeba usunąć w momencie usuwania modułu z jądra. Do tego celu służy funkcja *device_destroy*. [3]

```
void device_destroy(struct class *class, dev_t devt);
```

Argumenty funkcji:

- **class* – wskaźnik na klasę, do której należeć będzie urządzenie,
- *devt* – numer identyfikujący urządzenia znakowego (podać 0).

Zadanie – dodawanie urządzenia

1. Zapisać kod modułu *class* pod nazwą *device.c.*
2. Utworzyć globalny wskaźnik na zmienną typu *struct device* o nazwie *my_device.*
3. W trakcie inicjalizacji modułu dodać tworzenie urządzenia. W tym celu użyć funkcji *device_create,* której deklarację przedstawiono poniżej.
4. W trakcie usuwania modułu dodać usuwanie urządzenia. W tym celu użyć funkcji *device_destroy,* której deklarację przedstawiono poniżej. Kolejność operacji podczas

usuwania modułu powinna być odwrotna do kolejności podczas ładowania modułu do jądra.

5. Zamieścić kod modułu poniżej.

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/cdev.h>

struct class *my_class;
struct device *my_device;

static int __init on_init(void)
{
        pr_info("Module init\n");
        my_class = class_create(THIS_MODULE, "my_class");
        my_device = device_create(my_class, NULL, 0, NULL,
                                  "my_device");
        return 0;
}

static void __exit on_exit(void)
{
        pr_info("Module exit\n");
        device_destroy(my_class, 0);
        class_destroy(my_class);
}

module_init(on_init);
module_exit(on_exit);
MODULE_LICENSE("GPL v2");
```

6. Skompilować i załadować moduł.
7. Sprawdzić czy urządzenie pojawiło się w systemie plików.
   Użyć polecenia *ls -al /sys/class/my_class*, *ls /sys/devices/virtual/my_class* oraz *ls /sys/devices/virtual/my_class/my_device*. Wyniki zamieścić poniżej.

```
pi@pi:~ $ ls -al /sys/class/my_class
drwxr-xr-x  2 root root 0 Dec 31 17:38 .
drwxr-xr-x 60 root root 0 Sep  5 23:06 ..
lrwxrwxrwx  1 root root 0 Dec 31 17:40 my_device ->
../../devices/virtual/my_class/my_device

pi@pi:~ $ ls /sys/devices/virtual/my_class
my_device

pi@pi:~ $ ls /sys/devices/virtual/my_class/my_device
power  subsystem  uevent
```

Urządzenie zostało poprawnie utworzone i przydzielone do klasy my_class.

8. Usunąć moduł device z jądra.
9. Sprawdzić czy katalog /sys/devices/virtual/my_class/my_device istnieje. Wynik zamieścić poniżej.

```
pi@pi:~ $ ls /sys/devices/virtual/my_class/my_device
ls: cannot access '/sys/devices/virtual/my_class/my_device': No such
file or directory
```

Urządzenie *my_device* i klasa *my_class* zostały poprawnie usunięte z systemu.

Atrybuty urządzeń i komunikacja z modułem za pomocą sysfs

Wprowadzenie

Atrybut urządzenia w systemie Linux to właściwość lub cecha związana z danym urządzeniem, która określa jego parametry lub sposób działania. Atrybuty mogą odnosić się do specyficznych informacji o urządzeniu, takich jak jego identyfikator, typ, stan, czy konfiguracja. Są one często wykorzystywane przez system operacyjny i aplikacje do identyfikacji i zarządzania sprzętem.

W Linuxie atrybuty urządzeń mogą być odczytywane i ustawiane przez różne mechanizmy, takie jak na przykład *sysfs*.

Sysfs w systemie Linux to wirtualny system plików, który udostępnia hierarchiczną strukturę danych. W odróżnieniu od tradycyjnych systemów plików, które przechowują trwałe dane na nośnikach, wirtualne systemy plików w Linuxie, takie jak */proc* czy */sys*, same w sobie nie zawierają fizycznych danych ani plików z dysków. Służą one głównie do prezentowania informacji systemowych i sprzętowych dynamicznie (w momencie odczytu lub zapisu).

W tym rozdziale atrybuty urządzenia zostaną wykorzystane do komunikacji z modułem jądra. Dostęp do atrybutów ograniczony jest za pomocą praw dostępu. Trzy podstawowe tryby dostępu to:

- atrybut do odczytu,
- atrybut do zapisu,
- atrybut do odczytu i zapisu.

W celu dodania atrybutu urządzenia należy utworzyć funkcję do jego odczytu/zapisu i zarejestrować go za pomocą makrodefinicji *DEVICE_ATTR_RO* przyjmuje ono jako argument nazwę zmiennej, w której przechowywana będzie wartość atrybutu. Makro to tworzy strukturę typu *device_attribute* o nazwie *dev_attr_[nazwa podanej zmiennej]*, która reprezentuje atrybut urządzenia.

Poniżej przedstawiono przykład, który przedstawia tworzenie atrybutu urządzenia tylko do odczytu.

```
static int my_attr;
ssize_t my_attr_show(struct device *dev,
                     struct device_attribute *attr,
                      char *buf)
{
        /* my_attr read code */
}
DEVICE_ATTR_RO(my_attr);
```

Argumenty funkcji:

- **dev* – wskaźnik na urządzenie, którego atrybut dotyczy,
- **attr* – wskaźnik na strukturę atrybutu urządzenia utworzoną za pomocą *DEVICE_ATTR_RO*,
- **buf* – wskaźnik na bufor, przez który przekazuje się odczytaną wartość.

Funkcja powinna zwrócić ilość poprawnie odczytanych znaków lub ujemny kod błędu.

W celu dodania funkcjonalności zapisu atrybutu należy zdefiniować analogiczną funkcję do *[nazwa atrybutu]_show* o nazwie *[nazwa atrybutu]_**store***, której przykładowe użycie przedstawiono poniżej.

```
ssize_t my_attr_store(struct device *dev,
                      struct device_attribute *attr,
                       const char *buf, size_t count);
{
        /* my_attr write code */
}
```

Argumenty funkcji:

- **dev* – wskaźnik na urządzenie, którego atrybut dotyczy,
- **attr* – wskaźnik na strukturę atrybutu urządzenia utworzoną za pomocą *DEVICE_ATTR_RW*,
- **buf* – wskaźnik na bufor, przez który przekazuje się odczytaną wartość,
- *count* – liczba znaków do zapisania.

Funkcja powinna zwrócić ilość poprawnie zapisanych znaków.

Po dodaniu atrybutu należy utworzyć plik reprezentujący go w systemie plików. Robi się to za pomocą funkcji *device_create_file*, której deklarację przedstawiono poniżej. [3]

```
int device_create_file(struct device *dev,
                       const struct device_attribute *attr);
```

Argumenty funkcji:

- *dev – wskaźnik na urządzenie, którego atrybut dotyczy,
- *attr – wskaźnik na strukturę atrybutu urządzenia utworzoną za pomocą *DEVICE_ATTR_RO*.

Funkcja zwraca wartość 0, jeśli zostanie wykonana poprawnie. W przypadku wystąpienia błędu zwraca odpowiedni kod błędu.

W momencie usuwania modułu z jądra, konieczne jest usunięcie utworzonego wcześniej pliku. Do tego celu służy funkcja *device_remove_file*, której deklarację przedstawiono poniżej. [3]

```
void device_remove_file(struct device *dev,
                        const struct device_attribute *attr);
```

Argumenty funkcji:

- *dev – wskaźnik na urządzenie, którego atrybut dotyczy,
- *attr – wskaźnik na strukturę atrybutu urządzenia utworzoną za pomocą *DEVICE_ATTR_RO*.

Zadanie - tworzenie atrybutu urządzenia tylko do odczytu

1. Zapisać kod modułu *device* pod nazwą *attribute.c.*
2. Dodać zmienną globalną *static int my_attr = 17,* w której przechowywana będzie wartość atrybutu.
3. Zdefiniować funkcję odczytu atrybutu.
   Funkcja ta zostanie wywołana w momencie odczytu pliku atrybutu. Kluczowym elementem jest nazwa, która musi odpowiadać następującemu wzorcowi: *[nazwa atrybutu podana przy jego rejestracji]_show.*
4. Zarejestrować atrybut tylko do odczytu za pomocą makra DEVICE_ATTR_RO, jako argument podać *my_attr.*
5. W trakcie inicjalizacji modułu dodać tworzenie pliku atrybutu za pomocą funkcji *device_create_file*.
6. W trakcie usuwania modułu dodać usuwanie pliku atrybutu za pomocą funkcji *device_remove_file*.
7. Skompiluj i załaduj moduł. Umieść jego kod poniżej.

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/cdev.h>

struct class *my_class;
struct device *my_device;

static int my_attr = 17;

static ssize_t my_attr_show(struct device *dev,
                            struct device_attribute *attr,
                            char *buf)
{
        return sprintf(buf, "%d\n", my_attr);
}
DEVICE_ATTR_RO(my_attr);

static int __init on_init(void)
{
        pr_info("Module init\n");
        my_class = class_create(THIS_MODULE, "my_class");
        my_device = device_create(my_class, NULL, 0, NULL,
                                  "my_device");
        device_create_file(my_device, &dev_attr_my_attr);
        return 0;
}
```

```
static void __exit on_exit(void)
{
        pr_info("Module exit\n");
        device_remove_file(my_device, &dev_attr_my_attr);
        device_destroy(my_class, 0);
        class_destroy(my_class);
}

module_init(on_init);
module_exit(on_exit);

MODULE_LICENSE("GPL v2");
```

8. Sprawdzić czy atrybut urządzenia został utworzony: *ls/sys/devices/virtual/my_class/my_device* i umieść wynik poniżej. Jakie atrybuty oprócz *my_attr* posiada urządzenie?

```
pi@pi:~ $ ls /sys/devices/virtual/my_class/my_device
my_attr  power  subsystem  uevent
```

Oprócz *my_attr* urządzenie posiada jeszcze 3 atrybuty:

- *power* - ten plik zawiera informacje związane z zarządzaniem energią dla urządzenia,
- *subsystem* - ten plik jest symbolicznym linkiem do katalogu reprezentującego klasę urządzenia, do której należy dane urządzenie,
- *uevent* - służy do komunikacji z systemem *udev* i zawiera informacje o zdarzeniach dotyczących urządzenia.

9. Odczytaj zawartość parametru za pomocą polecenia *cat /sys/devices/virtual/my_class/my_device/my_attr*, a wynik umieść poniżej.

```
pi@pi:~ $ cat /sys/devices/virtual/my_class/my_device/my_attr
17
```

Zadanie - obsługa odczytu i zapisu atrybutu urządzenia

1. Zamienić makro *DEVICE_ATTR_RO* na *DEVICE_ATTR_RW*, które zarejestruje atrybut z prawami do odczytu i zapisu.
2. Zdefiniować funkcję zapisu atrybutu. Zostanie ona wykonana podczas zapisu do pliku atrybutu, a jej nazwa musi wyglądać następująco: *[nazwa atrybutu podana przy jego rejestracji]_store.* Do konwersji ciągu znaków na liczbę wykorzystaj funkcję *kstrtol*, która zwraca *0* w przypadku powodzenia lub kod błędu.
3. Poniżej zamieścić kod funkcji oraz makro z punktów 11. i 12.

```
static ssize_t my_attr_store(struct device *dev,
                             struct device_attribute attr,
                             const char*buf, size_t count)
{
        kstrtol(buf, 0, (long int*) &my_attr);
        return count;
}
DEVICE_ATTR_RW(my_attr);
```

4. Usunąć i ponownie załadować moduł do jądra.
5. Przetestować możliwość zapisu atrybutu. Zapisać wartość *10,* a następnie ją odczytać. Do zapisu użyć polecenia *sudo sh -c "echo 10 >> [ścieżka do pliku atrybutu].* Wynik zamieścić poniżej.

```
pi@pi:~ $ sudo sh -c "echo 10 >>
/sys/devices/virtual/my_class/my_device/my_attr"
pi@pi:~ $ cat /sys/devices/virtual/my_class/my_device/my_attr
10
```

Obsługa błędów

Wprowadzenie

Pisanie kodu modułu jądra systemu Linux wymaga szczególnej uwagi, ponieważ moduł ma bezpośredni dostęp do kluczowych zasobów systemu, takich jak na przykład przestrzeń adresowa jądra. Nieodpowiednie zarządzanie tymi zasobami może prowadzić do poważnych problemów, takich jak zawieszenie systemu, krytyczne błędy czy konieczność ponownego uruchomienia urządzenia.

Dlatego obsługa błędów w kodzie modułu jest niezbędna Przykładowo przed próbą zapisu danych w pamięci należy upewnić się, że alokacja pamięci zakończyła się sukcesem. Każda operacja potencjalnie podatna na błędy tj. alokacja zasobów, operacje na urządzeniach czy komunikacja z innymi modułami – powinna być odpowiednio obsłużona i zweryfikowana.

Kody błędów

W Linuxie istnieją zdefiniowane kody typowych błędów występujących podczas pracy systemu. Każda funkcja, której poprawne wykonanie nie jest w 100% pewne, w przypadku niepowodzenia powinna zwracać kod błędu lub wskaźnik *ERR_PTR*, natomiast jeśli funkcja wykona się poprawnie, powinna zwrócić wartość *0.*

Kody błędów zwracane są jako ujemne wartości (znak *minus* jest dodawany w momencie zwracania wartości przez funkcję np. *return -EIO*), listę kilku przykładowych kodów przedstawiono poniżej. [4]

Błąd	Kod błędu	Opis błędu
EPERM	1	Operacja nie jest dozwolona, brak odpowiednich uprawnień.
ENOENT	2	Nie znaleziono pliku lub katalogu.
ESRCH	3	Nie znaleziono procesu odpowiadającego podanym kryteriom.
EINTR	4	Systemowe wywołanie zostało przerwane przez sygnał.
EIO	5	Wystąpił błąd wejścia/wyjścia.
ENXIO	6	Nie istnieje urządzenie lub adres, do którego próbujesz się odwołać.
E2BIG	7	Lista argumentów przekazana do programu jest za długa.
ENOEXEC	8	Nieprawidłowy format pliku wykonywalnego.
EBADF	9	Nieprawidłowy deskryptor pliku.
ECHILD	10	Brak procesów potomnych do oczekiwania.

Funkcje PTR_ERR, ERR_PTR i IS_ERR

W systemie Linux wiele funkcji zwraca wskaźnik na strukturę lub zasób, ale w przypadku błędu mogą zamiast tego zwrócić specjalną wartość wskazującą na kod błędu. Aby poprawnie zinterpretować informację o błędzie należy przekonwertować ten wskaźnik na kod błędu korzystając z funkcji:

- *IS_ERR* zwraca wartość 1 jeśli podany w argumencie wskaźnik reprezentuje błąd lub wartość 0 w przeciwnym przypadku,
- *PTR_ERR* zwraca kod błędu, jako argument przyjmuje wskaźnik z zakodowanym błędem,
- *ERR_PTR* zwraca wskaźnik w którym zakodowany jest błąd, jako argument przyjmuje kod błędu.

Obsługa błędów podczas inicjalizacji modułu

Podczas inicjalizacji modułu jądra często wykonujemy różne operacje, które wymagają weryfikacji ich poprawności. Jeśli któraś z nich zakończy się niepowodzeniem, dalsza część procesu inicjalizacji staje się zbędna i należy ją przerwać, co wiąże się z koniecznością cofnięcia dotychczasowych operacji. Przykładem może być sytuacja, gdy pamięć dla urządzenia została poprawnie zaalokowana, ale nie udało się utworzyć atrybutu – w takim przypadku cała inicjalizacja musi zostać anulowana, a pamięć zwolniona.

W takich przypadkach powszechnie używa się instrukcji *goto*. Pozwala ona na szybkie przeskoczenie do innej części funkcji i pominięcie kodu, który nie powinien zostać wykonany. Przydaje się to, gdy konieczne jest cofnięcie zmian w odwrotnej kolejności do ich wprowadzania. Użycie instrukcji *goto* upraszcza kod i jest standardowym rozwiązaniem w takim zastosowaniu.

Poniżej przedstawiono wykorzystanie instrukcji *goto* podczas obsługi błędów inicjalizacji modułu. Istotna jest kolejność wykonywania operacji – cofanie dokonanych zmian odbywa się w odwrotnej kolejności niż w przypadku inicjalizacji.

```
static int __init on_init(void)
{
        int err = 0;

        <utworzenie klasy urządzenia>
        <jeśli wystąpił błąd> goto out_ret_err;

        <utworzenie urządzenia>
        <jeśli wystąpił błąd> goto out_delete_class;

        <utworzenie atrybutu>
        <jeśli wystąpił błąd> goto out_delete_device;

        /* Zakończenie inicjalizacji */
        goto out_ret_err;
```

```
out_delete_device:
        <usunięcie urządzenia>
out_delete_class:
        <usunięcie klasy urządzenia>
out_ret_err:
        return err;
}
```

Zadanie – dodanie obsługi błędów

Celem zadania jest dodanie obsługi błędów do modułu *attribute*. Należy sprawdzić czy użyte funkcje konwertujące ciągi znaków oraz liczby (*kstrtol* i *sprintf*), wykonują się poprawnie, oraz czy inicjalizacja modułu przebiegła pomyślnie (tworzenie klasy, urządzenia i pliku urządzenia). W przypadku wykrycia błędu należy zamieścić odpowiedni wpis w buforze diagnostycznym i zwrócić kod błędu.

1. Zapisać kod modułu *attribute* pod nazwą *errors.c.*
2. Dodać obsługę błędów do funkcji *my_attr_store.* W tym celu:
   - utworzyć zmienną *int err = 0*,
   - do zmiennej przypisać wynik funkcji *kstrtol*, która konwertuje ciąg znaków na liczbę typu *long.* Deklarację funkcji przedstawiono poniżej. [3]

```
static inline int __must_check kstrtol(const char *s,
                                       unsigned int base,
                                       long *res)
```

Argumenty funkcji:

- **s* – wskaźnik na ciąg znaków do przekonwertowania,
- *base* – podstawa systemu liczbowego, w którym zapisana jest liczba (podanie liczby *0* powoduje automatyczne dobranie formatu np. *hex*)
- **res* - wskaźnik na zmienną,w której ma być umieszczony wynik konwersji.

Funkcja zwraca wartość *0* w przypadku poprawnego wykonania, lub ujemny kod błędu jeśli konwersja się nie powiedzie.

- sprawdzić czy funkcja zwróciła wartość niezerową, w przypadku wystąpienia błędu wyświetlić odpowiednią informację w buforze diagnostycznym za pomocą makra *pr_err* i zwrócić wartość zmiennej *err.*

3. Dodać obsługę błędów do funkcji *my_attr_show.* Wyświetlić komunikat o błędzie w buforze diagnostycznym i zwrócić kod błędu (uwaga: funkcja *sprintf* zwraca wartości dodatnie w przypadku powodzenia i ujemne jeśli wystąpi błąd).
4. Zamieścić kod funkcji *my_attr_store*i *my_attr_show* poniżej.

```
static ssize_t my_attr_show(struct device *dev,
                            struct device_attribute *attr,
                            char *buf)
{
        int err = 0;
        err = sprintf(buf, "%d\n", my_attr);
        if (err <= 0)
                pr_err("Number to string conversion failed.\n");
        return err;
}

static ssize_t my_attr_store(struct device *dev,
                             struct device_attribute *attr,
                             const char *buf, size_t count)
{
        int err = 0;
        err = kstrtol(buf, 0, (long int*) &my_attr);
        if (err) {
                pr_err("String to number conversion failed.\n");
                return err;
        }
        return count;
}
```

5. Przetestować działanie obsługi błędów wpisując do atrybutu ciąg znaków *„abc”*, a wynik i informację z bufora diagnostycznego zamieścić poniżej.

```
pi@pi:~ $ sudo sh -c "echo abc >>
/sys/devices/virtual/my_class/my_device/my_attr"
sh: 1: echo: echo: I/O error

pi@pi:~ $ dmesg
[42379.642705] Module init
[42398.152543] String to number conversion failed.
```

6. Dodać obsługę błędów do funkcji inicjalizującej moduł. W tym celu:
   1. Utworzyć zmienną *int err = 0*.
   2. Utworzyć następujące etykiety i dodać pod nimi odpowiednie instrukcje:
      1. *out_device_destroy* – usunąć urządzenie,
      2. *out_class_destroy* – usunąć klasę urządzenie,
      3. *out_ret_err* – zwrócić wartość zmiennej *err*.
   3. Sprawdzić za pomocą funkcji *IS_ERR* czy wskaźnik *my_class* nie oznacza błędu. Jeśli tak, to przypisać kod błędu do zmiennej *err* (funkcja *PTR_ERR*), zapisać informację do bufora diagnostycznego i wykonać skok do właściwej etykiety.
   4. Obsłużyć błąd tworzenia urządzenia (*my_device*) analogicznie jak w punkcie 6.3.

5. Obsłużyć błąd tworzenia pliku atrybutu, jak w punktach 6.3 i 6.4 (uwaga: funkcja *device_create_file* zwraca kod błędu normalnie). Po bezbłędnym wykonaniu instrukcji wykonać skok do etykiety *out_ret_err*.
6. Zamieścić kod funkcji inicjalizującej moduł poniżej.

```
static int __init on_init(void)
{
        int err = 0;

        pr_info("Module init\n");

        my_class = class_create(THIS_MODULE, "my_class");
        if (IS_ERR(my_class)) {
                pr_err("Class creation failed.\n");
                err = PTR_ERR(my_class);
                goto out_ret_err;
        }

        my_device = device_create(my_class, NULL, 0, NULL,
                                  "my_device");
        if (IS_ERR(my_device)) {
                pr_err("Device creation failed.\n");
                err = PTR_ERR(my_device);
                goto out_class_destroy;
        }

        err = device_create_file(my_device,
                                 &dev_attr_my_attr);
        if (err) {
                pr_err("Device creation failed.\n");
                goto out_device_destroy;
        }

        goto out_ret_err;

out_device_destroy:
        device_destroy(my_class, 0);
out_class_destroy:
        class_destroy(my_class);
out_ret_err:
        return err;
}
```

Urządzenie znakowe

Wprowadzenie

W systemie Linux istnieje kilka typów urządzeń, z których dwa główne to urządzenia znakowe i urządzenia blokowe.
Urządzenia znakowe przesyłają dane w sposób strumieniowy, bajt po bajcie, bez buforowania i bez opóźnień. Są prostsze w implementacji, ponieważ często nie wymagają dodatkowego zarządzania blokami danych czy implementacji mechanizmu obsługi cache. Tego rodzaju urządzenia są wykorzystywane do operacji, które nie wymagają dużej przepustowości. Przykłady urządzeń znakowych to terminale, porty szeregowe, urządzenia audio.
Urządzenia blokowe przesyłają dane w większych niż kilka KB blokach, co pozwala na buforowanie i bardziej efektywne zarządzanie danymi. Są wykorzystywane głównie do przechowywania i odczytu dużych ilości danych. Przykłady urządzeń znakowych to dyski twarde, dyski SSD, napędy USB.
Niniejsza instrukcja przedstawia przykłady użycia urządzenia znakowego.

Inicjalizacja i usuwanie urządzenia znakowego

Inicjalizacja urządzenia znakowego polega na jego utworzeniu i konfiguracji, oraz zarejestrowaniu pliku urządzenia w systemie. Należy pamiętać o konieczności zapewnienia obsługi błędów, które mogą wystąpić podczas operacji związanych z ww. operacjami.

Numery major i minor

Każde urządzenie znakowe i blokowe jest reprezentowane w systemie za pomocą specjalnych numerów *major* i *minor:*

- *major* – Reprezentuje sterownik urządzenia w systemie i nie może się powtarzać.
- *minor* – Służy do identyfikacji konkretnej instancji urządzania w sterowniku. Nie może powtarzać się w obrębie sterownika, ale w systemie już tak.

Numery te służą do identyfikacji urządzenia i sterownika w systemie.
Jądro Linux umożliwia dynamiczne pozyskanie dostępnego numeru *major*, bez konieczności ręcznego sprawdzania czy jest już zajęty, za pomocą funkcji *alloc_chrdev_region,* której deklarację przedstawiono poniżej. [3]

```
int alloc_chrdev_region(dev_t *dev, unsigned baseminor,
                        unsigned count,
                          const char *name);
```

Argumenty funckji:

- **dev* – adres zmiennej typu *dev_t* (podać *&my_devt*),
- *baseminor* – pierwszy numer *minor* (podać *0),*

- *count* – ilość numerów *minor* do zarejestrowania (podać *1),*
- **name* – nazwa urządzenia (podać *„my_cdev”*).

Funkcja zwraca wartość *0* w przypadku poprawnego wykonania, w przeciwnym wypadku zwraca ujemny kod błędu.

Po uzyskaniu numerów *major* i *minor*, należy zainicjalizować urządzenie znakowe za pomocą funkcji *cdev_init*, której deklarację podano poniżej. [3]

```
void cdev_init(struct cdev *cdev,
               const struct file_operations *fops);
```

Argumenty funckji:

- **cdev* – wskaźnik na strukturę urządzenia znakowego *cdev* (podać *&my_cdev*),
- *fops – struktura typu file_operations zawierająca adresy callbacków obsługujących operacje wejścia/wyjścia (open, release, read, write itd.) (podać NULL).

Po zainicjalizowaniu urządzenia, trzeba dodać je do systemu za pomocą funkcji *cdev_add*, której deklarację podano poniżej. [3]

```
int cdev_add(struct cdev *p, dev_t dev, unsigned count);
```

Argumenty funckji:

- **p* – wskaźnik na strukturę urządzenia znakowego *cdev* (podać *&my_cdev*),
- *dev* – numer identyfikujący urządzenie znakowe (podać *my_devt*),
- *count* – liczba numerów *minor* skojarzonych z tym urządzeniem (podać *1*).

Funkcja zwraca wartość 0 w przypadku poprawnego wykonania, w przeciwnym wypadku zwraca ujemny kod błędu.

Podczas usuwania modułu z jądra, należy usunąć wcześniej utworzone urządzenie znakowe i zwolnić pozyskane numery *major* i *minor.* Służą do tego funkcje *cdev_del* i *unregister_chrdev_region.* Deklaracje obu funkcji przedstawiono poniżej. [3]

```
void cdev_del(struct cdev *p);

void unregister_chrdev_region(dev_t from, unsigned count);
```

Argumenty funckji:

- **p* – wskaźnik na strukturę urządzenia znakowego *cdev* (podać *&my_cdev*),
- *from* – początkowy numer identyfikujący urządzenie znakowe (podać *my_devt*),
- count – liczba numerów minor skojarzonych z tym urządzeniem (podać 1).

Zadanie – tworzenie urządzenia znakowego

1. Zapisać kod modułu *errors* pod nazwą *char_dev.c* i usunąć część związaną z atrybutami urządzenia.
2. Dołączyć plik nagłówkowy *linux/fs.h*, który udostępnia mechanizmy niezbędne do obsługi operacji na plikach.

3. Utworzyć zmienną globalną identyfikatora urządzenia *static dev_t my_devt*. Jest to zmienna typu *dev_t*, która przechowuje wartość numeru *major* i *minor.*
4. Zarejestrować numery urządzenia znakowego w systemie.
   Do rejestracji i pozyskania numerów identyfikujących wykorzystać funkcję *alloc_chrdev_region.*
5. Utworzyć globalną zmienną *my_cdev* typu *static struct cdev* i zainicjalizować ją za pomocą funkcji *cdev_init*.
6. Dodać urządzenie znakowe do kernela. W tym celu wykorzystać funkcję *cdev_add*, której deklarację przedstawiono poniżej.
7. W funkcji *device_create* jako argument *devt* podać zmienną *my_devt (*zamiast *0).*
8. Dodać do kodu deinicjalizującji modułu, wyrejestrowanie urządzenia znakowego z systemu. W tym celu w odpowiedniej kolejności użyć następujących funkcji: *cdev_del* i *unregister_chrdev_region*, których deklaracje przedstawiono poniżej. Dodatkowo podać jako argument *devt* funkcji *device_destroy* zmienną *my_devt* (zamiast *0*).
9. Skompilować i załadować moduł *char_dev* do jądra.
   Kod funkcji inicjalizującej i deinicjalizującej moduł zamieścić poniżej.

```
static int __init on_init(void)
{
        int err = 0;

        pr_info("Module init.\n");

        err = alloc_chrdev_region(&my_devt, 0, 1,
                                  "my_device");
        if (err) {
                pr_err("Can't allocate chrdev.\n");
                goto out_ret_err;
        }

        my_class = class_create(THIS_MODULE, "my_class");
        if (IS_ERR(my_class)) {
                pr_err("Class creation failed.\n");
                err = PTR_ERR(my_class);
                goto out_deregister_chrdev_region;
        }

        cdev_init(&my_cdev, NULL);

        err = cdev_add(&my_cdev, my_devt, 1);
        if (err) {
                pr_err("Character device add failed.\n");
                goto out_class_destroy;
        }

        my_device = device_create(my_class, NULL, my_devt,
                                  NULL, "my_device");
        if (IS_ERR(my_device)) {
```

```
                pr_err("Device creation failed.\n");
                err = PTR_ERR(my_device);
                goto out_cdev_delete;
        }

        goto out_ret_err;

out_cdev_delete:
        cdev_del(&my_cdev);
out_class_destroy:
        class_destroy(my_class);
out_deregister_chrdev_region:
        unregister_chrdev_region(my_devt, 1);
out_ret_err:
        return err;
}

static void __exit on_exit(void)
{
        pr_info("Module exit.\n");
        device_destroy(my_class, 0);
        cdev_del(&my_cdev);
        unregister_chrdev_region(my_devt, 1);
        class_destroy(my_class);
}
```

10. Sprawdzić, czy w katalogu */dev* pojawił się plik urządzenia znakowego (polecenie *ls -l | grep my_device*), wynik zamieścić poniżej, a na jego podstawie odpowiedzieć na pytania:
    - Co oznacza litera *c* w pierwszej kolumnie wyniku?
    - Jaki numer *major* i *minor* został przydzielony nowemu urządzeniu?
    - Jaki typ urządzenia oznacza ten numer *major*?

```
pi@pi:~ $ ls -l /dev | grep my_device
crw------- 1 root root    239,   0 Jan  1 18:14 my_device
```

Litera *c* w pierwszej kolumnie wyniku oznacza, że jest to urządzenie znakowe (ang. character).

Przydzielony numer *major* to: 239. Ten numer należy do puli numerów przydzielanych dynamicznie przez jądro, dla urządzeń znakowych.

Przydzielony numer minor to: 0.

11. Odczytać plik urządzenia poleceniem *head -n1*[1] */dev/my_device*, a wynik umieścić poniżej.

```
pi@pi:~ $ sudo head -n1 /dev/my_device
head: cannot open '/dev/my_device' for reading: No such device or
address
```

[1] Polecenie *head z dodatkową flagą -n1* wyświetla jedną linijkę wskazanego pliku, zamiast domyślnych 10 linijek.

Nie można otworzyć pliku, ponieważ nie zaimplementowano jeszcze obsługi operacji otwierania i zamykania.

Operacje na pliku urządzenia znakowego

Operacje na pliku urządzenia znakowego w systemie Linux pozwalają użytkownikom i aplikacjom komunikować się z urządzeniem.
Poniżej przedstawiono przykład komunikacji ze sterownikiem. Wpisanie danych do pliku urządzenia powoduje ustawienie podanej wartości napięcia na wyjściu przetwornika cyfra-analog MCP4921.
Komenda *sh* pozwala wykonać ciąg kilku poleceń (*echo* i *przekierowanie strumienia danych*) z uprawnieniami przydzielonymi przez polecenie *sudo*. Polecenie *sudo echo » ..* wywoła z uprawnieniami administratora jedynie komendę *echo* i wpisanie danych do pliku się nie powiedzie.

```
pi@pi:~ $ sudo sh -c "echo 1000 >> /dev/mcp4921"
```

Komunikację ze sterownikiem w jądrze zapewniają wywołania systemowe (ang. *system calls*).
Jest to mechanizm, który umożliwia programom działającym w przestrzeni użytkownika komunikację z jądrem systemu operacyjnego w celu wykonywania uprzywilejowanych operacji. Przykłady takich wywołań to:

- otwarcie pliku lub urządzenia, przygotowuje go do operacji odczytu, zapisu lub obu naraz (*open),*
- zamknięcie pliku (*close*),
- przesłanie danych z przestrzeni użytkownika do urządzenia (*write*)
- pobranie danych do buforaw przestrzeni użytkownia z urządzenia (*read*)

W celu przygotowania pliku do odczytu lub zapisu (np. pozyskanie dostępu do prywatnych danych pliku) należy utworzyć definicje funkcji *open* i *release.* Przykład ich definicji przedstawiono poniżej.

```
static int device_open(struct inode *inode,
                       struct file *filp)
{
        /* device file open */
}

static int device_release(struct inode *inode,
                          struct file *filp)
{
        /* device file open */
}
```

Funkcje powinny zwrócić ilość odczytanych lub zapisanych bajtów w przypadku poprawnego wykonania, lub ujemny kod błędu jeśli on wystąpi.

Obsługę funkcjonalności zapisu i odczytu pliku urządzenia znakowego definiuje się w funckjach *read* i *write*. Poniżej przedstawiono ich przykładowe definicje.

```
static ssize_t device_read(struct file *filp,
                           char __user *buf,
                           size_t count, loff_t *f_pos)
{
        /* device file read */
}
static ssize_t device_write(struct file *filp,
                            const char __user *buf,
                            size_t count, loff_t *f_pos)
{
        /* device file write */
}
```

Wskaźnik **buf* zawiera adres bufora na dane, który służy do wymiany danych pomiędzy przestrzenią jądra i użytkownika.
Funkcje powinny zwrócić ilość odczytanych lub zapisanych bajtów w przypadku poprawnego wykonania, lub ujemny kod błędu jeśli on wystąpi.

Aby umożliwić komunikację ze sterownikiem za pomocą wywołań systemowych, należy powiązać funkcje obsługujące operacje na plikach z odpowiednimi wywołaniami. Służy do tego struktura *file_operations*, której przykład przedstawiono poniżej. Dzięki temu podczas np. wpisywania do pliku urządzenia, wywołana zostanie funkcja zdefiniowana w sterowniku, która obsłuży dane.

```
static struct file_operations fops = {
        .owner = THIS_MODULE,
        .open = device_open,        //przypisanie funkcji open
        .release = device_release   //przypisanie funkcji release
};
```

Zadanie – obsługa otwierania i zamykania pliku urządzenia

1. Zapisać kod modułu *char_dev* pod nazwą *char_dev_open_close.c*.
2. Dodać definicje funkcji *device_open* i *device_release* zgodnie z zamieszczonymi poniżej deklaracjami.
   Funkcje powinny wpisać do bufora diagnostycznego informację o tym, że zostały uruchomione:
   - "my_device file open." w funkcji *device_open,*
   - "my_device file close." w funkcji *device_release.*
3. Dodać globalną strukturę *fops* typu *file_operations* i zainicjalizować ją jak przedstawiono poniżej. Dodatkowo przekazać jej adres jako argument **fops* funkcji *cdev_init (*zamiast *Null),*

4. Skompilować i załadować moduł *char_dev_open_close* do jądra.
5. Przetestować działanie funkcji poleceniem *head /dev/my_device*, a zawartość bufora diagnostycznego zamieścić poniżej.

```
pi@pi:~ $ dmesg
[...]
[  203.524727] my_device file open.
[  203.530422] my_device file close.
```

6. Zamieścić kod funkcji *device_open* i *device_close* poniżej.

```
static int device_open(struct inode *inode, struct file *filp)
{
        pr_info("my_device file open.\n");
        return 0;
}

static int device_release(struct inode *inode,
                          struct file *filp)
{
        pr_info("my_device file close.\n");
        return 0;
}
```

Zadanie – Obsługa zapisu i odczytu danych z pliku urządzenia

W tym zadaniu zostanie dodana tylko obsługa funkcji *write* i *read*, które spowodują wpisanie do bufora diagnostycznego informacji o swoim wykonaniu.

1. Zapisać kod modułu *char_dev_open_close* pod nazwą *char_dev_read_write.c*.
2. Dodać definicje funkcji *device_read* i *device_write*, zgodnie deklaracjami zamieszczonymi poniżej. Podczas wpisywania i odczytywania z pliku, w buforze diagnostycznym powinna znaleźć się informacja o wykonanej czynności, tak jak przy otwarciu i zamknięciu pliku.
3. Dodać do inicjalizacji zmiennej *fops* funkcje odpowiedzialne za zapis i odczyt pliku. Definicję struktury *fops* z zaznaczonymi polami odpowiednich wskaźników przedstawiono poniżej.

```
static struct file_operations fops = {
        .owner = THIS_MODULE,
        .open = device_open,
        .release = device_release,
        .read = device_read,
        .write = device_write
};
```

4. Skompilować moduł i załadować go do jądra.

5. Poleceniem *sudo sh -c "echo 123 >> /dev/my_device"* zapisać ciąg znaków do pliku urządzenia.
6. Poleceniem *sudo head -n1 /dev/my_device* odczytać uprzednio wpisaną zawartość z pliku urządzenia.
7. Usunąć moduł z jądra i zamieścić wydruk z bufora diagnostycznego przedstawiający serię wykonanych operacji poniżej.

```
pi@pi:~ $ dmesg
[ 2371.514098] Module init.
[ 2388.421309] my_device file open.
[ 2388.421423] my_device write to file.
[ 2388.421452] my_device file close.
[ 2401.431217] my_device file open.
[ 2401.431292] my_device read from file.
[ 2401.434309] my_device file close.
[ 2413.795448] Module exit.
```

8. Zamieścić kod funkcji *device_read* i *device_write* poniżej.

```
static ssize_t device_read(struct file *filp,
                           char __user *buf,
                           size_t count, loff_t * f_pos)
{
        pr_info("my_device read from file.\n");
        return count;
}

static ssize_t device_write(struct file *filp,
                            const char __user *buf,
                            size_t count, loff_t *f_pos)
{
        pr_info("my_device write to file.\n");
        return count;
}
```

Wymiana danych z urządzeniem znakowym

Moduły działające w przestrzeni jądra oraz programy w przestrzeni użytkownika korzystają z odrębnych przestrzeni adresowych. Aby umożliwić bezpieczną wymianę danych między przestrzenią użytkownika a przestrzenią jądra, stosuje się dedykowane funkcje jądra. Funkcja *copy_to_user* służy do przesyłania danych z przestrzeni jądra do przestrzeni użytkownika. Natomiast funkcja *copy_from_user* umożliwia przenoszenie danych z przestrzeni użytkownika do przestrzeni jądra. Obie funkcje weryfikują poprawność adresów i prawa dostępu do danych.
Poniżej przedstawiono ich deklaracje. [3]

```
static __always_inline unsigned long __must_check
copy_from_user(void *to, const void __user *from,
               unsigned long n);
```

Argumenty funkcji:

- **to* – adres docelowy w przestrzeni jądra, pod który mają zostać przekopiowane dane,
- **from* – adres źródłowy w przestrzeni użytkownika, z którego mają zostać przekopiowane dane,
- *n* – ilość bajtów do przekopiowania.

Funkcja zwraca liczbę bajtów, których nie udało się przekopiować. W przypadku powodzenia, liczba ta powinna być równa *0*.

```
static __always_inline unsigned long __must_check
copy_to_user(void __user *to, const void *from,
             unsigned long n);
```

Argumenty funkcji:

- **to* – adres docelowy w przestrzeni użytkownika, pod który mają zostać przekopiowane dane,
- **from* – adres źródłowy w przestrzeni jądra, z którego mają zostać przekopiowane dane,
- *n* – ilość bajtów do przekopiowania.

Funkcja zwraca liczbę bajtów, których nie udało się przekopiować. W przypadku powodzenia, liczba ta powinna być równa *0*.

Zadanie – komunikacja z sterownika z przestrzenią użytkownika

1. W module *char_dev_read_write* dodać tablicę do przechowywania wpisywanych danych. Tablica powinna być zadeklarowana jako *static char device_buffer[64]*.
2. W funkcji *device_write* przekopiować dane spod adresu *buf* (przestrzeń użytkownika) do tablicy *device_buffer* (przestrzeń jądra) za pomocą funkcji *copy_from_user*.
    - W przypadku niepowodzenia przy kopiowaniu danych zwrócić kod błędu *-EFAULT*.
    - Jeśli kopiowanie danych powiedzie się, funkcja powinna zwrócić zmienną *count*, dzięki temu przestrzeń użytkownika zostanie poinformowana, że przekazano wszystkie znaki.
3. W funkcji *device_read* przekopiować dane z tablicy *device_buffer* do przestrzeni użytkownika za pomocą funkcji *copy_to_user*.
    - Jako argument *n* funkcji *copy_to_user* podać wartość zwróconą przez funkcję *strlen(device_buffer)*.
    - Obsługę błędów zrealizować podobnie jak w punkcie 2.
4. Skompilować i załadować moduł do jądra.
5. Poleceniem *echo* zapisać do pliku *„my_device buffer is not empty.”*.

6. Poleceniem *head* odczytać plik urządzenia i zamieścić wynik poniżej.

```
pi@pi:~ $ sudo sh -c "echo my_device buffer is not empty. >>
/dev/my_device"
pi@pi:~ $ sudo head -n1 /dev/my_device
my_device buffer is not empty.
```

7. Zamieścić kod funkcji *device_write* i *device_read* wraz z deklaracją tablicy *device_buffer* poniżej.

```
static char device_buffer[64];

static ssize_t device_write(struct file *filp,
                            const char __user *buf,
                            size_t count, loff_t *f_pos)
{
        int err = 0;

        pr_info("my_device write to file.\n");

        err = copy_from_user(device_buffer, buf, count);
        if (err) {
                pr_err("Copying data from userspace failed.\n");
                return -EFAULT;
        }
        return count;
}


static ssize_t device_read(struct file *filp, char __user *buf,
                           size_t count, loff_t *f_pos)
{
        int err = 0;

        pr_info("my_device read from file.\n");

        err = copy_to_user(buf, device_buffer,
                           strlen(device_buffer));
        if (err) {
                pr_err("Copying data to userspace failed.\n");
                return -EFAULT;
        }
        return count;
}
```

8. Dodać kompletny kod modułu poniżej.

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>
```

```
static char device_buffer[64];

static int device_open(struct inode *inode, struct file *filp)
{
        pr_info("my_device file open.\n");
        return 0;
}

static int device_release(struct inode *inode, struct file *filp)
{
        pr_info("my_device file close.\n");
        return 0;
}

static ssize_t device_write(struct file *filp,
                            const char __user *buf,
                            size_t count, loff_t *f_pos)
{
        int err = 0;

        pr_info("my_device write to file.\n");

        err = copy_from_user(device_buffer, buf, count);
        if (err) {
                pr_err("Copying data from userspace failed.\n");
                return -EFAULT;
        }
        return count;
}

static ssize_t device_read(struct file *filp, char __user *buf,
                           size_t count, loff_t *f_pos)
{
        int err = 0;

        pr_info("my_device read from file.\n");

        err = copy_to_user(buf, device_buffer,
                           strlen(device_buffer));
        if (err) {
                pr_err("Copying data to userspace failed.\n");
                return -EFAULT;
        }
        return count;
}

static struct file_operations fops = {
        .owner = THIS_MODULE,
        .open = device_open,
        .release = device_release,
        .read = device_read,
        .write = device_write
};
```

```
static int __init on_init(void)
{
        int err = 0;

        pr_info("Module init.\n");

        err = alloc_chrdev_region(&my_devt, 0, 1, "my_device");
        if (err) {
                pr_err("Can't allocate chrdev.\n");
                goto out_ret_err;
        }

        my_class = class_create(THIS_MODULE, "my_class");
        if (IS_ERR(my_class)) {
                pr_err("Class creation failed.\n");
                err = PTR_ERR(my_class);
                goto out_deregister_chrdev_region;
        }

        cdev_init(&my_cdev, NULL);

        err = cdev_add(&my_cdev, my_devt, 1);
        if (err) {
                pr_err("Character device add failed.\n");
                goto out_class_destroy;
        }

        my_device = device_create(my_class, NULL, my_devt, NULL,
                                  "my_device");
        if (IS_ERR(my_device)) {
                pr_err("Device creation failed.\n");
                err = PTR_ERR(my_device);
                goto out_cdev_delete;
        }

        goto out_ret_err;

out_cdev_delete:
        cdev_del(&my_cdev);
out_class_destroy:
        class_destroy(my_class);
out_deregister_chrdev_region:
        unregister_chrdev_region(my_devt, 1);
out_ret_err:
        return err;
}

static void __exit on_exit(void)
{
        pr_info("Module exit.\n");
        device_destroy(my_class, 0);
        cdev_del(&my_cdev);
        unregister_chrdev_region(my_devt, 1);
        class_destroy(my_class);
}
```

```
module_init(on_init);
module_exit(on_exit);

MODULE_LICENSE("GPL v2");
```

Urządzenie miscellaneous device

Wprowadzenie

W celu uproszczenia obsługi mniej skomplikowanych sterowników, w systemi Linux istnieją specjalne urządzenia znakowe typu *misc* (ang. miscellaneous device). Ta kategoria urządzeń pozwala m.in. uniknąć konieczności zajmowania się zarządzaniem numerami *major* i *minor*. Wszystkie urządzenia *misc* korzystają z tego samego numeru major, który jest przypisany specjalnie dla nich (zwykle *10*). Każde urządzenie *misc* ma unikalny numer minor, który pozwala odróżnić je od innych urządzeń w tej kategorii.
Innym ułatwieniem jest automatyczna inicjalizacja urządzenia i klasy oraz utworzenie pliku urządzenia w katalogu */dev*, co znacząco upraszcza kod modułu.

Konfiguracja *miscdevice* wymaga utworzenia i zarejestrowania specjalnej struktury typu *miscdevice*, której przykład przedstawiono poniżej.

```
static struct miscdevice my_miscdevice = {
        .minor = MISC_DYNAMIC_MINOR,
        .name = "my_miscdevice",
        .fops = &fops,
};
```

Znaczenie pól struktury *miscdevice*:

- *.minor* – numer *minor* urządzenia *misc, MISC_DYNAMIC_MINOR* to makro pozwalające na automatyczne przydzielenie wolnego numeru przez jądro,
- *.name* – nazwa urządzenia,
- *.fops* – wskaźnik na strukturę z funkcjami obsługującymi operacje na plikach.

Do rejestracji i usuwania *miscdevice* wykorzystuje się funkcje *misc_register* i *misc_deregister*. Poniżej przedstawiono definicje obu funkcji. [3]

```
int misc_register(struct miscdevice *misc);
```

Funkcja jako argument przyjmuje wskaźnik na strukturę opisującą urzadzenie *misc* typu *miscdevice*. W przypadku poprawnego zarejestrowania urządzenia misc zwraca wartość *0*, a w przeciwnym wypadku ujemny kod błędu.

```
void misc_deregister(struct miscdevice *misc);
```

Funkcja jako argument przyjmuje wskaźnik na strukturę opisującą urzadzenie *misc* typu *miscdevice*.

Zadanie – komunikacja za pomocą miscdevice

1. Zapisać kod modułu *char_dev_read_write pod nazwą misc_dev.c*.
2. Usunąć z funkcji inicjalizującej moduł kod związany z:
   - rejestracją numeru urządzenia znakowego (*alloc_chrdev_region*),
   - tworzeniem klasy urządzenia (class_create),
   - inicjalizacją i dodawaniem urządzenia znakowego (*cdev_init* i *cdev_add*),
   - tworzeniem urządzenia i jego pliku (*device_create*).
3. Usunąć z funkcji deinicjalizującej moduł kod związany z:
   - usunięciem urządzenia i jego pliku (*device_destroy*),
   - usunięciem urządzenia znakowego (*cdev_del*),
   - wyrejestrowaniem numeru urządzenia znakowego (*unregister_chrdev_region*),
   - usunięciem klasy urządzenia (*class_destroy*).
4. Usunąć wszystkie globalne wskaźniki i zmienną my_devt.
5. Usunąć funkcje *device_open* i *device_release* oraz stosowne powiązania ze struktury *fops*. Mechanizm *miscdevice* dostarczony przez jądro Linux zapewnia automatyczną obsługę wywołań systemowych *open* i *release*.
6. Dołączyć plik nagłówkowy <*linux/miscdevice.h*>, który dostarcza funkcje odpowiadające za obsługę urządzenia *misc*.
7. Utworzyć globalną zmienną *my_miscdevice* typu *miscdevice* i zainicjalizowć jej pola zgodnie z przykładem podanym w opisie.
8. W funkcji inicjalizującej moduł zarejestrować urządzenie typu *misc*, w tym celu użyć funkcji *misc_register*.
9. W funkcji deinicjalizującej moduł wyrejestrować urządzenie typu *misc*. W tym celu użyć funkcji *misc_deregister*.
10. Skompilować i załadować moduł do jądra.
11. Poleceniem *echo* zapisać do pliku *„my_miscdevice buffer is not empty.”*.
12. Poleceniem *head* odczytać plik urządzenia i zamieścić wynik poniżej.

```
pi@pi:~ $ sudo sh -c "echo my_miscdevice buffer is not empty. >>
/dev/my_miscdevice"
pi@pi:~ $ sudo head -n1 /dev/my_miscdevice
my_miscdevice buffer is not empty.
```

13. Zamieścić kod funkcji inicjalizującej i deinicjalizującej moduł poniżej.

```
static int on_init(void)
{
        int err = 0;
        pr_info("Module init\n");
        err = misc_register(&my_miscdevice);
        if (err)
                pr_err("Misc device registration failed!");
        return err;
```

```
}

static void on_exit(void)
{
        pr_info("Module exit\n");
        misc_deregister(&my_miscdevice);
}
```

14. Dodać kompletny kod modułu poniżej.

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/miscdevice.h>

static char device_buffer[64];

static ssize_t device_read(struct file *filp, char __user *buf,
                           size_t count, loff_t *f_pos)
{
        int err = 0;
        pr_info("my_device read from file.\n");

        err = copy_to_user(buf, device_buffer,
                           strlen(device_buffer));
        if (err) {
                pr_err("Copying data to userspace failed.\n");
                return -EFAULT;
        }

        return count;
}

static ssize_t device_write(struct file *filp,
                            const char __user *buf,
                            size_t count, loff_t *f_pos)
{
        int err = 0;
        pr_info("my_device write to file.\n");

        err = copy_from_user(device_buffer, buf, count);
        if (err) {
                pr_err("Copying data from userspace failed.\n");
                return -EFAULT;
        }
        return count;
}

static struct file_operations fops = {
        .owner = THIS_MODULE,
        .read = device_read,
        .write = device_write
};
```

```
static struct miscdevice my_miscdevice = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = "my_miscdevice",
    .fops = &fops,
};

static int on_init(void)
{
        int err = 0;
        pr_info("Module init\n");

        err = misc_register(&my_miscdevice);
        if (err)
                pr_err("Misc device registration failed!");

        return err;
}

static void on_exit(void)
{
        pr_info("Module exit\n");
        misc_deregister(&my_miscdevice);
}

module_init(on_init);
module_exit(on_exit);

MODULE_LICENSE("GPL v2");
```

Drzewo urządzeń

Wprowadzenie

W systemie Linux dostępny jest mechanizm, który pozwala oddzielić opis „sprzętu”, na którym działa system od opisu działania systemu. Nazywany jest on drzewem urządzeń (ang. device tree) i jest to struktura danych definiującą parametry „sprzętu” dostępnego dla systemu operacyjnego.
Drzewo urządzeń prezentowane jest w systemie plików w katalogu */proc/device-tree.* Zawiera ono pliki i podkatalogi odzwierciedlające strukturę i właściwości drzewa urządzeń, takie jak nazwy węzłów, ich atrybuty i hierarchię sprzętową. Poniżej przedstawiono przykładową zawartość tego katalogu.

```
pi@pi:~ $ ls /proc/device-tree/
'#address-cells'   cam0_clk         cam_dummy_reg   cpus
 leds              name             serial-number   system
 aliases           cam0_regulator   chosen          memory@0
 __overrides__    '#size-cells'     thermal-zones   arm-pmu
 cam1_clk          clocks           memreserve      phy
 soc               axi              cam1_regulator  compatible
 interrupt-parent  model            reserved-memory __symbols__
```

Zadanie – odczyt nazwy platformy sprzętowej z drzewa urządzeń

Odczytać zawartość pliku z nazwą urządzenia */proc/device-tree/model* (użyć polecenia *cat*) Zamieścić wyniki poniżej.

```
pi@pi:~ $ cat /proc/device-tree/model
Raspberry Pi Zero W Rev 1.1
```

Nakładki drzewa urządzeń i kompilacja

Aby móc dynamicznie modyfikować drzewo urządzeń (w trakcie pracy systemu), istnieją nakładki drzewa urządzeń (ang. overlays). Pozwalają one rozszerzać strukturę drzewa w momencie gdy jest ono już załadowane do systemu.
W celu umożliwienia rekonfiguracji urządzeń, bez konieczności modyfikacji i ponownej kompilacji ich sterowników, jądro umożliwia dostarczenie danych konfiguracyjnych za pomocą drzewa urządzeń w postaci właściwości węzłów.
Poniżej przedstawiono zawartość przykładowego pliku *.dts* zawierającego kod nakładki.

Celem nakładki jest dodanie nowego urządzenia o nazwie *mtm_device* do drzewa urządzeń. Urządzenie jest dodawane do korzenia (/) istniejącego drzewa. Nowy węzeł *mtm_device* posiada identyfikator compatible oraz dodatkową właściwość *string_param*.

```
/dts-v1/;
/plugin/;

/ {
        compatible = "brcm,bcm2835";

        fragment@0 {
                target-path = "/";

                __overlay__ {
                        mtm_device {
                                compatible = "mtm";
                                string_param = "mtm device param";
                        };
                };
        };
};
```

Opis składni nakładki: [4]

- */dts-v1/* - Określa wersję formatu pliku Device Tree Source.
- */plugin/* - Informuje kompilator o tym, że plik jest nakładką.
- */* - Reprezentuje korzeń drzewa urządzeń. Wszystkie węzły są zagnieżdżone w tym węźle.
- właściwość *compatible* - jest używana do dopasowania nakładki do odpowiedniego sprzętu.
- *fragment@0* - Definiuje jeden fragment nakładki. Fragmenty są używane do modyfikacji określonych węzłów drzewa urządzeń.
- *Target-path = "/"* - Wskazuje, że nakładka ma zastosowanie do węzła głównego drzewa urządzeń (korzenia /).
- *__overlay__* - Specjalna sekcja, która zawiera nowe węzły lub zmiany do istniejących.
- *mtm_device* - Definiuje nowy węzeł w drzewie urządzeń o nazwie mtm_device.
- *string_param = "mtm device"* - Dodaje właściwość *string_param* z wartością *"mtm device"*.

Proces kompilacji drzewa urządzeń polega na przekształceniu opisów sprzętu zapisanych w plikach tekstowych *.dts* w binarny plik *.dtb*, który jest ładowany podczas startu systemu.
Do kompilacji nakładek służy polecenie przedstawione poniżej.

```
dtc -@ -I dts -O dtb -o [plik wyjściowy] [plik wejściowy]
```

Opis paramtreów polecenia:

- -@ - Włącza obsługę symboli i fragmentów, wymagane do tworzenia nakładek (overlays).
- *-I dts* - Określa format pliku wejściowego jako Device Tree Source (plik źródłowy w formacie tekstowym.
- *-O dtbo* - Określa format pliku wejściowego jako Device Tree Blob (skompilowany plik nakładki)
- *-o [plik wyjściowy]* – ustawia nazwę pliku wyjściowego.

Zadanie – dodanie nakładki drzewa urządzeń

1. Skopiować kod przykładowej nakładki do pliku my_overlay.dts.
2. Skompilować nakładkę za pomocą poniższej komendy, której wynikiem będzie skompilowany plik nakładki *my_overlay.dtbo.*

```
dtc -@ -I dts -O dtb -o my_overlay.dtbo my_overlay.dts
```

3. Załadować nakładkę za pomocą polecenia:

```
sudo dtocerlay my_overlay.dtbo
```

4. Sprawdzić czy w katalogu /proc/device-tree pojawił się węzeł o nazwie mtm_device. Użyć polecenia ls /proc/device-tree | grep mtm_device, a wynik umieścić poniżej.

```
pi@pi:~ $ ls /proc/device-tree | grep mtm_device
mtm_device
```

5. Odczytać zawartość katalogu węzła *mtm_device* i jego właściwości. Wynik umieścić w poniżej.

```
pi@pi:~ $ ls /proc/device-tree/mtm_device/
compatible  name  string_param

pi@pi:~ $ cat /proc/device-tree/mtm_device/string_param
mtm device param

pi@pi:~ $ cat /proc/device-tree/mtm_device/name
mtm_device

pi@pi:~ $ cat /proc/device-tree/mtm_device/compatible
mtm
```

Urządzenie platformowe

Wprowadzenie

Urządzenie platformowe w systemie Linux to sprzęt, który jest bezpośrednio związany z platformą, taką jak mikrokontroler lub system-on-chip (SoC), i nie korzysta ze standardowych magistrali jak np. SPI, I2C, PCI czy USB. Przykładem urządzenia platformowego może być dioda LED podłączona do portu GPIO.

Tworzenie instancji urządzenia platformowego na podstawie drzewa urządzeń

Sterownik może obsługiwać wiele urządzeń (wiele węzłów drzewa urządzeń), z tego powodu zachodzi konieczność dopasowania odpowiedniego sterownika do urządzenia podczas jego dodawania do systemu. Na przykład podczas dodawania do systemu kilku węzłów, które reprezentują diody LED, jądro wyszukuje i dopasowuje do nich właściwy sterownik sterownik.

To dopasowanie polega na wyszukaniu w drzewie urządzeń węzłów z polem *compatible* o zawartości takiej jak pole *compatible* identyfikatora sterownika i utworzenia urządzeń reprezentujących te węzły w jądrze.

Identyfikator sterownika jest przechowywany w tablicy typu *of_device_id* i zawiera pole o nazwie *compatible*. Inicjalizacja i rejestracja takiej tablicy została przedstawiona poniżej.

Tablica *of_device_id* powinna dodatkowo zostać przypisana do sterownika platformowego.

```
static const struct of_device_id mtm_of_id[] = {
        { .compatible = "mtm" },
        { /* sentinel */ }
};
MODULE_DEVICE_TABLE(of, mtm_of_id);
```

Aby mechanizm dopasowujący węzły do sterowników działał poprawnie, tablica identyfikatorów musi zawierać pusty element (tzw. *Sentinel*) na końcu. Dzięki niemu mechanizm dopasowujący rozpoznaje koniec tablicy.

Dopasowanie sterownika do urządzenia

W celu umożliwienia dopasowania sterownika do urządzenia, w module definiuje się funkcje *probe* i *remove*, które są wywoływane w momencie, gdy jądro znajdzie pasujący sterownik i doda urządzenie. Przykładowe definicje obu funkcji przedstawiono poniżej.

```
static int mtm_probe(struct platform_device *dev)
{
        dev_info(&dev->dev, "probed\n");
        return 0;
}

static int mtm_remove(struct platform_device *dev)
{
        dev_info(&dev->dev, "removed\n");
        return 0;
}
```

Jako argument obie funkcje dostarczają wskaźnik na strukturę typu *strucrt platform_device,* która reprezentuje urządzenie platformowe. Obie funkcje powinny zwrócić wartość *0* w przypadku poprawnego wykonania, lub ujemny kod błędu jeśli taki wystąpi.

W momencie dodawania urządzenia, jądro wywołuje funkcję *probe* dla każdego znalezionego węzła. W tej funkcji sterownik może przeprowadzić konfigurację urządzenia (np. Skonfigurować piny GPIO do obsługi diody LED), i przygotować je do pracy (np. Ustawić domyślny stan diod LED).

Analogicznie w momencie usuwania modułu z jądra, wywoływana jest funkcja *remove* dla każdego węzła skojarzonego ze sterownikiem. Odpowiedzialna ona za zwolnienie wszelkich zasobów, które zostały zaalokowane w trakcie działania sterownika.

Obie funkcje muszą zostać zdefiniowane w kodzie sterownika i wskazane przy jego konfiguracji w strukturze typu *platform_driver* opisanej w kolejnym rozdziale.

W powyższym przykładzie została użyta makrodefinicja dev_info, która zapisuje wiadomość do bufora diagnostycznego, dodajc infrmację od jakiego urządzenia ta wiadomość pochodzi. Aby rozróżnić od jakiego sterownika i urzadzenia pochodzą wiadomości zapisane w buforze diagnostycznym, jądro dostarcza makra (np. *dev_info*, *err_info*). Są to makra obsługujące poziomy logów (podobnie jak w przypadku np. *pr_info, pr_err* itd.). Oprócz ciągu znaków, który ma być zapisany w buforze, makra te przyjmują jeszcze wskaźnik na zmienną typu *struct device.*

Rejestracja sterownika platformowego

W celu uproszczenia rejestracji i usuwania sterownika platformowego w momencie ładowania i usuwania modułu z jądra, jądro dostarcza specjalne makro *module_platform_driver,* które zastępuje funkcje *__init* i *__exit* modułu.

Jako argument makro to przyjmuje strukturę typu *platform_driver*, która opisuje sterownik platformowy. Przykład użycia tego makra przedstawiono poniżej. W kodzie inicjalizującym strukturę, do sterownika przypisano funkcje *mtm_probe, mtm_remove* oraz tablicę *mtm_of_id* typu *of_device_id.*

```
static struct platform_driver mtm_driver = {
        .probe = mtm_probe,
        .remove = mtm_remove,
        .driver = {
                .name = "mtm_driver",
                .of_match_table = mtm_of_id,
                .owner = THIS_MODULE,
        },
};
module_platform_driver(mtm_driver);
```

Zadanie – dodawanie i usuwanie urządzenia platformowego

1. Utworzyć plik modułu o nazwie *mtm_dt.c* i dołączyć do niego następujące pliki nagłówkowe:
   - *linux/of.h* – zawiera definicje i funkcje związane z obsługą drzewa urządzeń,
   - *linux/platform_device.h* - zawiera definicje i funkcje związane z obsługą urządzeń platformowych.
2. Dodać tablicę identyfikatorów sterownika typu *static struct of_device_id* o nazwie *mtm_of_id[]* i zainicjalizować pole *.compatible* jej pierwszego elementu wartością *"mtm"*. (jak we wstępie).
3. Zarejestrować tablicę identyfikatorów *mtm_of_id* za pomocą makra *MODULE_DEVICE_TABLE.*
4. Dodać funkcję *mtm_probe* i *mtm_remove* (jak we wstępie).
5. Dodać i zainicjalizować strukturę opisującą sterownik, typu *static struct platform_driver* o nazwie *mtm_driver.*
   Zarejestrować sterownik za pomocą makra *module_platform_driver* (jak we wstępie).
6. Skompilować i załadować moduł *mtm_dt* do jądra (uwaga: nakładka drzewa urządzeń również musi być załadowana, aby funkcje *mtm_probe* i *mtm_remove* zostały wywołane).
7. Usunąć moduł z jądra i zamieścić 2 ostatnie wpisy z bufora diagnostycznego poniżej oraz zaznaczyć nazwę sterownika i urządzenia.

```
pi@pi:~ $ dmesg
[21917.892155] mtm_driver mtm_device: probed
[21920.789271] mtm_driver mtm_device: removed
```

8. Zamieścić kompletny kod sterownika platformowego poniżej.

```
#include <linux/of.h>
#include <linux/platform_device.h>

static int mtm_probe(struct platform_device *dev)
{
```

```
        dev_info(&dev->dev, "probed\n");
        return 0;
}

static int mtm_remove(struct platform_device *dev)
{
        dev_info(&dev->dev, "removed\n");
        return 0;
}

static const struct of_device_id mtm_of_id[] = {
        { .compatible = "mtm" },
        { },
};
MODULE_DEVICE_TABLE(of, mtm_of_id);

static struct platform_driver mtm_driver = {
        .probe = mtm_probe,
        .remove = mtm_remove,
        .driver = {
                .name = "mtm_device",
                .of_match_table = mtm_of_id,
                .owner = THIS_MODULE,
        },
};
module_platform_driver(mtm_driver);

MODULE_LICENSE("GPL v2");
```

Odczyt właściwości węzła z drzewa urządzeń

Dzięki dodaniu właściwości węzłów w drzewie urządzeń możliwa jest rekonfiguracja sterownika bez konieczności ponownej jego kompilacji. Wystarczy jedynie skompilować i ponownie załadować nakładkę drzewa.

W celu odczytania zawartości właściwości węzła, która zawiera liczbę, należy użyć funckji *of_property_read_u32*, której deklarację przedstawiono poniżej. [3]

```
static inline int of_property_read_u32(const struct device_node *np,
                                       const char *propname,
                                       u32 *out_value);
```

Argumenty funkcji:

- **np* – wskaźnik na węzeł urządzenia (podać *dev->dev.of_node)*,
- **propname* – nazwa parametru (podać ”*int_param”*),
- **out_value* – wskaźnik na zmienną, do której ma być zwrócona pobrana wartość parametru (podać *&int_param*).

Funkcja zwraca wartość *0* w przypadku powodzenia, lub ujemny kod błędu, jeśli taki wystąpi.

Zadanie - *dodanie obsługi odczytu właściwości węzła w sterowniku*

1. Zmodyfikować nakładkę drzewa urządzeń, dodając do węzła parametr liczbowy, wtym celu:
    - Usunąć nakładkę drzewa urządzeń z systemu poleceniem:

```
sudo dtoverlay -r [nazwa nakładki]
```

    - W pliku nakładki drzewa urządzeń, dodać do węzła *mtm_device* parametr *int_param = <17>*.
    - Skompilować nakładkę za pomocą polecenia *dtc* i ponownie załadować ją do systemu poleceniem *dtoverlay.*
2. Odczytać zawartość właściwości węzła, w tym celu:
    - Wodać zmienną typu *unsigned int* o nazwie *int_param,*
    - Odczytać z węzła urządzenia parametr *int_param* i zapisać jego wartość do zmiennej *int_param.* W tym celu użyć funkcji *of_property_read_u32.*
    - W przypadku wystąpienia błędu (np. gdy nie znaleziono parametru o podanej nazwie) funkcja *mtm_probe* powinna zwrócić kod błędu i zamieścić informację w buforze diagnostycznym za pomocą makra *dev_err*, jak poniżej.

```
dev_err(&dev->dev, "Could not get param from dt node.\n");
```

    - Wpisać do bufora diagnostycznego wartość zmiennej *int_param*, jak przedstawiono poniżej.

```
dev_info(&dev->dev, "int_param: %d\n", int_param);
```

3. Skompilować i przetestować moduł, w tym celu:
    - Skompilować i załadować moduł do jądra.
    - Usunąć moduł z jądra i zamieścić ostatnie 3 wpisy z bufora diagnostycznego poniżej.

```
pi@pi:~ $ dmesg
[28634.992992] mtm_driver mtm_device: int_param: 17
[28634.993045] mtm_driver mtm_device: probed
[28636.319175] mtm_driver mtm_device: removed
```

    - Zamieścić kod funckji *mtm_probe* poniżej.

```
static int mtm_probe(struct platform_device *dev)
{
        int err = 0;
        unsigned int int_param;

        err = of_property_read_u32(dev->dev.of_node,
                                   "int_param", &int_param);
        if (err) {
                dev_err(&dev->dev, "Could not get param
                                    from dt node.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "int_param: %d\n", int_param);
        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}
```

Problem dostępu do globalnej zmiennej modułu przez wiele instancji urządzenia

Jeden sterownik urządzenia może obsługiwać wiele urządzeń tego samego rodzaju (klasy) jednocześnie. Kiedy system operacyjny dopasuje węzeł drzewa urządzeń do danego sterownika, automatycznie wywołuje funkcję probe, która została zdefiniowana w kodzie sterownika. Problem pojawia się, ponieważ zarówno funkcja probe, jak i pozostały kod sterownika są wspólne dla wszystkich obsługiwanych urządzeń.

Przykładem takiej sytuacji są dwie diody LED, mrugające z różnymi częstotliwościami. Sterownik musi przechowywać informację o częstotliwości i stanie diod LED. Wykorzystanie w tym celu zmiennych globalnych prowadzi do konfliktu – obie diody korzystające z tych samych zmiennych będą mrugały z tą samą częstotliwością, a stan jednej z nich będzie nadpisywany przez drugą.

Zadanie - przedstawienie problemu z wieloma instancjami urządzenia

1. Zapisać kod modułu *mtm_dt_param* pod nazwą *mtm_dt_multi.c*.
2. Utworzyć nową zmienną globalną typu *unsigned int* o nazwie *int_global_param,* do której urządzenia będą zapisywać dane.
3. Podczas dodawania urządzenia do systemu (funckja *probe*) odczytać właściwość z węzła urządzenia i zapisać w globalnej zmiennej. W tym celu*:*
   - usunąć zmienną lokalną *int_param,*
   - w funkcji *of_property_read_u32* zmienić *&int_param* na *&int_global_param,*
   - w funkcji *dev_info* zmienić *int_param* na *int_global_param.*

4. W momencie usuwania urządzenia z systemu (funkcja *remove*) zapisać do bufora diagnostycznego informację o wartości zmiennej globalnej za pomocą poniższego kodu.

```
dev_info(&dev->dev, "int_param: %d\n", int_global_param);
```

5. W kodzie nakładki drzewa urządzeń, umieścić 3 węzły przedstawione poniżej:

```
mtm_device_1 {
      compatible = "mtm";
      int_param = <111>;
};

mtm_device_2 {
      compatible = "mtm";
      int_param = <222>;
};

mtm_device_3 {
      compatible = "mtm";
      int_param = <333>;
};
```

6. Skompilować moduł i nakładkę drzewa urządzeń oraz załadować je do jądra.
7. Usunąć moduł z jądra i zamieścić poniżej wiadomości zamieszczone w buforze diagnostycznym. Zaznaczyć miejsca, w które świadczą o nadpisywaniu wartości zmiennej globalnej przez kolejne instancję urządzenia. Wytłumacz co się stało.

```
pi@pi:~ $ dmesg
[40758.241034] mtm_driver mtm_device_1: int_global_param: 111
[40758.241092] mtm_driver mtm_device_1: probed
[40758.241804] mtm_driver mtm_device_2: int_global_param: 222
[40758.241851] mtm_driver mtm_device_2: probed
[40758.242512] mtm_driver mtm_device_3: int_global_param: 333
[40758.242558] mtm_driver mtm_device_3: probed
[54509.262187] mtm_driver mtm_device_3: int_param: 333
[54509.262233] mtm_driver mtm_device_3: removed
[54509.271946] mtm_driver mtm_device_2: int_param: 333
[54509.271993] mtm_driver mtm_device_2: removed
[54509.272168] mtm_driver mtm_device_1: int_param: 333
[54509.272193] mtm_driver mtm_device_1: removed
```

Urządzenie *mtm_device_3* jako ostatnie nadpisało wartość globalnej zmiennej, z tego powodu podczas usuwania urządzeń, zapisały one do bufora diagnostycznego błędną wartość.

8. Zamieścić kod funkcji *mtm_probe, mtm_remove* i deklarację zmiennej *int_global_param* poniżej.

```
Static unsigned int int_global_param;
```

```
static int mtm_probe(struct platform_device *dev)
{
        int err = 0;

        err = of_property_read_u32(dev->dev.of_node, "int_param",
                                   &int_global_param);
        if (err) {
                dev_err(&dev->dev,
                        "Could not get param from dt node.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "int_global_param: %d\n",
                 int_global_param);
        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}

static int mtm_remove(struct platform_device *dev)
{
        dev_info(&dev->dev, "int_global_param: %d\n",
                 int_global_param);
        dev_info(&dev->dev, "removed\n");
        return 0;
}
```

Rozwiązanie problemu dostępu do globalnej zmiennej modułu przez wiele instancji urządzenia – prywatne dane urządzenia.

Aby zapewnić możliwość przechowywania informacji dotyczących poszczególnych instancji urządzenia, struktura *device* (przedstawiona w rodziale „Urządzenia”), która reprezentuje urządzenie, zawiera specjalny wskaźnik o nazwie d*river_data* do którego można zapisać adres prywatnej struktury z danymi urządzenia. Dzięki temu prywatne dane urządzenia dostępne będą poprzez strukturę *device,* która znajduje się wewnątrz struktury *platform_device.* Dostęp do tej struktury umożliwia wskaźnik dostarczony jako argument funkcji *probe* i *remove*.

Aby dodać do sterownika obsługę prywatnych danych urządzenia, należy zadeklarować strukturę z tymi danymi i dynamicznie zaalokować dla niej pamięć w momencie dodawania urządzenia do systemu, natomiast w trakcie usuwania urządzenia, pamięć musi zostać zwolniona.

Jądro Linux udostępnia specjalne funkcje, które pozwalają na automatyczne zwalnianie zasobów w momencie usuwania urządzenia z systemu. Są to funkcje z przedrostkiem *devm_* w nazwie, który oznacza alokację zasobów zarządzaną przez urządzenia (*device-managed*

resource allocation). Przykładem takiej funckji jest *devm_kzalloc,* której deklarację przedstawiono poniżej. [3]

```
static inline void *devm_kzalloc(struct device *dev,
                                 size_t size, gfp_t gfp);
```

Argumenty funkcji:

- *dev - wskaźnik do struktury *struct device*, która reprezentuje urządzenie, z którym pamięć jest powiązana. Dzięki temu zaalokowana pamięć jest zwalniana automatycznie podczas usuwania urządzenia.
- size – rozmiar pamięci do zaalokowania, w bajtach.
- gfp - flagi GFP (Get Free Pages), które określają zasady alokacji pamięci.

Funkcja zwraca wskaźnik na początek zaalokowanej struktury danych jeśli wykona się poprawnie, w przeciwnym przypadku zwraca adres z zakodowanym błędem.

Zadanie – rozwiązanie problemu wielu instancji urządzenia

1. Zapisać kod modułu *mtm_dt_param* pod nazwą *mtm_dt_priv_data.c*.
2. Usunąć globalną zmienną *int_global_param* i dodać definicję poniższej struktury do do kodu modułu.

```
struct mtm_device {
        unsigned int int_param;
};
```

3. W funkcji *mtm_probe:*
   - Dodać wskaźnik typu *struct mtm_device* * o nazwie *mtm_device.*
   - Zaalokować pamięć dla struktury danych za pomocą funkcji *devm_kzalloc,* jako argumenty *podać:*
     - *&dev→dev*,
     - *sizeof(struct mtm_device)*,
     - *GFP_KERNEL* – jest to flaga ozaczająca, że pamięć zostanie zaalokowana w przrstrzeni jądra i jeżeli wystąpi taka konieczność, funkcja może poczekać na dostępne miejsce w pamięći (funkcja blokująca).

     Dodać obsługę błędów (uwaga: funkcja zwraca wskaźnik).
   - Ustawić wskaźnik na prywatne dane urządzenia za pomocą dedykowanej funkcji *dev_set_drvdata*, której deklarację przedstawiono poniżej. [3]

```
static inline void dev_set_drvdata(struct device *dev,
                                   void *data);
```

Argumenty funkcji:

- **dev* – wskaźnik na strukturę urządzenia,
- **data* – wskaźnik na dane, które mają zostać ustawione jako prywatne dane urządzenia.

- W funkcji *of_property_read_u32* zmienić *&int_global_param* na *&mtm_device->int_param*.
- W funkcji *dev_info* zmienić *int_global_param* na *mtm_device->int_param.*

4. W funkcji *mtm_remove:*
   - Uzyskać wskaźnik na prywatne dane urządzenia, w tym celu dodać wskaźnik typu *struct mtm_device* * o nazwie *mtm_device* i ustawić go na adres zwrócony przez funkcję *dev_get_drvdata*, której deklaracja została przedstawiona poniżej. [3]

```
static inline void *dev_get_drvdata(const struct device *dev);
```

Funkcja przyjmuje jako argument wskaźnik na urządzenie i zwraca adres prywatnych danych urządzenia.

   - W funkcji *dev_info* zmienić *int_global_param* na *mtm_device->int_param.*
5. Skompilować i załadować moduł.
6. Usunąć moduł z jądra i zamieścić informacje z bufora diagnostycznego, pochodzące od sterownika, poniżej. Czy urządzenia nadpisały nieswoje zmienne?

```
pi@pi:~ $ dmesg
[53119.633397] mtm_driver mtm_device_1: int_param: 111
[53119.633456] mtm_driver mtm_device_1: probed
[53119.633709] mtm_driver mtm_device_2: int_param: 222
[53119.633748] mtm_driver mtm_device_2: probed
[53119.633956] mtm_driver mtm_device_3: int_param: 333
[53119.633993] mtm_driver mtm_device_3: probed
[53120.631312] mtm_driver mtm_device_3: int_param: 333
[53120.631357] mtm_driver mtm_device_3: removed
[53120.637123] mtm_driver mtm_device_2: int_param: 222
[53120.637267] mtm_driver mtm_device_2: removed
[53120.647571] mtm_driver mtm_device_1: int_param: 111
[53120.647610] mtm_driver mtm_device_1: removed
```

Urządzenia nie nadpisały cudzych zmiennych, zapis i odczyt był wykonany na prywatnej zmiennej każdego urządzenia z osobna.

7. Zamieścić kod modułu *mtm_dt_priv_data* poniżej.

```
#include <linux/of.h>
#include <linux/platform_device.h>

struct mtm_device {
        unsigned int int_param;
};
```

```
static int mtm_probe(struct platform_device *dev)
{
        int err = 0;
        struct mtm_device *mtm_device;

        mtm_device = devm_kzalloc(&dev->dev,
                                  sizeof(struct mtm_device), GFP_KERNEL);
        if (IS_ERR(mtm_device)) {
                dev_err(&dev->dev,
                        "Could not allocate device data\n");
                goto out_ret_err;
        }

        dev_set_drvdata(&dev->dev, mtm_device);

        err = of_property_read_u32(dev->dev.of_node, "int_param",
                                   &mtm_device->int_param);
        if (err) {
                dev_err(&dev->dev,
                        "Could not get param from dt node.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "int_param: %d\n",
                 mtm_device->int_param);
        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}

static int mtm_remove(struct platform_device *dev)
{
        struct mtm_device * mtm_device = dev_get_drvdata(&dev->dev);
        dev_info(&dev->dev, "int_param: %d\n", int_global_param);
        dev_info(&dev->dev, "removed\n");
        return 0;
}

static const struct of_device_id mtm_of_id[] = {
        { .compatible = "mtm" },
        { },
};
MODULE_DEVICE_TABLE(of, mtm_of_id);

static struct platform_driver mtm_driver = {
        .probe = mtm_probe,
        .remove = mtm_remove,
        .driver = {
                .name = "mtm_driver",
                .of_match_table = mtm_of_id,
                .owner = THIS_MODULE,
        },
};
module_platform_driver(mtm_driver);
MODULE_LICENSE("GPL v2");
```

Urządzenie platformowe na przykładzie sterownika diod LED

Wprowadzenie

GPIO (General Purpose Input/Output) w Raspberry Pi Zero W to uniwersalne piny służące do komunikacji z urządzeniami zewnętrznymi, takimi jak czujniki, diody LED czy silniki. Każdy pin GPIO może być skonfigurowany jako wejście lub wyjście.

W systemie Linux sterowanie pinami odbywa się inaczej niż w systemach bare-metal. Sterownik urządzenia nie modyfikuje bezpośrednio rejestrów procesora, które odpowiadają za kontrolę GPIO. Zamiast tego sterownik korzysta z funkcji udostępnianych przez jądro, do komunikacji z kontrolerem GPIO, który jest zdefiniowany w drzewie urządzeń i to on steruje pinami.

Korzystanie z GPIO za pośrednictwem drzewa urządzeń w systemie Linux pozwala na przypisywanie pinów GPIO do sterowników urządzeń podczas pracy systemu. Dzięki temu unika się bezpośredniego definiowania numerów pinów w kodzie, co pozwala na zmianę konfiguracji bez konieczności modyfikacji sterownika.

W poniższej instrukcji zaprezentowany został sposób sterowani pinami GPIO z użyciem sterownika platformowego i aplikacji napisanej w języku Python, na przykładzie obsługi diod LED i przycisków podłączonych do kontrolera Raspberry Pi Zero W.

Aplikacja ilustruje sposób komunikacji ze sterownikiem za pomocą wirtualnego systemu plików *sysfs*. Poniżej przedstawiono interfejs graficzny aplikacji napisanej w języku Python, dzięki której użytkownik może sterować diodami LED podłączonymi do kontrolera Raspberry Pi Zero W za pomocą klawiatury.

Dostęp do GPIO w drzewie urządzeń

W celu dodania pinów GPIO do węzła drzewa urządzeń należy dodać specjalną właściwość, o nazwie *[nazwa pinu]-gpio*. Poniżej przedstawiono przykładowy węzeł nakładki drzewa urządzeń, w którym dodane zostały właściwości reprezentujące piny GPIO o nazwach *led* i *btn*.

```
my_gpio {
        compatible = "mtm,my_gpio";
        led-gpio = <&gpio 25 0x02>;
        btn-gpio = <&gpio 17 0x11>;
};
```

Poniżej przedstawiono fragment kodu, który korzysta z właściwości węzła *my_gpio* aby zapalić diodę LED. Za pomocą funkcji *devm_gpiod_get* pozyskiwany jest deskryptor pinu, a funkcja *gpiod_set_value* ustawia wartość pinu na *1* (stan wysoki na pinie).

```
/* ... */
led = devm_gpiod_get(&dev->dev, "led", GPIOD_ASIS);
gpiod_set_value(led, 1);
/* ... */
```

Dodanie pinu GPIO do urządzenia wymaga dodania w węźle specjalnej właściwości, której wzór przedstawiono poniżej.

```
[nazwa pinu]-gpio = <&gpio [numer pinu] [flagi pinu]>;
```

Opis powyższej konfiguracji:

- [nazwa pinu] – własna nazwa pinu, za pomocą której sterownik będzie mógł identyfikować pin.
- &gpio – etykieta odwołująca się do węzła kontrolera GPIO, który również jest zdefiniowany w drzewie urządzeń.
- [numer pinu] – numer pinu odpowiadający fizycznemu pinowi platformy Raspberry Pi Zero W.
- [flagi pinu] – flagi konfiguracyjne pinu określające dodatkowe właściwości taki jak np. poziom logiczny w stanie aktywnym czy funkcje pull-up/pull-down.

Poniżej przedstawiono opis wszystkich flag dotyczących pinów GPIO. [4]

- Bit 0 – określa poziom logiczny dla stanu aktywnego:
    - *0* – stan wysoki jako aktywny,
    - *1* – stan niski jako aktywny
- Bit 1 – oznacza tryb pracy pinu:
    - *0* – push-pull,
    - *1* – single-ended

- Bit 2 – definiuje typ wyjścia pinu:
    - *0* – open-source,
    - *1* – open-drain
- Bit 3 – określa w jakim stanie powinno być urzymywane wyjście podczas trybu uśpienia:
    - *0* – stan poprzedni
    - *1* – stan resetu
- Bit 4 – decyduje o stanie aktywności rezystora podciągającego (pull-up)
    - *0* – wyłączony,
    - *1* – włączony
- Bit 5 – decyduje o stanie aktywności rezystora ściągającego (pull-down):
    - *0* – wyłączony,
    - *1* – włączony

Aby umożliwić modułowi jądra sterowanie pinami GPIO należy skorzystać z dedykowanych funkcji, które udostępnia jądro systemu Linux. Pozwalają one na pozyskanie deskryptora, który opisuje zasoby pinu GPIO i sterowanie tym pinem.

Aby uzyskać deskryptor pinu GPIO należy skorzystać z funkcji *devm_gpiod_get*, której deklarację przedstawiono poniżej. [3]

```
struct gpio_desc *__must_check devm_gpiod_get(struct device *dev,
                                              const char *con_id,
                                              enum gpiod_flags flags);
```

Argumenty funkcji:

- **dev* – wskaźnik na strukturę urządzenia,
- **con_id* – nazwa pinu GPIO podana w węźle drzewa urządzeń (np. *"led"*),
- *flags* – opcjonalne flagi konfiguracyjne pinu GPIO, wartość *GPIOD_ASIS* (*0*) oznacza, że flagi podane w drzewie urządzeń nie będą nadpisane.

Funkcja zwraca wskaźnik na deskryptor pinu GPIO lub w przypadku niepowodzenia wskaźnik z kodem błędu. Dzięki zastosowaniu tej funkcji, jądro automatycznie zwolni zasoby związane z pinem GPIO w momencie usuwania urządzenia z systemu.

Do ustawionia stanu pinu GPIO służy funkcja *gpiod_set_value*, której deklarację przedstawiono poniżej. [3]

```
void gpiod_set_value(struct gpio_desc *desc, int value);
```

Argumenty funkcji:

- **desc* – wskaźnik na deskryptor pinu GPIO,
- *value* – wartość logiczna, jaka ma zostać ustawiona na pinie.

Do odczytywania stanu pinu GPIO służy funkcja gpiod_get_value, której deklarację przedstawiono poniżej. [3]

```
int gpiod_get_value(const struct gpio_desc *desc);
```

Funkcja jako argument przyjmuje wskaźnik na deskryptor pinu GPIO, którego wartość ma zostać odczytana. W przypadku poprawnego wykonania zwraca wartość logiczną pinu GPIO, w przeciwnym przypadku zawraca ujemny kod błędu.

Zadanie – sterowanie diodą LED z użyciem drzewa urządzeń

1. Utworzyć plik nakładki drzewa urządzeń o nazwie *my_gpio.dts*.
2. W nakładce utworzyć węzeł urządzenia o nazwie *my_gpio* i dodać do niego następujące właściwości:
   - *compatible* o wartości ”mtm,my_gpio”,
   - *led-gpio* oznaczające pin GPIO o numerze *25*, aktywny w stanie wysokim, w trybie wyjścia typu *push-pull*,
   - *btn-gpio* oznaczające pin GPIO o numerze *17*, aktywny w stanie niskim, z aktywnym rezystorem podciągającym (pull-up).
3. Skompilować nakładkę i załadować ją do systemu.
4. Zamieścić kod nakładki poniżej.

```
/dts-v1/;
/plugin/;

/ {
        compatible = "brcm,bcm2835";

        fragment@0 {
                target-path = "/";

                __overlay__ {
                        my_gpio {
                                compatible = "mtm,my_gpio";
                                led-gpio = <&gpio 25 0x02>;
                                btn-gpio = <&gpio 17 0x11>;
                        };
                };
        };
};
```

5. Otworzyć plik modułu jądra o nazwie my_gpio.c, który jest dostępny w załącznikach.
6. Dołączyć plik nagłówkowy *linux/gpio/consumer.h*, który zawiera definicje, pozwalające na zarządzanie i obsługę pinów GPIO.
7. Dodać obsługę prywatnych danych instancji urządzenia, w tym celu:
   - Zdefiniować strukturę *my_gpio* zawierającą:

    - wskaźnik na strukturę typu gpio_desc o nazwie *led,*
    - wskaźnik na strukturę typu *platform_device* o nazwie *pdev,*
  - W funkcji *my_gpio_probe* utworzyć wskaźnik na strukturę typu *my_gpio* o nazwie *my_gpio* oraz zainicjalizować go za pomocą funkcji *devm_kzalloc* i dodać obsługę błędów.
  - Przypisać strukturę z prywatnymi danymi urządzenia (*my_gpio*) do instancji urządzenia za pomocą funkcji *dev_set_drvdata.*
  - Do wskaźnika *pdev* przypisać adres struktury typu *device* o nazwie *dev.*
8. Uzyskać deskryptor pinu GPIO, wykorzystać funkcję *devm_gpiod_get,* jako argumenty podać:
   - *&dev->dev* – wskaźnik na strukturę reprezentującą urządzenie,
   - *"led"* – nazwa pinu zdefiniowana w nakładce drzewa urządzeń,
   - *GPIOD_ASIS* – flaga konfiguracyjna, oznaczająca że użyte mają zostać flagi ustawione w nakładce drzewa urządzeń.

   Wartość zwróconą przez funkcję zapisać do wskaźnika o nazwie *led* znajdującego się w strukturze prywatnych danych urządzenia (*my_gpio*), a w przypadku wystąpienia błędu, poprawnie go obsłużyć.
9. Włączyć diodę LED w momencie dodawania nowego urządzenia (funkcja *probe*). Do tego celu użyć funkcji *gpiod_set_value.*
10. Wyłączyć diodę LED w momencie usuwania urządzenia (funkcja *remove*), w tym celu:
    - w funkcji *my_gpio_remove* uzyskać dostęp do prywatnych danych urządzenia, za pomocą funkcji *dev_get_drvdata,*
    - ustawić stan pinu GPIO odpowiadającego za sterowanie diodą na wartość *0*.
11. Skompilować moduł i załadować go do jądra.
12. Przetestować działanie modułu i zamieścić poniżej kod funkcji *my_gpio_probe*, *my_gpio_probe*, oraz struktury *my_gpio*.

```
struct my_gpio {
        struct gpio_desc *led;
        struct platform_device *pdev;
};

static int my_gpio_probe(struct platform_device *dev)
{
        int err = 0;
        struct my_gpio *my_gpio;

        my_gpio = devm_kzalloc(&dev->dev, sizeof(struct my_gpio),
                                GFP_KERNEL);
        if (IS_ERR(my_gpio)) {
                dev_err(&dev->dev, "Memory allocation failed.\n");
                err = PTR_ERR(my_gpio);
                goto out_ret_err;
        }
```

```
        dev_set_drvdata(&dev->dev, my_gpio);

        my_gpio->led = devm_gpiod_get(&dev->dev, "led", GPIOD_ASIS);
        if (IS_ERR(my_gpio->led)) {
                dev_err(&dev->dev,
                        "GPIO descriptor acquisition failed.\n");
                err = PTR_ERR(my_gpio->led);
                goto out_ret_err;
        }

        gpiod_set_value(my_gpio->led, 1);

        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}

static int my_gpio_remove(struct platform_device *dev)
{
        struct my_gpio *my_gpio = dev_get_drvdata(&dev->dev);
        gpiod_set_value(my_gpio->led, 0);
        dev_info(&dev->dev, "removed\n");
        return 0;
}
```

Zadanie – odczyt stanu przycisku

1. Zaimplementować obsługę drugiego pinu GPIO, w tym celu:
   - Do struktury prywatnych danych urządzenia dodać wskaźnik na deskryptor pinu GPIO o nazwie ”*btn”*.
   - W funckji *my_gpio_probe* uzyskać deskryptor pinu GPIO o nazwie *”btn”* i zapisać go do wskaźnika *btn (*w strukturze *my_gpio)*. Użyć funkcji *devm_gpiod_get*.
2. Ustawić stan diody LED na podstawie stanu przycisku w momencie dodawania urządzenia do systemu (funkcja *probe*). W tym celu:
   - Utworzyć zmienną lokalną typu *int* o nazwie *btn_state*.
   - Odczytać stan przycisku i zapisać go do zmiennej . Wykorzystać funkcję *gpiod_get_value.*
   - W funkcji ustawiającej stan pinu odpowiedzialnego za sterowanie diodą LED, zamiast wartości *1* użyć wartości zmiennej *btn_state.*
3. Skompilować i załadować moduł do jądra.
4. Przetestować jego działanie i umieścić poniżej kod funkcji *my_gpio_probe* wraz ze strukturą *my_gpio.*

```
struct my_gpio {
        struct gpio_desc *led;
        struct gpio_desc *btn;
        struct platform_device *pdev;
};

static int my_gpio_probe(struct platform_device *dev)
{
        int err = 0;
        int btn_state;
        struct my_gpio *my_gpio;

        my_gpio = devm_kzalloc(&dev->dev, sizeof(struct my_gpio),
                               GFP_KERNEL);
        if (IS_ERR(my_gpio)) {
                dev_err(&dev->dev, "Memory allocation failed.\n");
                err = PTR_ERR(my_gpio);
                goto out_ret_err;
        }

        dev_set_drvdata(&dev->dev, my_gpio);

        my_gpio->led = devm_gpiod_get(&dev->dev, "led", GPIOD_ASIS);
        if (IS_ERR(my_gpio->led)) {
                dev_err(&dev->dev,
                        "GPIO descriptor acquisition failed.\n");
                err = PTR_ERR(my_gpio->led);
                goto out_ret_err;
        }

        my_gpio->btn = devm_gpiod_get(&dev->dev, "btn", GPIOD_ASIS);
        if (IS_ERR(my_gpio->btn)) {
                dev_err(&dev->dev,
                        "GPIO descriptor acquisition failed.\n");
                err = PTR_ERR(my_gpio->btn);
                goto out_ret_err;
        }

        btn_state = gpiod_get_value(my_gpio->btn);
        if (btn_state < 0) {
                dev_err(&dev->dev,
                        "Failed to read GPIO pin state.\n");
                err = btn_state;
                goto out_ret_err;
        }

        gpiod_set_value(my_gpio->led, btn_state);

        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}
```

Zadanie - współpraca sterownika z aplikacją użytkownika

Aby umożliwić komunikację sterownika platformowego z aplikacją działającą w przestrzeni użytkownika, sterownik GPIO musi utworzyć odpowiednie pliki w systemie plików. Dzięki temu aplikacja będzie mogła za jego pośrednictwem sterować diodami LED.

1. Przygotować nakładkę drzewa urządzeń zawierającą cztery węzły o nazwach *led0, led1.. led3* z właściwościami reprezentującymi piny GPIO o numerach 22, 25, 5 i 16. Właściwości powinny być nazwane *led-gpio*.
2. Usunąć część kodu związaną z obsługą przycisku oraz włączaniem diody LED (tylko funkcja *gpiod_set_value*) w module *my_gpio.*
3. Dodać obsługę atrybutu sysfs o nazwie *my_gpio_value*, w tym celu:
   - Zarejestrować atrybut tylko do zapisu za pomocą makra *DEVICE_ATTR_WO.*
   - Dodać funkcję zapisu atrybutu, którą przedstawiono poniżej.

```
static ssize_t my_gpio_value_store(struct device *dev,
                                   struct device_attribute *attr,
                                   const char *buf, size_t count)
{
        int err = 0;
        int value;
        struct my_gpio *my_gpio = dev_get_drvdata(dev);
        /* ... */
        return count;
}
```

   - Dodać funkcję odczytu atrybutu, którą przedstawiono poniżej.

```
static ssize_t my_gpio_value_show(struct device *dev,
                                  struct device_attribute *attr,
                                  char *buf)
{
        int value;
        struct my_gpio *my_gpio = dev_get_drvdata(dev);
        /* ... */
        return sprintf(buf, "%lu\n", (unsigned long)value);
}
```

   - W momencie dodawania urządzenia (funkcja *probe*) utworzyć plik atrybutu dostępny w przestrzeni użytkownika. Użyć funkcji *device_create_file*.
   - W momencie usuwania urządzenia(funkcja *remove*) usunąć plik atrybutu z systemu plików za pomocą funkcji *device_remove_file.*
4. Ustawić stan diody LED na wartość wpisaną do pliku atrybutu, w tym celu:
   - W funkcji *my_gpio_value_store* ustawić stan pinu GPIO na wartość z bufora *buf*. Do konwersji znaków na liczbę użyć funkcji *kstrtol*. Dodać obsługę błędów.

- W funkcji *my_gpio_value_show* odczytać stan pinu GPIO i zapisać w zmiennej *value*.

5. Skompilować moduł oraz nakładkę drzewa urządzeń i załadować je do systemu.
6. Przetestować działanie wpisując i odczytując stany diod za pomocą poniższych poleceń.

```
sudo sh -c "echo 0 >> /sys/devices/platform/led[0-3]/my_gpio_value"
sudo cat /sys/devices/platform/led[0-3]/my_gpio_value
```

7. Zamieścić kod modułu *my_gpio* poniżej.

```
#include <linux/of.h>
#include <linux/platform_device.h>
#include <linux/gpio/consumer.h>

struct my_gpio {
        struct gpio_desc *led;
        struct platform_device *pdev;
};

static ssize_t my_gpio_value_show(struct device *dev,
                                  struct device_attribute *attr,
                                  char *buf)
{
        int value;
        struct my_gpio *my_gpio = dev_get_drvdata(dev);

        value = gpiod_get_value(my_gpio->led);
        return sprintf(buf, "%lu\n", (unsigned long)value);
}

static ssize_t my_gpio_value_store(struct device *dev,
                                   struct device_attribute *attr,
                                   const char *buf, size_t count)
{
        int err = 0;
        int value;
        struct my_gpio *my_gpio = dev_get_drvdata(dev);

        err = kstrtol(buf, 0, (unsigned long*)&value);
        if (err) {
                dev_err(dev, "Bad input number.\n");
                return err;
        }

        gpiod_set_value(my_gpio->led, value);
        return count;
}
DEVICE_ATTR_RW(my_gpio_value);
```

```
static int my_gpio_probe(struct platform_device *dev)
{
        int err = 0;

        struct my_gpio *my_gpio;

        my_gpio = devm_kzalloc(&dev->dev, sizeof(struct my_gpio),
                               GFP_KERNEL);
        if (IS_ERR(my_gpio)) {
                dev_err(&dev->dev, "Memory allocation failed.\n");
                err = PTR_ERR(my_gpio);
                goto out_ret_err;
        }

        my_gpio->pdev = dev;

        dev_set_drvdata(&dev->dev, my_gpio);

        my_gpio->led = devm_gpiod_get(&dev->dev, "led", GPIOD_ASIS);
        if (IS_ERR(my_gpio->led)) {
                dev_err(&dev→dev,
                        "GPIO descriptor acquisition failed.\n");
                err = PTR_ERR(my_gpio->led);
                goto out_ret_err;
        }

        err = device_create_file(&dev→dev,
                                 &dev_attr_my_gpio_value);
        if (err) {
                dev_err(&dev->dev, "Device file creation failed.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}


static int my_gpio_remove(struct platform_device *dev)
{
        device_remove_file(&dev->dev, &dev_attr_my_gpio_value);
        dev_info(&dev->dev, "removed\n");

        return 0;
}


static const struct of_device_id my_gpio_of_id[] = {
        { .compatible = "mtm,my_gpio" },
        { },
};
MODULE_DEVICE_TABLE(of, my_gpio_of_id);
```

```
static struct platform_driver my_gpio_driver = {
        .probe = my_gpio_probe,
        .remove = my_gpio_remove,
        .driver = {
                .name = "my_gpio",
                .of_match_table = my_gpio_of_id,
                .owner = THIS_MODULE,
        },
};
module_platform_driver(my_gpio_driver);

MODULE_LICENSE("GPL v2");
```

8. Zapisać kod aplikacji działającej w przestrzeni użytkownika pod nazwą *my_gpio.py*. Kod aplikacji przedstawiono poniżej.

```
from time import sleep
import curses
from math import floor

curses.KEY_ENTER = 10

class led_driver:
        def __init__(self, led):
                self.LED_PATH = "/sys/devices/platform/led" + str(led) +
                                "/my_gpio_value"

        def __set_value(self, value):
                file = open(self.LED_PATH, "w")         ## otworzenie pliku
                file.write(str(value))                  ## Zapis do pliku
                file.close()                            ## Zamknięcie pliku

        def __all_off(self):
                for i in range(4):
                        self.__set_value(0)

        def on(self):
                self.__set_value(1)

        def off(self):
                self.__set_value(0)

        def get_value(self):
                file = open(self.LED_PATH, "r")
                value = int(file.readline())
                file.close()
                return value

        def toggle(self):
                if (self.get_value()):
                        self.off()
                        return 0
                else:
                        self.on()
                        return 1

class box:
        def __init__(self, screen, x, y, str, w, h):
```

```
                self.screen = screen
                self.x = x
                self.y = y
                self.str = str
                self.w = w
                self.h = h
                self.color = curses.color_pair(0)

        def draw(self):
                for line in range(self.h):
                        for char in range(self.w):
                                self.screen.addch(self.y+line, self.x+char,
                                                  " ", self.color)

                str_len = len(self.str)
                self.screen.addstr(floor(self.y+self.h/2), floor(self.x+
                                   (self.w-str_len)/2), self.str,
                                   self.color | curses.A_BOLD)

        def set_color(self, color):
                self.color = color

        def set_text(self, str):
                self.str = str

class led_box(box):
        def __init__(self, screen, x, y, str, w=7, h=3):
                super().__init__(screen, x, y, str, w, h)
                self.set_color(curses.color_pair(2))

class led_pointer(box):
        def __init__(self, screen, x, y, str, w=7, h=3):
                super().__init__(screen, x, y, str, w, h)
                self.set_color(curses.color_pair(0))

class led:
        def __init__(self, screen, x, y, str):
                self.led_b = led_box(screen, x, y, str, 11, 3)
                self.pointer = led_pointer(screen, x+15, y+1, "", 6, 1)
                self.activate()

        def draw(self):
                self.led_b.draw()
                self.pointer.draw()

        def activate(self):
                self.pointer.set_text("<---")

        def deactivate(self):
                self.pointer.set_text("")

        def on(self):
                self.led_b.set_color(curses.color_pair(1))

        def off(self):
                self.led_b.set_color(curses.color_pair(2))

def main(stdscr):
        curses.start_color()
        curses.init_pair(1, curses.COLOR_WHITE, curses.COLOR_GREEN)
        curses.init_pair(2, curses.COLOR_WHITE, curses.COLOR_RED)
```

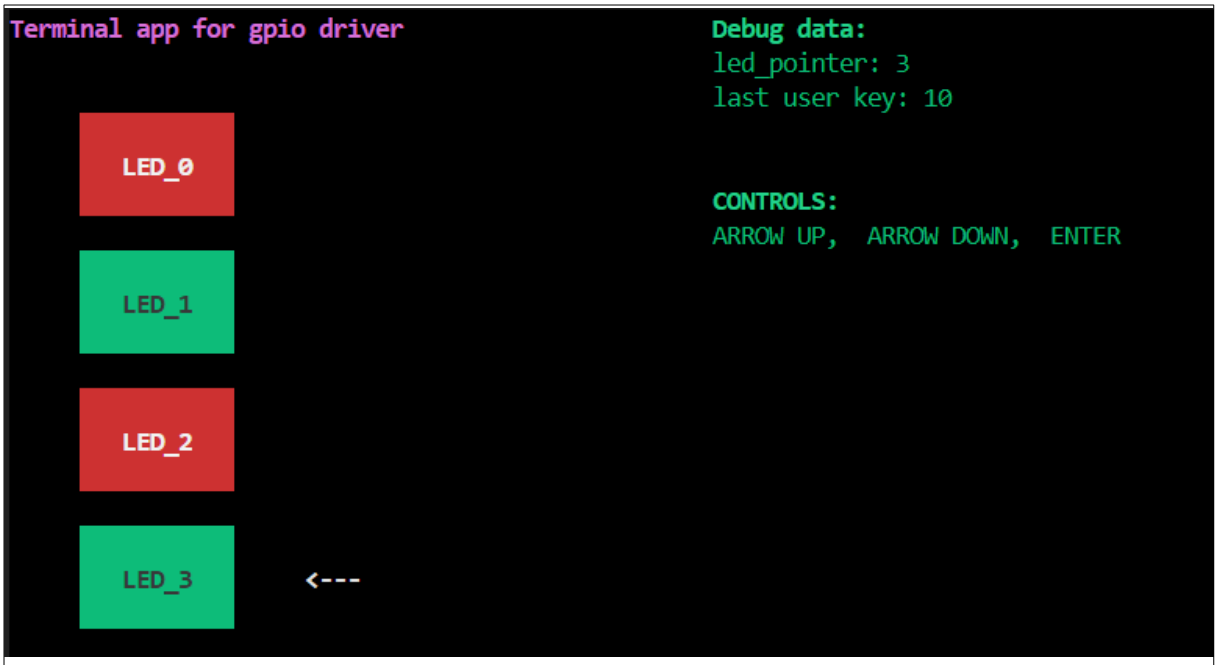
```
curses.init_pair(3, curses.COLOR_GREEN, curses.COLOR_BLACK)
curses.init_pair(4, curses.COLOR_MAGENTA, curses.COLOR_BLACK)


leds = [led(stdscr, 5, 3, "LED_0"),
        led(stdscr, 5, 7, "LED_1"),
        led(stdscr, 5, 11, "LED_2"),
        led(stdscr, 5, 15, "LED_3")]

leds_drv = [led_driver(0),
            led_driver(1),
            led_driver(2),
            led_driver(3)]

led_pointer = 0
user_char = None

curses.curs_set(0)

while True:
        # debug
        stdscr.addstr(0, 50, "Debug data:", curses.A_BOLD |
                      curses.color_pair(3))
        stdscr.addstr(1, 50, "led_pointer: " + str(led_pointer),
                      curses.color_pair(3))
        stdscr.addstr(2, 50, "last user key: " + str(user_char),
                      curses.color_pair(3))

        # legend
        stdscr.addstr(5, 50, "CONTROLS:", curses.A_BOLD |
                      curses.color_pair(3))
        stdscr.addstr(6, 50, "ARROW UP,  ARROW DOWN,  ENTER",
                      curses.color_pair(3))

        # Title
        stdscr.addstr(0, 0, "Terminal app to for gpio driver",
                      curses.A_BOLD | curses.color_pair(4))

        # Led pointer controll
        for i, led_i in enumerate(leds):
                if(led_pointer == i):
                        led_i.activate()
                else:
                        led_i.deactivate()

                led_i.draw()

        # User input control
        user_char = stdscr.getch()

        if (user_char == curses.KEY_DOWN):
                led_pointer += 1
                if (led_pointer == len(leds)): led_pointer = 0
        elif (user_char == curses.KEY_UP):
                led_pointer -= 1
                if (led_pointer < 0): led_pointer = len(leds)-1
        elif (user_char == curses.KEY_ENTER):
                state = leds_drv[led_pointer].toggle()
                if (state):
                        leds[led_pointer].on()
                else:
```

```
                                leds[led_pointer].off()
                stdscr.erase()
curses.wrapper(main)
```

9. Uruchomić aplikację poniższym poleceniem.

```
sudo python my_gpio.py
```

10. Poniżej zamieścić zrzut ekranu przedstawiający interfejs aplikacji z włączonymi diodami led numer 1 i 2.



Urządzenie platformowe na przykładzie sterownika serwomechanizmu

Prosty sterownik serwomechanizmu umożliwia cykliczne przełączanie cewek silnika krokowego, który dla celów niniejszej instrukcji został zasymulowany za pomocą czterech diod LED. Sterowanie polega na przełączaniu diod w określonej kolejności – zapalana jest kolejna dioda, w taki sposób, że zawsze zaświecona jest tylko jedna dioda.

Cykliczne sterowani pinami GPIO - Timer

Aby móc cyklicznie przełączać diody LED, potrzeba mechanizmu, który zapewni możliwość wykonywania zadań w określonych odstępach czasu. Do tego celu nadają się timery dostępne w jądrze Linux. Pozwalają one na wykonanie wskazanej funkcji, nazywanej callbackiem, po upływie zadanego czasu. Aby zapewnić cykliczne wywołanie callbacka, timer musi zostać ponownie uzbrojony w tymże callbacku.

Podstawowym mechanizmem odmierzania czasu w jądrze Linux jest zegar systemowy. Jego częstotliwość zależy od wartości stałej o nazwie *HZ*, która jest zdefiniowana wewnątrz jądra. Domyślnie na platformie Raspberry Pi Zero W wartość *HZ* ustawiona jest na *1000[Hz]*.

Czas w jądrze systemu Linux jest wyrażany za pomocą *jiffies*. Jest to zmienna, która przechowuje liczbę taktów zegara systemowego od momentu uruchomienia systemu.

Poniżej przedstawiono definicję makra służącego do skonfigurowania timera. [3]

```
#define timer_setup(timer, callback, flags)                \
      __init_timer((timer), (callback), (flags))
```

Argumenty makrodefinicji:

- *timer* – wskaźnik typu *struct timer_list* reprezentujący timer,
- *callback* – funkcja typu *void*, która ma zostać wywołana po odliczeniu zadanego czasu,
- *flags* - pozwala na określenie opcjonalnych właściwości timera.

W celu zwolnienia zasobów timera podczas usuwania urządzenia z systemu, należy użyć funkcji del_timer_sync, która dezaktywuje timer. Jej deklaracja została przedstawiona poniżej. [3]

```
int del_timer_sync(struct timer_list *timer);
```

Funkcja jako argument przyjmuje wskaźnik na instancję timera, oraz zwraca wartość *0* jeśli dezaktywuje pracujący timer, lub *1* w przyeciwnym przypadku.

Aby zapewnić, że funkcja wskazana w argumencie *callback* będzie wywoływana cyklicznie, należy po każdym jej wykonaniu ponownie przezbroić timer za pomocą funkcji przedstawionej poniżej. Ustawia ona czas timera oraz aktywuje go. [3]

```
int mod_timer(struct timer_list *timer,
              unsigned long expires);
```

Argumenty funkcji:

- **timer* - wskaźnik typu *struct timer_list* reprezentujący timer,
- *expires* – wartość czasu jiffies, w którym zostanie wywołana funkcja podana w argumencie *callback* makra *timer_setup*. Należy podać *jiffies* + *msecs_to_jiffies(T)*, gdzie *T* to czas w milisekundach, po którym ma zostać uruchomiony callback.

Funkcja modyfikuje licznik timera i zwraca wartość *0* w przypadku, gdy zmodyfikowano licznik aktywnego timera lub *0* jeśli nieaktywnego. Dodatkowo funkcja automatycznie uruchamia niektywny timer.

Dostęp do prywatnych danych urządzenia w funkcjach

Podczas tworzenia sterowników urządzeń występują sytuacje, w których funkcja jądra nie przekazuje w argumencie wskaźnika na urządzenie, dzięki któremu można by uzyskać dostęp do jego prywatnych danych. Takim przykładem jest funkcja callback timera, której deklarację przedstawiono poniżej. [3]

```
static void servo_timer_callback(struct timer_list * timer);
```

Powyższa funckja przekazuje jedynie wskaźnik na strukturę timera. Ponieważ nie ma tutaj wskaźnika na strukturę urządzenia struct_device, nie można "w prosty" sposób uzyskać dostępu do struktury z danymi urządzenia.

Makro container_of

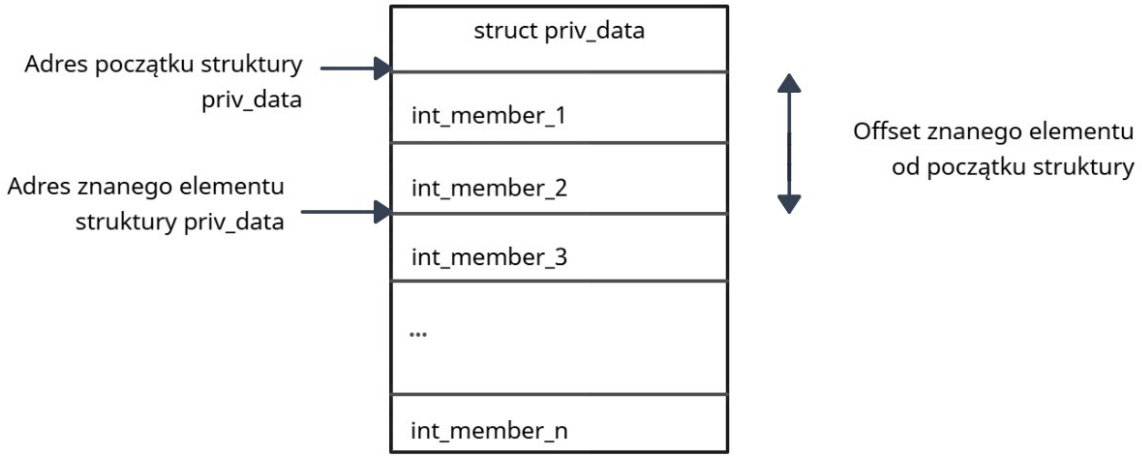
Aby poradzić sobie z tym problemem, jądro systemy Linux udostępnia specjalną makrodefinicję o nazwie container_of. Pozwala ona pozyskać wskaźnik na strukturę danych, która zawiera przynajmniej jeden element o znanym adresie. Poniżej przedstawiono makro container_of, które zwraca wskaźnik na strukturę, do której należy wskazany element. [3]

```
#define container_of(ptr, type, member) ({                      \
        void *__mptr = (void *)(ptr);                           \
        static_assert(__same_type(*(ptr), ((type *)0)->member) ||\
                      __same_type(*(ptr), void),                \
                      "pointer type mismatch in container_of()");\
        ((type *)(__mptr - offsetof(type, member))); })
```

Argumenty makrodefinicji:

- *ptr* – wskaźnik na znany element struktury,
- *type* – typ struktury w której znajduje się znany element,
- *member* – nazwa znanego elementu.

Działanie makra *container_of* przedstawia poniższa grafika.



Znając typ struktury i adres jednego jej elementu, preprocesor za pomocą makra *offsetof* oblicza offset tego elementu od początku struktury.
Znając wartość tego offsetu i adres elementu, za pomocą operacji odejmowania oblicza adres początku struktury danych:

[adres elementu] – [offset elementu] = adres początku struktury

Zadanie - dodanie obsługi timera i GPIO

1. Utworzyć plik nakładki drzewa urządzeń o nazwie *servo.dts* i skopiować do niego poniższy kod, który dodaje węzeł urządzenia *servo* z właściwością *leds_gpio*, która reprezentuje tablicę pinów GPIO..

```
/dts-v1/;
/plugin/;

/ {
        compatible = "brcm,bcm2835";

        fragment@0 {
                target-path = "/";

                __overlay__ {
                        servo {
                                compatible = "mtm,servo";
                                leds-gpios = <&gpio 22 0x02>,
                                             <&gpio 25 0x02>,
                                             <&gpio 5 0x02>,
```

```
                                                <&gpio 16 0x02>;
                        };
                };
        };
};
```

2. Otworzyć plik źródłowy modułu o nazwie *servo.c*, który jest dostępny w załącznikach.
3. Dodać obsługę pinów GPIO, w tym celu:
   - w funckji *servo_probe* uzyskać tablicę deskryptorów pinów GPIO za pomocą funkcji *devm_gpiod_get_array,* której deklarację przedstawiono poniżej. Zapisać zwróconą wartość do wskaźnika *gpio_descs* (znajdującego się w strukturze *servo*). [3]

```
struct gpio_descs *__must_check devm_gpiod_get_array(struct device *dev,
                                                     const char *con_id,
                                                     enum gpiod_flags flags);
```

Argumenty funkcji:

- **dev* – wskaźnik na strukturę urządzenia,
- **con_id* – nazwa tablicy pinów GPIO podana w węźle drzewa urządzeń (np. *"leds"*),
- *flags* – opcjonalne flagi konfiguracyjne pinów GPIO, wartość *GPIOD_ASIS* (*0*) oznacza, że flagi podane w drzewie urządzeń nie będą nadpisane.

Funkcja zwraca wskaźnik na strukturę typu *struct gpio_descs*, wewnątrz której znajduje się tablica deskryptorów typu *struct gpio_desc*. W przypadku wystąpienia błędu, zwrócony zostaje wskaźnik z zakodowanym błędem.

- Dodać funkcję typu *void* o nazwie *servo_set_leds*, która podaną wartość na diodach LED (wyświetlacz binarny). Funkcja powinna przyjmować dwa argumenty:
  - *struct servo *servo*
  - *int value – wartość 0-15*

  Deskryptor pojedynczej diody dostępny jest w *servo→leds→desc[<numer diody LED>]*

4. Dodać obsługę timera, który co 100 milisekund zinkrementuje wartość licznika diodowego (licznik binarny wyświetlany za pomocą diod LED), w tym celu:
   - Dodać definicję funkcji callback, której deklarację przedstawiono poniżej, oraz stworzyć wewnątrz niej zmienną typu *static int* o nazwie leds_ctr,

```
static void servo_timer_callback(struct timer_list * timer);
```

Jako argument funkcja przyjmuje wskaźnik na instancję timera.

- Wewnątrz callbacka:
  - uzyskać dostęp do prywatnych danych urządzenia, w tym celu utworzyć wskaźnik na prywatne dane typu *struct servo* o nazwie *servo* i przekazać do niego wynik działania makra *container_of* jak przedstawiono poniżej

```
struct servo *servo = container_of(timer, struct servo,
                                   timer);
```

    - jako wskaźnik na znany element podać adres przekazany w argumencie callbacka – *timer,*
    - jako typ podać typ struktury - *struct servo,*
    - jako element podać nazwę znanego elementu – *timer.*
  - za pomocą funkcji *servo_set_leds* ustawiać stan diod LED na kolejne wartości inkrementowanego licznika.
- W funkcji *servo_probe* skonfigurować timer za pomocą makra *timer_setup*, i uruchomić go za pomocą funkcji *mod_timer* (ustawić timer na 500ms).
- W funkcji *servo_remove* usunąć timer za pomocą funkcji *del_timer_sync* i wyłączyć wszystkie diody led.

5. Skompilować i załadować kod modułu oraz nakładki drzewa urządzeń.
6. Przetestować działanie sterownika i zamieścić poniżej jego kod.

```
#include <linux/of.h>
#include <linux/platform_device.h>
#include "linux/gpio/consumer.h"
#include "linux/timer.h"


#define LEDS_NUMBER 4


struct servo {
        struct gpio_descs *leds;
        struct platform_device *pdev;
        struct timer_list timer;

};


static void servo_set_leds(struct servo *servo, int value)
{
        int led_ctr = 0;
        for (led_ctr = 0; led_ctr < LEDS_NUMBER; led_ctr++)
                gpiod_set_value(servo->leds->desc[led_ctr],
                                ((value >> led_ctr) & 0x01));
}
```

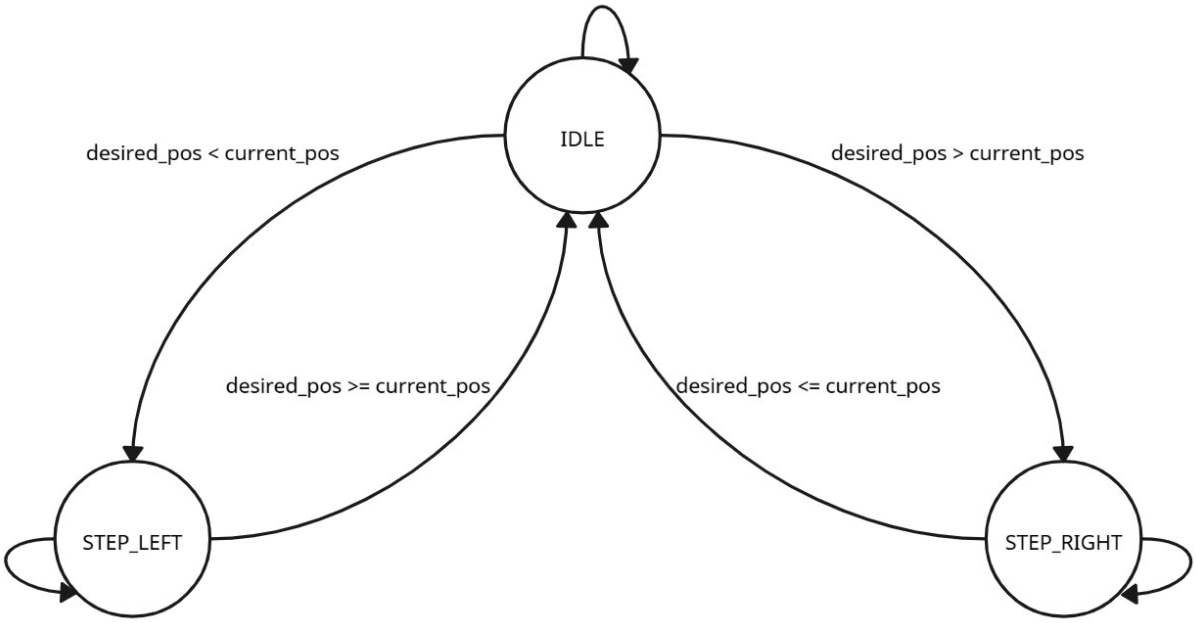
```
static void servo_timer_callback(struct timer_list * timer)
{
        static int leds_ctr = 0;
        struct servo *servo = container_of(timer, struct servo,
                                           timer);
        servo_set_leds(servo, leds_ctr);
        leds_ctr++;
        if (leds_ctr == 16)
                leds_ctr = 0;

        mod_timer(timer, jiffies + msecs_to_jiffies(100));
}


static int servo_probe(struct platform_device *dev)
{
        int err = 0;
        struct servo *servo;

        servo = devm_kzalloc(&dev->dev, sizeof(struct servo),
                             GFP_KERNEL);
        if (IS_ERR(servo)) {
                dev_err(&dev->dev, "Memory allocation failed.\n");
                err = PTR_ERR(servo);
                goto out_ret_err;
        }

        servo->pdev = dev;
        dev_set_drvdata(&dev->dev, servo);
        servo->leds = devm_gpiod_get_array(&dev->dev, "leds",
                                           GPIOD_OUT_LOW);
        if (IS_ERR(servo->leds)) {
                dev_err(&dev->dev,
                        "GPIO descriptor acquisition failed.\n");
                err = PTR_ERR(servo->leds);
                goto out_ret_err;
        }

        timer_setup(&servo->timer, &servo_timer_callback, 0);
        mod_timer(&servo->timer, jiffies + msecs_to_jiffies(100));
        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}

static int servo_remove(struct platform_device *dev)
{
        struct servo *servo = dev_get_drvdata(&dev->dev);
        del_timer_sync(&servo->timer);
        dev_info(&dev->dev, "removed\n");
        return 0;
}
```

```
static const struct of_device_id servo_of_id[] = {
        { .compatible = "mtm,servo" },
        { },
};
MODULE_DEVICE_TABLE(of, servo_of_id);

static struct platform_driver servo_driver = {
        .probe = servo_probe,
        .remove = servo_remove,
        .driver = {
                .name = "servo",
                .of_match_table = servo_of_id,
                .owner = THIS_MODULE,
        },
};
module_platform_driver(servo_driver);

MODULE_LICENSE("GPL v2");
```

Obsługa serwomechanizmu

Aby uzyskać możliwość sterowania serwomechanizmem z pomocą wirtualnego systemu plików, należy dodać do sterownika logikę sterującą krokami silnika krokowego. Bieguny silnika krokowego serwomechanizmu zostały zasymulowanie za pomocą czterech diod LED które są cyklicznie przełączane zgodnie z ich kolejnością. Pojedyncze przełączenie diody symuluje jeden krok silnika krokowego.

Kontrolę kroków silnika umożliwi maszyna stanów, której diagram przedstawiono poniżej.



Opis stanów automatu:

- *IDLE* – jeśli zadana pozycja serwa jest mniejsza od pozycji bieżącej, to przejdź do stanu *STEP_LEFT,*
  jeśli zadana pozycja serwa jest większa od pozycji bieżącej, to przejdź do stanu *STEP_RIGHT,*
  w innym przypadku pozostań w stanie *IDLE.*
- *STEP_LEFT* – jeśli zadana pozycja jest mniejsza od pozycji bieżącej, to wywołaj funkcję *servo_step_left*, zdekrementuj licznik bieżącej pozycji i pozostań w obecnym stanie.
  W innym przypadku przejdź do stanu *IDLE*.
- *STEP_RIGHT* - jeśli zadana pozycja jest większa od pozycji bieżącej, to wywołaj funkcję *servo_step_right*, zinkrementuj licznik bieżącej pozycji i pozostań w obecnym stanie.
  W innym przypadku przejdź do stanu *IDLE*.

Zadanie – sterownik serwomechanizmu

1. Dodać do kodu modułu *servo* obsługę kontroli pozycji serwomechanizmu, w tym celu:
   - Dodać definicję stanów automatu, jak poniżej:

```
enum servo_state {IDLE, STEP_LEFT, STEP_RIGHT};
```

   - Do struktury z prywatnymi danymi urządzenia dodać zmienne reprezentujące :
     - stan automatu kontrolującego serwo – *enum servo_state state*,
     - bieżącą pozycje serwa – *int current_pos*,
     - docelową pozycjae serwa – *int desired_pos*,
     - licznik pomocniczy do sterowania diodami LED – *int led_ctr.*
   - Dodać funkcję, która zapali tylko jedną diodę LED o podanym numerze (*static void servo_set_led*).
   - Dodać funkcję, które odpowiednio zwiększą i zmniejszą licznik pomocniczy (*static void servo_step_left* i *static void servo_step_right*). Licznik może przyjmować tylko wartości 0, 1, 2, 3.
   - W funkcji *servo_timer_callback* dodać maszynę stanów, która będzie kontrolować pozycję serwa.
     Po wykonaniu maszyny stanów, zapalić diodę LED o numerze odpowiadającemu wartości zmiennej *led_ctr* za pomocą funkcji servo_set_led i ponownie uzbroić timer.
2. Dodać możliwość zadawania pozycji docelowej serwa za pomocą pliku atrybutu w wirtualnym systemie plików */sys*. W tym celu:
   - Zarejestrować atrybut tylko do zapisu o nazwie servo_desired_pos. Użyć makra DEVICE_ATTR_WO.

- Dodać funkcję zapisu atrybutu, która wpisaną przez użytkownika liczbę, zapisze do zmiennej *servo->desired_pos* (*static int servo_desired_pos_store*).
- Utworzyć plik atrybutu podczas dodawania urządzenia do systemu. Użyć funkcji *device_create_file*.
- Usunąć plik atrybutu w momencie usuwania urządzenia z systemu. Użyć funkcji *device_remove_file.*

3. Skompilować i załadować moduł do jądra.
4. Przetestować działanie sterownika i zamieścić jego kod poniżej.

```
#include <linux/of.h>
#include <linux/platform_device.h>
#include "linux/gpio/consumer.h"
#include "linux/timer.h"

#define LEDS_NUMBER 4

enum servo_state {IDLE, STEP_LEFT, STEP_RIGHT};

struct servo {
        struct gpio_descs *leds;
        struct platform_device *pdev;
        struct timer_list timer;
        enum servo_state state;
        int current_pos;
        int desired_pos;
        int led_ctr;
};

static ssize_t servo_desired_pos_store(struct device *dev,
                                       struct device_attribute *attr,
                                       const char *buf, size_t count)
{
        int err;
        struct servo *servo = dev_get_drvdata(dev);

        err = kstrtol(buf, 0, (long int*)(&servo->desired_pos));
        if (err) {
                dev_err(dev, "Bad input number.\n");
                return err;
        }

        return count;
}
DEVICE_ATTR_WO(servo_desired_pos);

static void servo_set_leds(struct servo *servo, int value)
{
        int led_ctr = 0;
        for (led_ctr = 0; led_ctr < LEDS_NUMBER; led_ctr++)
                gpiod_set_value(servo->leds->desc[led_ctr],
                                ((value >> led_ctr) & 0x01));
}
```

```
static void servo_set_led(struct servo *servo, int led)
{
        servo_set_leds(servo, (1 << led));
}

static void servo_step_left(struct servo *servo)
{
        servo->led_ctr--;
        if (servo->led_ctr < 0)
                servo->led_ctr = 3;
}

static void servo_step_right(struct servo *servo)
{
        servo->led_ctr++;
        if (servo->led_ctr > 3)
                servo->led_ctr = 0;
}

static void servo_timer_callback(struct timer_list * timer)
{
        struct servo *servo = container_of(timer, struct servo,
                                           timer);
        switch (servo->state) {
                case IDLE:
                        if (servo->current_pos > servo->desired_pos)
                                servo->state = STEP_LEFT;
                        else if (servo->current_pos <
                                 servo->desired_pos)
                                servo->state = STEP_RIGHT;
                        else
                                servo->state = IDLE;
                        break;

                case STEP_LEFT:
                        if (servo->current_pos > servo->desired_pos)
{
                                servo_step_left(servo);
                                servo->current_pos--;
                                servo->state = STEP_LEFT;
                        } else {
                                servo->state = IDLE;
                        }
                        break;

                case STEP_RIGHT:
                        if(servo->current_pos < servo->desired_pos)
{
                                servo_step_right(servo);
                                servo->current_pos++;
                                servo->state = STEP_RIGHT;
                        } else {
                                servo->state = IDLE;
                        }
                        break;
```

```
                default:
                        break;
        }

        servo_set_led(servo, servo->led_ctr);
        mod_timer(timer, jiffies + msecs_to_jiffies(100));
}

static int servo_probe(struct platform_device *dev)
{
        int err = 0;
        struct servo *servo;

        servo = devm_kzalloc(&dev->dev, sizeof(struct servo),
                             GFP_KERNEL);
        if (IS_ERR(servo)) {
                dev_err(&dev->dev, "Memory allocation failed.\n");
                err = PTR_ERR(servo);
                goto out_ret_err;
        }

        servo->pdev = dev;
        dev_set_drvdata(&dev->dev, servo);

        servo->leds = devm_gpiod_get_array(&dev->dev, "leds",
                                           GPIOD_OUT_LOW);
        if (IS_ERR(servo->leds)) {
                dev_err(&dev->dev,
                        "GPIO descriptor acquisition failed.\n");
                err = PTR_ERR(servo->leds);
                goto out_ret_err;
        }

        err = device_create_file(&dev->dev,
                                 &dev_attr_servo_desired_pos);
        if (err) {
                dev_err(&dev->dev, "Device file creation failed.\n");
                goto out_ret_err;
        }

        timer_setup(&servo->timer, &servo_timer_callback, 0);
        mod_timer(&servo->timer, jiffies + msecs_to_jiffies(100));

        dev_info(&dev->dev, "probed\n");

out_ret_err:
        return err;
}

static int servo_remove(struct platform_device *dev)
{
        struct servo *servo = dev_get_drvdata(&dev->dev);

        del_timer_sync(&servo->timer);
        device_remove_file(&dev->dev, &dev_attr_servo_desired_pos);
```

```
        dev_info(&dev->dev, "removed\n");
        return 0;
}

static const struct of_device_id servo_of_id[] = {
        { .compatible = "mtm,servo" },
        { },
};
MODULE_DEVICE_TABLE(of, servo_of_id);

static struct platform_driver servo_driver = {
        .probe = servo_probe,
        .remove = servo_remove,
        .driver = {
                .name = "servo",
                .of_match_table = servo_of_id,
                .owner = THIS_MODULE,
        },
};
module_platform_driver(servo_driver);

MODULE_LICENSE("GPL v2");
```

Urządzenia SPI/I2C na przykładzie sterowników układów scalonych

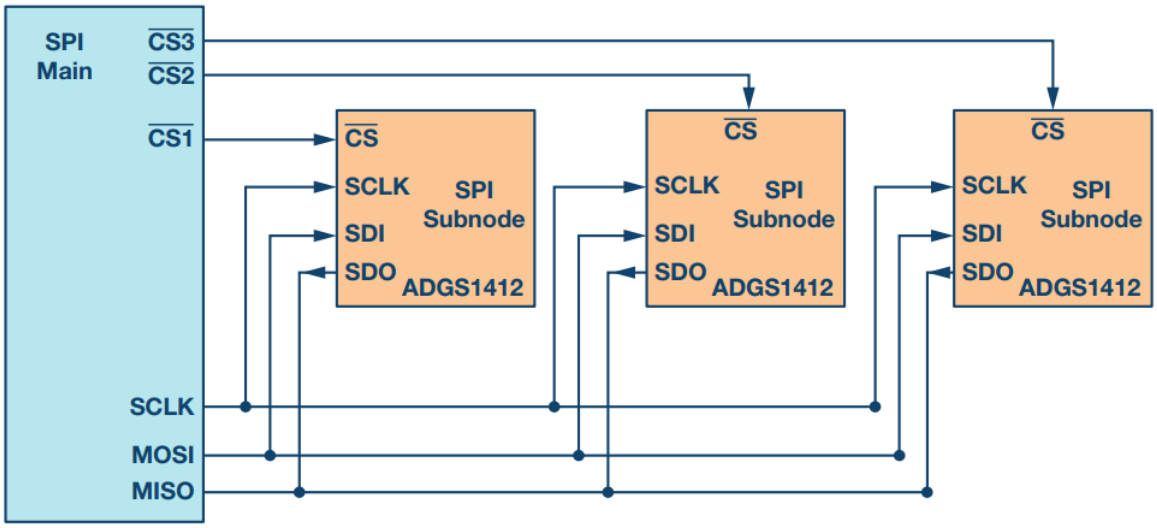
Wprowadzenie

Sterowniki magistral w systemie Linux pozwalają na efektywną komunikację między procesorem a urządzeniami peryferyjnymi za pośrednictwem protokołów takich jak SPI i I²C. Rozdział ten omawia mechanizmy obsługi tych magistral, prezentując sposób ich implementacji w jądrze Linux oraz ich zastosowanie na platformie Raspberry Pi Zero W do sterowania urządzeniami zewnętrznymi.

Magistrala SPI

SPI (Serial Peripheral Interface) to szybka, synchroniczna magistrala komunikacyjna typu master-slave. Wykorzystuje cztery główne linie: SCLK (serial clock), MOSI (master output slave input), MISO (master input slave output) i CS (chip-select). Charakteryzuje się dużą szybkością transmisji, brakiem adresacji urządzeń i wymaga dedykowanej linii SS dla każdego urządzenia slave.

Poniżej przedstawiono schemat połączeń w magistrali SPI. [5]



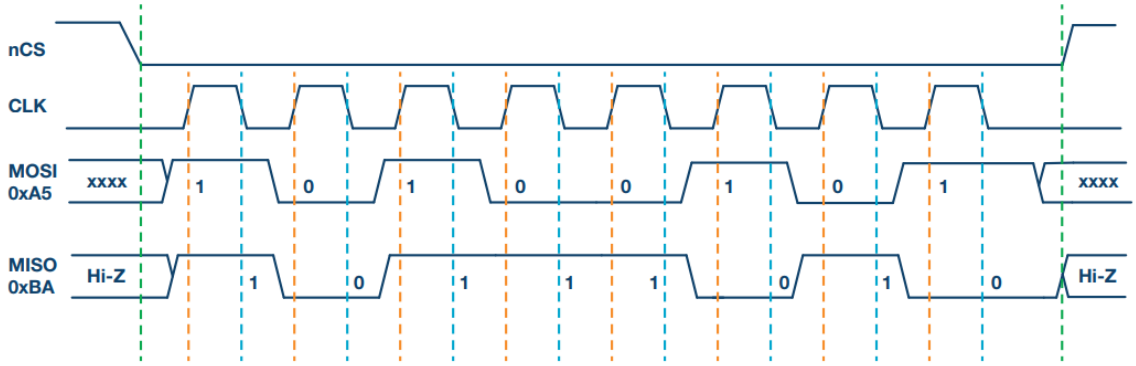
Działanie magistrali opiera się na przesyłaniu pojedynczych bitów zgodnie z taktem zegara. W zależności od ustawienia flag konfiguracyjnych obsługiwane są cztery tryby pracy wymienione poniżej.

- Tryb 0 – stan spoczynkowy zegara – niski, dane próbkowane w momencie narastającego zbocza zegara i przesuwane w momencie opadającego. (CPOL=0, CPHA=0)

- Tryb 1 -stan spoczynkowy zegara – niski, dane próbkowane w momencie opadającego zbocza zegara i przesuwane w momencie narastającego. (CPOL=0, CPHA=1)
- Tryb 2 -stan spoczynkowy zegara – wysoki, dane próbkowane w momencie opadającego zbocza zegara i przesuwane w momencie narastającego. (CPOL=1, CPHA=0)
- Tryb 3 -stan spoczynkowy zegara – wysoki, dane próbkowane w momencie narastającego zbocza zegara i przesuwane w momencie opadającego. (CPOL=1, CPHA=1)

Flaga CPOL oznacza polaryzację zegara, natomiast CPHA fazę zegara i decyduje przy którym zboczu mają być próbkowane dane.

Poniżej przedstawiono przykładowy przebieg komunikacji na magistrali SPI w trybie 0 (CPOL=0, CPHA=0). [5]

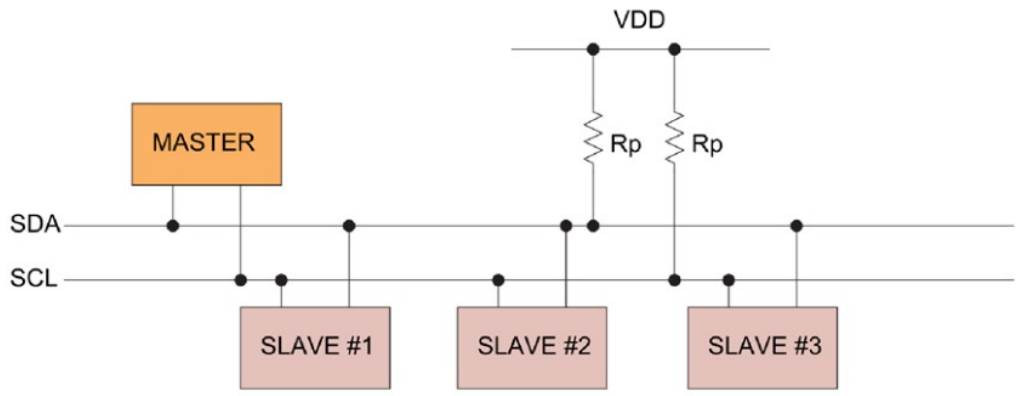


Magistrala I2C

I2C (Inter-Integrated Circuit) to synchroniczna magistrala komunikacyjna, umożliwiająca wymianę danych między urządzeniami. Wykorzystuje dwie linie sygnałowe: SDA (Serial Data Line) do przesyłania danych i SCL (Serial Clock Line) do synchronizacji. Magistrala obsługuje konfigurację master-slave, gdzie urządzenie master steruje komunikacją.

I2C umożliwia podłączenie wielu urządzeń dzięki 7-bitowemu adresowaniu. Charakteryzuje się prostotą implementacji i niską prędkością przesyłu danych (standardowo do 100 kHz, Fast Mode do 400 kHz, a Fast Mode+ do 1 MHz). Wymaga rezystorów podciągających na liniach SDA i SCL.

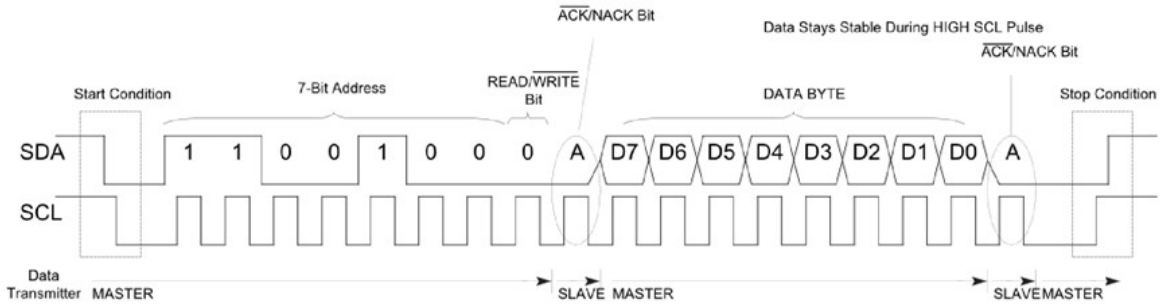
Poniżej przedstawiono schemat połączeń w magistrali I2C. [6]



Rozpoczęcie komunikacji na magistrali I2C jest inicjowane przez układ typu master, który wysyła sygnał START na linii SDA (dane). Master określa adres urządzenia slave, z którym chce nawiązać komunikację, a następnie przesyła dane. Komunikacja kończy się, gdy master wysyła sygnał STOP na linii SDA, co informuje wszystkie urządzenia slave, że transmisja została zakończona. Sygnały START i STOP są kontrolowane przez mastera, zapewniając synchronizację i prawidłowe zakończenie wymiany danych.

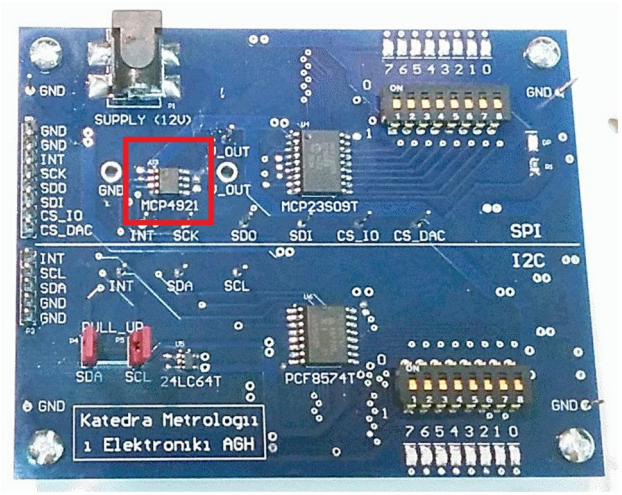
Sygnał START generowany jest, gdy linia SDA przechodzi ze stanu wysokiego na niski, a stan SCL jest wysoki, natomiast sygnał STOP generowany jest, gdy linia SDA przechodzi ze stanu niskiego na wysoki, przy wysokim poziomie SCL.

Poniżej przedstawiono przykładowy przebieg komunikacji na magistrali I2C.



Komunikacja sterownika ze scalonym przetwornikiem C\A z magistralą SPI

MCP4921 to 12-bitowy przetwornik cyfrowo-analogowy (DAC) z interfejsem SPI, produkowany przez firmę Microchip. Poniżej przedstawiono grafikę z zaznaczonym układem scalonym znajdującym się na płytce zestawu do ćwiczeń.



Układ MCP4921 nie korzysta z adresacji rejestrów, sterowanie przetwornikiem odbywa się za pomocą pojedynczego, szesnastobitowego rejestru poleceń. Poniżej przedstawiono grafikę ilustrującą zawartość tego rejestru. [7]

REGISTER 5-1: WRITE COMMAND REGISTER

Upper Half:							
W-x	W-x	W-x	W-0	W-x	W-x	W-x	W-x
$\overline{A}$/B	BUF	$\overline{GA}$	$\overline{SHDN}$	D11	D10	D9	D8
bit 15							bit 8

Lower Half:							
W-x	W-x	W-x	W-x	W-x	W-x	W-x	W-x
D7	D6	D5	D4	D3	D2	D1	D0
bit 7							bit 0

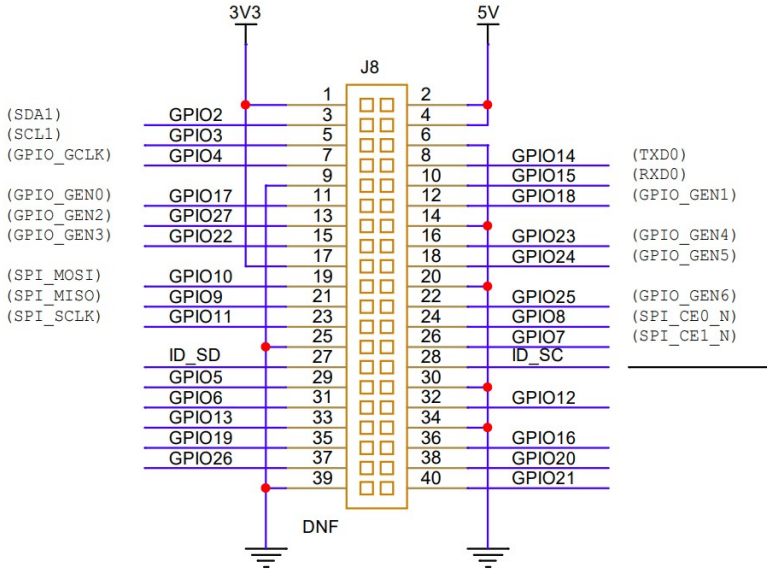
Znaczenie poszczególnych bitów w rejestrze poleceń:

- *!A/B* – bit wyboru kanału (w wersji MCP4921 dostępny jest tylko kanał A),
- *BUF* – bit kontroli buforowania napięcia odniesienia przetwornika (*0* – buforowanie wyłączone, *1* – buforowanie włączone)
- *!GA* – bit kontrolujący wzmocnienie napięcia wyjściowego przetwornika (*0* – wzmocnienie *x2*, *1* – wzmocnienie *x1*),
- *!SHDN* – bit kontrolujący tryb uśpienia układu (*0* – przetwornik wyłączony, *1* – przetwornik włączony).
- *D11-D0* – bity danych, wartość w zakresie *0-4095*.

W celu ustawienia napięcia wyjściowego przetwornika C/A, należy przygotować dwubajtową ramkę danych i wysłać ją do układu za pomocą magistrali SPI.

Zadanie – podłączenie układu MCP4921 do kontrolera Raspberyy Pi Zero W

5. Podłączyć zasilanie płytki zestawu do pin numer *2 (5V)* i numer *6 (GND)* kontrolera Raspberry Pi Zero W. Poniżej przedstawiono schemat złącza goldpin kontrolera. [8]



6. Podłączyć piny magistrali *SPI* na płytce zestawu do następujących pinów kontrolera:
   - GPIO10 (pin numer 19) – MOSI,
   - GPIO9 (pin numer 21) – MISO,
   - GPIO11 (pin numer 23) – SCK,
   - GPIO8 (pin numer 8) – CS_IO.

Nakładka drzewa urządzeń dla sterownika SPI

Podobnie jak w przypadku sterownika platformowego, jądro systemu Linux udostępnia zestaw funkcji i makr upraszających obsługę urządzenia SPI w systemie. Aby skorzystać z tego udogodnienia, należy dodać urządzenie SPI za pośrednictwem drzewa urządzeń.

Wewnątrz drzewa urządzeń istnieje węzeł *spi0* reprezentujący kontroler SPI dostępny w platformie Raspberry Pi Zero W.

Dodanie do drzewa urządzenia korzystającego z magistrali *spi* wymaga utworzenia nowego węzła reprezentującego urządzenie SPI wewnątrz węzła *spi0*. Poniżej przedstawiono nakładkę drzewa urządzeń, która dodaje urządzenie wewnątrz węzła kontrolera *spi0.*

```
/dts-v1/;
/plugin/;

/ {
        compatible = "brcm,bcm2835";
        fragment@1 {
```

```
                target = <&spi0>;
                __overlay__ {
                        status = "okay";
                        #address-cells = <1>;
                        #size-cells = <0>;

                        mcp4921: mcp4921@0{
                                compatible = "microchip,mcp4921";
                                status = "okay";
                                reg = <0>;
                                spi-max-frequency = <100000>;
                                spi-bits-per-word = <8>;
                        };
                };
        };
};
```

Opis właściwości:

- *target = <&spi0>* - określa docelowy węzeł kontrolera SPI, do którego zostanie dodane urządzenie (nowy węzeł *mcp4921*),
- *status = "okay"* – aktywuje urządzenie SPI0 oraz podłączone do niego urządzenia,
- *#address-cells* i *#size-cells* - ustawiają sposób adresowania urządzeń na magistrali SPI:
  - *#address-cells = <1>* - oznacza, że adresowanie urządzeń wymaga jednego numeru (np. jednego numeru chip-select),
  - #size-cells = <0> - oznacza, że urządzenia nie wymagają określania rozmiaru przestrzeni adresowej,
- *mcp4921: mcp4921@0* - definiuje węzeł urządzenia MCP4921 podłączonego do magistrali SPI, z adresem urządzenia 0 (chip-select 0).
- *reg = <0>* - określa numer pinu chip-select (CS),
- *spi-max-frequency = <100000>* - ustawia maksymalną częstotliwość pracy magistrali SPI dla urządzenia w hercach.
- *spi-bits-per-word = <8>* - określa, że transmisja danych odbywa się w 8-bitowych słowach.

W przypadku platformy Raspberry Pi Zero W, system operacyjny domyślnie udostępnia urządzenie *spidev*, za pomocą którego można korzystać z magistrali SPI z poziomu przestrzeni użytkownika. Niestety urządzenie *spidev* korzysta ze sprzętowych pinów CS, magistrali, co powoduje konflikt w przypadku dodania nowego urządzenia do magistrali.

W celu poprawnego utworzenia nowego urządzenia korzystającego z SPI, należy dezaktywować w systemie urządzenia *spidev*, za pomocą poniższego fragmentu nakładki drzewa urządzeń.

```
fragment@0 {
        target = <&spidev0>;
        __overlay__ {
                status = "disabled";
```

```
        };
};
```

fragment@0 wyłącza urządzenie spidev0, które domyślnie korzysta ze sprzętowego pinu CS0 w Raspberry Pi Zero W.

Zadanie – przygotowanie nakładki dla urządzenia MCP4921

1. Utworzyć plik nakładki drzewa urządzeń o nazwie *mcp4921.dts*.
2. Dodać do niego *fragment@0* służący do wyłączenia urządzenia *spidev0*.
3. Dodać *fragment@1*, który doda urządzenie *mcp4921*, a w nim:
   - właściwość *compatible = "microchip,mcp4921"*,
   - wybór pinu CS – numer *0*,
   - częstotliwość maksymalna magistrali – 100kHz,
   - długość słowa danych – *8* bitów.
4. Skompilować i załadować nakładkę do jądra.
5. Zamieścić poniżej kod nakładki.

```
/dts-v1/;
/plugin/;

/ {
        compatible = "brcm,bcm2835";
        fragment@0 {
                target = <&spidev0>;
                __overlay__ {
                        status = "disabled";
                };
        };

        fragment@1 {
                target = <&spi0>;
                __overlay__ {
                        status = "okay";
                        #address-cells = <1>;
                        #size-cells = <0>;

                        mcp4921: mcp4921@0{
                                compatible = "microchip,mcp4921";
                                status = "okay";
                                reg = <0>;
                                spi-max-frequency = <100000>;
                                spi-bits-per-word = <8>;
                        };
                };
        };
};
```

Przygotowanie minimalnego kodu sterownika SPI

Podobnie jak w przypadku sterownika platformowego, aby przygotować podstawowy kod sterownika SPI, należy:

- Utworzyć funkcje *probe* i *remove,* ich przykład przedstawiono poniżej.

```
static int mcp4921_probe(struct spi_device *dev)
{
        /* device init code*/
}

static void mcp4921_remove(struct spi_device *dev)
{
      /* device remove code*/
}
```

Funkcje dostarczają w argumencie wskaźnik na strukturę spi_device, która reprezentuje dodane urządzenie (podobnie jak *platform_device* w sterowniku platformowym).

- Utworzyć tablicę identyfikatorów urządzeń, dzięki której urządzenia zdefiniowane w drzewie urządzeń zostaną powiązane ze sterownikiem.

```
static const struct of_device_id mcp4921_of_id[] = {
        { .compatible = "microchip,mcp4921" },
        {},
};
MODULE_DEVICE_TABLE(of, mcp4921_of_id);
```

- Zainicjalizować strukturę typu *spi_driver* która służy do konfiguracji sterownika, poniżej przedstawiono jej przykład.

```
static struct spi_driver mcp4921_driver = {
        .probe = mcp4921_probe,
        .remove = mcp4921_remove,
        .driver = {
                .name = "mcp4921",
                .of_match_table = mcp4921_of_id,
                .owner = THIS_MODULE,
        },
};
module_spi_driver(mcp4921_driver);
```

Zadanie - przygotowanie minimalnego kodu sterownika SPI

1. Utworzyć plik źródłowy modułu o nazwie *mcp4921.c* i skopiować do niego poniższy kod.

```
#include <linux/spi/spi.h>

struct mcp4921 {
        struct spi_device *spidev;
};

static int mcp4921_probe(struct spi_device *dev)
{
        int err = 0;
        struct mcp4921 *mcp4921;

        mcp4921 = devm_kzalloc(&dev->dev, sizeof(struct mcp4921),
                               GFP_KERNEL);
        if (IS_ERR(mcp4921)) {
                dev_err(&dev->dev,
                        "Device data allocation failed.\n");
                err = PTR_ERR(mcp4921);
                goto out_ret_err;
        }

        mcp4921->spidev = dev;
        dev_set_drvdata(&dev->dev, mcp4921);

        dev_info(&dev->dev, "SPI DAC Driver Probed.\n");

out_ret_err:
        return err;
}

static void mcp4921_remove(struct spi_device *dev)
{
        struct mcp4921 *mcp4921 = dev_get_drvdata(&dev->dev);
        dev_info(&dev->dev, "SPI DAC Driver Removed.\n");
}

static const struct of_device_id mcp4921_of_id[] = {
        { .compatible = "microchip,mcp4921" },
        {},
};
MODULE_DEVICE_TABLE(of, mcp4921_of_id);

static struct spi_driver mcp4921_driver = {
        .probe = mcp4921_probe,
        .remove = mcp4921_remove,
        .driver = {
                .name = "mcp4921",
                .of_match_table = mcp4921_of_id,
                .owner = THIS_MODULE,
        },
};
module_spi_driver(mcp4921_driver);

MODULE_LICENSE("GPL v2");
```

2. Skompilować i załadować moduł do jądra.

3. Sprawdzić, czy w katalogu */proc/device-tree/soc/spi@7e204000/* pojawił się plik reprezentujący dodane urządzenie (*spi@7e204000* to kontroler *spi0*). Wynik zamieścić poniżej.

```
pi@pi:~ $ ls /proc/device-tree/soc/spi@7e204000/ | grep mcp4921
mcp4921@0
```

Komunikacja z przetwornikiem C/A

W celu ustawienia napięcia wyjściowego przetwornika MCP4921, należy wysłać do niego odpowiednią ramkę danych za pomocą magistrali SPI.

Możliwość wysyłania danych do urządzenia SPI zapewnia funkcja *spi_write*, której deklarację przedstawiono poniżej. [3]

```
static inline int spi_write(struct spi_device *spi,
                            const void *buf, size_t len);
```

Argumenty funkcji:

- **spi* – wskaźnik na urządzenie SPI typu *spi_device,*
- **buf* – wskaźnik na bufor z danymi do wysłania,
- *len* – liczba określająca ile danych z bufora ma zostać wysłane.

Funkcja zwraca *0* jeśli uda się poprawnie wysłać wszystkie dane, lub ujemny kod błędu w przeciwnym przypadku.

Zadanie - komunikacja z przetwornikiem C/A

1. Dodać funkcjonalność ustawiania napięcia na wyjściu przetwornika MCP4921, w tym celu:
    - Utworzyć funkcję *mcp4921_set,* która jako argumenty przyjmuje wskaźnik na urządzenie SPI (*struct spi_device*) oraz binarną wartość napięcia (0-4095).
    - W funkcji *mcp4921_set* przygotować ramkę danych dla przetwornika z następującymi ustawieniami: kanał A, buforowanie Vref wyłączone, wzmocnienie x1, przetwornik aktywny, wartość napięcia podana w argumencie funkcji *mcp4921_set.*
    - Za pomocą funkcji *spi_write* wysłać ramkę danych do przetwornika.
    - Funkcja powinna zwrócić wartość uzyskaną za pomocą *spi_write* (obsługa błędów).
2. W momencie dodawania urządzenia do systemu (funkcja *probe*) ustawić napięcie przetwornika na wartość binarną równą 2000, w tym celu wywołać funkcję *mcp4921_set* i dodać odpowiednią obsługę błędów.
3. Skompilować i załadować moduł do jądra.
4. Zamieścić poniżej zmierzoną wartość napięcia na wyjściu przetwornika.

```
1.60V
```

5. Zamieścić poniżej kod funkcji *mcp4921_set* i *mcp4921_probe*.

```
#define MCP4921_CFG_BITS (0x03 << 12)

static int mcp4921_set(struct spi_device *dev,
                       unsigned int voltage_12bit)
{
        unsigned int data = MCP4921_CFG_BITS |
                            (voltage_12bit & 0x0fff);
        unsigned char spi_buff[2];

        spi_buff[0] = ( (data & 0xff00) >> 8 );
        spi_buff[1] = ( data & 0x00ff );

        return spi_write(dev, spi_buff, 2);
}

static int mcp4921_probe(struct spi_device *dev)
{
        int err = 0;
        struct mcp4921 *mcp4921;

        mcp4921 = devm_kzalloc(&dev->dev, sizeof(struct mcp4921),
                               GFP_KERNEL);
        if (IS_ERR(mcp4921)) {
                dev_err(&dev->dev,
                        "Device data allocation failed.\n");
                err = PTR_ERR(mcp4921);
                goto out_ret_err;
        }

        mcp4921->spidev = dev;
        dev_set_drvdata(&dev->dev, mcp4921);

        err = mcp4921_set(dev, 2000);
        if (err) {
                dev_err(&dev->dev, "DAC set voltage failed.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "SPI DAC Driver Probed.\n");

out_ret_err:
        return err;
}
```

6. Dodać możliwość ustawiania napięcia za pomocą funkcji, która jako argument przyjmuje wartość napięcia w mV.

Ponieważ używanie liczb typu *float* nie jest zalecane ze względu na brak wsparcia układu jednostki zmiennoprzecinkowej w jądrze, poniższa funkcja konwertuje liczbę binarną na wartość napięcia za pomocą operacji na liczbach stałoprzecinkowych. [9]

```
#define BINARY_VAL_FOR_1mV 0x9ee0        /* unsigned Q1.15 -> 1,2412
                                            (1mV to wartość 1,2412
                                             dla przetwornika) */
static int mcp4921_set_mV(struct spi_device *dev,
                          unsigned int voltage_mV)
{
        unsigned int voltage_12bit = ((voltage_mV *
                                       BINARY_VAL_FOR_1mV) >> 15);

        return mcp4921_set(dev, voltage_12bit);
}
```

7. W momencie dodawania urządzenia do jądra ustawić wartość napięcia na 2000mV za pomocą funkcji *mcp4921_set_mV*.
8. Skompilować i załadować moduł do jądra. Zweryfikować poprawność działania sterownika i zamieścić poniżej wartość ustawionego napięcia.

```
2.00V
```

9. Zamieścić poniżej cały kod sterownika *mcp4921*.

```
#include <linux/spi/spi.h>

#define MCP4921_CFG_BITS (0x03 << 12)
#define BINARY_VAL_FOR_1mV 0x9ee0        /* unsigned Q1.15 -> 1,2412
                                            (1mV to wartość 1,2412
                                             dla przetwornika) */

struct mcp4921 {
        struct spi_device *spidev;
};

static int mcp4921_set(struct spi_device *dev,
                       unsigned int voltage_12bit)
{
        unsigned int data = MCP4921_CFG_BITS |
                            (voltage_12bit & 0x0fff);
        unsigned char spi_buff[2];

        spi_buff[0] = ( (data & 0xff00) >> 8 );
        spi_buff[1] = ( data & 0x00ff );

        return spi_write(dev, spi_buff, 2);
}
```

```
static int mcp4921_set_mV(struct spi_device *dev,
                          unsigned int voltage_mV)
{
        unsigned int voltage_12bit = ((voltage_mV *
                                       BINARY_VAL_FOR_1mV) >> 15);

        return mcp4921_set(dev, voltage_12bit);
}

static int mcp4921_probe(struct spi_device *dev)
{
        int err = 0;
        struct mcp4921 *mcp4921;

        mcp4921 = devm_kzalloc(&dev->dev, sizeof(struct mcp4921),
                               GFP_KERNEL);
        if (IS_ERR(mcp4921)) {
                dev_err(&dev->dev,
                        "Device data allocation failed.\n");
                err = PTR_ERR(mcp4921);
                goto out_ret_err;
        }

        mcp4921->spidev = dev;
        dev_set_drvdata(&dev->dev, mcp4921);

        err = mcp4921_set_mV(dev, 2000);
        if (err) {
                 dev_err(&dev->dev, "DAC set voltage failed.\n");
                 goto out_ret_err;
        }

        dev_info(&dev->dev, "SPI DAC Driver Probed.\n");

out_ret_err:
        return err;
}

static void mcp4921_remove(struct spi_device *dev)
{
        struct mcp4921 *mcp4921 = dev_get_drvdata(&dev->dev);
        dev_info(&dev->dev, "SPI DAC Driver Removed.\n");
}

static const struct of_device_id mcp4921_of_id[] = {
        { .compatible = "microchip,mcp4921" },
        {},
};
MODULE_DEVICE_TABLE(of, mcp4921_of_id);


static struct spi_driver mcp4921_driver = {
        .probe = mcp4921_probe,
        .remove = mcp4921_remove,
        .driver = {
                 .name = "mcp4921",
```

```
                .of_match_table = mcp4921_of_id,
                .owner = THIS_MODULE,
        },
};
module_spi_driver(mcp4921_driver);

MODULE_LICENSE("GPL v2");
```

Komunikacja "użytkownika" ze sterownikiem scalonego przetwornika C\A

Za pomocą atrybutu sysfs na przykładzie ustawiania napięcia przetwornika

1. Dodać możliwość ustawiania napięcia za pomocą atrybutu sysfs, w tym celu:
   - Zarejestrować atrybut sysfs tylko do zapisu, o nazwie *mcp4921_voltage.*
   - Dodać funkcję zapisu atrybutu o nazwie *mcp4921_voltage_store*.
   - W funkcji zapisu atrybutu:
     - uzyskać dostęp do prywatnych danych urządzenia,
     - przekonwertować dane z bufora *buf* na liczbę i zapisać w zmiennej *voltage_mV*,
     - sprawdzić czy wartość napięcia mieści się w zakresie 0-3300mV,
     - ustawić wartość napięcia za pomocą funkcji *mcp4921_set_mV.*
   - W funkcji *mcp4921_probe* utworzyć plik atrybutu za pomocą funkcji *device_create_file*.
   - W funkcji *mcp4921_remove* usunąć plik atrybutu za pomocą funkcji *device_remove_file*.
2. Skompilować i załadować moduł do jądra.
3. Przetestować działanie sterownika ustawiając wartość napięcia 1500mV i 4000mV za pomocą zapisu do pliku atrybutu urządzenia (). Poniżej zamieścić log z bufora diagnostycznego, wskazujący na zadziałanie obsługi błędów.

```
pi@pi:~ $ dmesg
[  116.827233] mcp4921 spi0.0: SPI DAC Driver Probed.
[  137.468431] mcp4921 spi0.0: Bad voltage value.
```

4. Zamieścić poniżej kod sterownika.

```
#include <linux/spi/spi.h>

#define MCP4921_REF_VOLTAGE_mV 3300
#define MCP4921_CFG_BITS (0x03 << 12)
#define BINARY_VAL_FOR_1mV 0x9ee0        /* unsigned Q1.15 -> 1,2412
                                            (1mV to wartość 1,2412
                                             dla przetwornika) */
```

```
struct mcp4921 {
        struct spi_device *spidev;
};

static int mcp4921_set(struct spi_device *dev,
                       unsigned int voltage_12bit)
{
        unsigned int data = MCP4921_CFG_BITS |
                            (voltage_12bit & 0x0fff);
        unsigned char spi_buff[2];
        spi_buff[0] = ( (data & 0xff00) >> 8 );
        spi_buff[1] = ( data & 0x00ff );
        return spi_write(dev, spi_buff, 2);
}

static int mcp4921_set_mV(struct spi_device *dev,
                          unsigned int voltage_mV)
{
        unsigned int voltage_12bit = ((voltage_mV *
                                       BINARY_VAL_FOR_1mV) >> 15);

        return mcp4921_set(dev, voltage_12bit);
}

static ssize_t mcp4921_voltage_store(struct device *dev,
                                     struct device_attribute *attr,
                                     const char *buf, size_t count)
{
        int err;
        int voltage_mV;
        struct mcp4921 *mcp4921 = dev_get_drvdata(dev);

        err = kstrtol(buf, 0, (long int*) &voltage_mV);
        if (err) {
                dev_err(dev, "Bad input number.\n");
                return err;
        }

        if (voltage_mV < 0 || voltage_mV > MCP4921_REF_VOLTAGE_mV) {
                dev_err(&mcp4921->spidev->dev,
                        "Bad voltage value.\n");
                return -EINVAL;
        }

        mcp4921_set_mV(mcp4921->spidev, voltage_mV);
        return count;
}
DEVICE_ATTR_WO(mcp4921_voltage);

static int mcp4921_probe(struct spi_device *dev)
{
        int err = 0;
        struct mcp4921 *mcp4921;

        mcp4921 = devm_kzalloc(&dev->dev, sizeof(struct mcp4921),
                               GFP_KERNEL);
```

```
        if (IS_ERR(mcp4921)) {
                dev_err(&dev->dev,
                        "Device data allocation failed.\n");
                err = PTR_ERR(mcp4921);
                goto out_ret_err;
        }

        mcp4921->spidev = dev;
        dev_set_drvdata(&dev->dev, mcp4921);

        err = mcp4921_set_mV(dev, 2000);
        if (err) {
                dev_err(&dev->dev, "DAC set voltage failed.\n");
                goto out_ret_err;
        }

        err = device_create_file(&dev->dev,
                                 &dev_attr_mcp4921_voltage);
        if (err) {
                dev_err(&dev->dev, "Device file creation failed.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "SPI DAC Driver Probed.\n");

out_ret_err:
        return err;
}

static void mcp4921_remove(struct spi_device *dev)
{
        struct mcp4921 *mcp4921 = dev_get_drvdata(&dev->dev);
        device_remove_file(&dev->dev, &dev_attr_mcp4921_voltage);
        dev_info(&dev->dev, "SPI DAC Driver Removed.\n");
}

static const struct of_device_id mcp4921_of_id[] = {
        { .compatible = "microchip,mcp4921" },
        {},
};
MODULE_DEVICE_TABLE(of, mcp4921_of_id);


static struct spi_driver mcp4921_driver = {
        .probe = mcp4921_probe,
        .remove = mcp4921_remove,
        .driver = {
                .name = "mcp4921",
                .of_match_table = mcp4921_of_id,
                .owner = THIS_MODULE,
        },
};
module_spi_driver(mcp4921_driver);

MODULE_LICENSE("GPL v2");
```

Za pomocą urządzenia znakowego – łańcuchy znakowe

Komunikację ze sterownikiem można również zrealizować za pomocą mechanizmu urządzenia znakowego, przez dostęp do pliku urządzenia w katalogu */dev.*

Urządzenia znakowe (również typu misc) posiadają możliwość przechowywania prywatnych danych, które są przypisane do struktury typu *struct file* reprezentującej otarty plik urządzenia (struktura ta jest dostarczana jako argument funkcji obsługujących operacje na plikach). Wewnątrz struktury typu *struct file* znajduje się wskaźnik typu *void* o nazwie *private_data,* który może zostać zainicjalizowany prywatnymi danymi urządzenia.

W przypadku urządzenia typu misc jądro zapewnia automatyczną obsługę funkcji *open* i *release* (funkcje operacji na plikach). Bardzo istotne jest, że w funkcji odpowiadającej za otwarcie pliku (*open*) wskaźnik *private_data* jest ustawiany na strukturę urządzenia misc (*struct miscdevice*).

Aby uzyskać dostęp do prywatnych danych urządzenia w funkcji obsługującej operację zapisu pliku urządzenia (*write*), należy skorzystać z makra c*ontainer_of*, za pomocą którego możliwe jest pozyskanie adresu struktury z danymi prywatnymi, pod warunkiem, że znajduje się tam zmienna typu *struct miscdevice*. Poniżej przedstawiono przykład użycia makra *container_of.*

```
struct mcp4921 {
        struct spi_device *spidev;
        struct miscdevice mdev;
};

/* funkcja obsługująca zapis pliku urządzenia znakowego */
static ssize_t mcp4921_write(struct file *filp, const char *buf,
                             size_t count, loff_t *f_pos)
{
        /* filp->private_data zawiera adres zmiennej mdev*/
        struct mcp4921 *mcp4921 = container_of(filp->private_data,
                                               struct mcp4921,
                                               mdev);
        /* ... */
}
```

1. Usunąć z kodu sterownika *mcp4921.c* część związaną z atrybutem sysfs.
2. Dodać do sterownika urządzenie znakowe w postaci *miscdevice,* w tym celu:
   - Do struktury prywatnych danych urządzenia dodać zmienną typu *struct miscdevice* o nazwie *mdev.*
   - Stworzyć funkcję obsługującą operację zapisu do pliku urządzenia znakowego, poniżej przedstawiono deklarację tej funkcji. Uzyskać dostęp do prywatnych danych urządzenia za pomocą makra *container_of.*

```
static ssize_t mcp4921_write(struct file *filp,
```

```
                                        const char *buf, size_t count,
                                        loff_t *f_pos);
```

- W funkcji *mcp4921_write* ustawić napięcie wyjściowe przetwornika C/A na wartość zapisaną w buforze *buf*. Dodać obsługę błędów (m.in. błędna wartość napięcia).
- Dodać strukturę typu *file_operations* o nazwie *fops* jak poniżej:

```
static struct file_operations fops = {
        .owner = THIS_MODULE,
        .write = mcp4921_write
};
```

- W momencie dodawania urządzenia do systemu (funkcja *probe*):
  - Zainicjalizować strukturę *mdev* następującymi wartościami:

```
/* miscdevice config */
mcp4921->mdev.minor = MISC_DYNAMIC_MINOR;
mcp4921->mdev.name = "mcp4921";
mcp4921->mdev.fops = &fops;
```

  - Zarejestrować urządzenie misc za pomocą funkcji *misc_register*.
- W momencie usuwania urządzenia z systemu (funkcja *remove*) wyrejestrować urządzenie misc za pomocą funkcji *misc_deregister*.

3. Skompilować i załadować moduł do jądra.
4. Przetestować działanie sterownika wpisując do pliku urządzenia wartość 1500 i 10000. Poniżej zamieścić log z informacją o błędzie.

```
pi@pi:~ $ dmesg
[  116.827233] mcp4921 spi0.0: SPI DAC Driver Probed.
[  137.468431] mcp4921 spi0.0: Bad voltage value.
```

5. Zamieścić kod sterownika poniżej.

```
#include <linux/spi/spi.h>
#include <linux/miscdevice.h>

#define MCP4921_REF_VOLTAGE_mV 3300
#define MCP4921_CFG_BITS (0x03 << 12)
#define MCP4921_MAX_WRITE_SIZE 8
#define BINARY_VAL_FOR_1mV 0x9ee0        /* unsigned Q1.15 -> 1,2412
                                            (1mV to wartość 1,2412
                                             dla przetwornika) */
struct mcp4921 {
        struct spi_device *spidev;
        struct miscdevice mdev;
};
```

```
int mcp4921_set(struct spi_device *dev, unsigned int voltage_12bit)
{
        unsigned int data = MCP4921_CFG_BITS |
                            (voltage_12bit & 0x0fff);
        unsigned char spi_buff[2];

        spi_buff[0] = ( (data & 0xff00) >> 8 );
        spi_buff[1] = ( data & 0x00ff );

        return spi_write(dev, spi_buff, 2);
}


int mcp4921_set_mV(struct spi_device *dev, unsigned int voltage_mV)
{
        /* Konwersja Q16.15 do Q16.0 */
        unsigned int voltage_12bit = ((voltage_mV *
                                       BINARY_VAL_FOR_1mV) >> 15);

        return mcp4921_set(dev, voltage_12bit);
}

static ssize_t mcp4921_write(struct file *filp, const char *buf,
                             size_t count, loff_t *f_pos)
{
        int err = 0;
        char kspace_buffer[MCP4921_MAX_WRITE_SIZE] = "";
        int voltage_mV;

        struct mcp4921 *mcp4921 = container_of(filp->private_data,
                                               struct mcp4921,
                                               mdev);

        dev_info(&mcp4921->spidev->dev, "Write to device file.\n");

        if (count > MCP4921_MAX_WRITE_SIZE) {
                dev_err(&mcp4921->spidev->dev,
                        "Bad input number.\n");
                return -ERANGE;
        }

        if (copy_from_user(kspace_buffer, buf, count)) {
                dev_err(&mcp4921->spidev->dev,
                        "Getting data from userspace failed.\n");
                return -EFAULT;
        }

        err = kstrtol(kspace_buffer, 0,
                      (unsigned long *) &voltage_mV);
        if (err) {
                dev_err(&mcp4921->spidev->dev,
                        "Bad input value.\n");
                return err;
        }
```

```
        if (voltage_mV < 0  ||  voltage_mV > MCP4921_REF_VOLTAGE_mV)
{
                dev_err(&mcp4921->spidev->dev,
                        "Bad voltage value.\n");
                return -EINVAL;
        }

        err = mcp4921_set_mV(mcp4921->spidev, voltage_mV);
        if (err) {
                dev_err(&mcp4921->spidev->dev,
                        "SPI communication failed.\n");
                return err;
        }

        return count;
}


static struct file_operations fops = {
        .owner = THIS_MODULE,
        .write = mcp4921_write
};


static int mcp4921_probe(struct spi_device *dev)
{
        int err = 0;
        struct mcp4921 *mcp4921;

        mcp4921 = devm_kzalloc(&dev->dev, sizeof(struct mcp4921),
                               GFP_KERNEL);
        if (IS_ERR(mcp4921)) {
                dev_err(&dev->dev,
                        "Device data allocation failed.\n");
                err = PTR_ERR(mcp4921);
                goto out_ret_err;
        }

        mcp4921->mdev.minor = MISC_DYNAMIC_MINOR;
        mcp4921->mdev.name = "mcp4921";
        mcp4921->mdev.fops = &fops;
        mcp4921->spidev = dev;
        dev_set_drvdata(&dev->dev, mcp4921);

        err = misc_register(&mcp4921->mdev);
        if (err) {
                dev_err(&dev->dev,
                        "Misc device registration failed.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "SPI DAC Driver Probed.\n");

out_ret_err:
        return err;
}
```

```
static void mcp4921_remove(struct spi_device *dev)
{
        struct mcp4921 *mcp4921 = dev_get_drvdata(&dev->dev);
        misc_deregister(&mcp4921->mdev);
        dev_info(&dev->dev, "SPI DAC Driver Removed.\n");
}

static const struct of_device_id mcp4921_of_id[] = {
        { .compatible = "microchip,mcp4921" },
        {},
};
MODULE_DEVICE_TABLE(of, mcp4921_of_id);

static struct spi_driver mcp4921_driver = {
        .probe = mcp4921_probe,
        .remove = mcp4921_remove,
        .driver = {
                .name = "mcp4921",
                .of_match_table = mcp4921_of_id,
                .owner = THIS_MODULE,
        },
};
module_spi_driver(mcp4921_driver);

MODULE_LICENSE("GPL v2");
```

Za pomocą urządzenia znakowego – mechanizm ioctl

Przetwornik MCP4921 posiada cztery bity konfiguracyjne, które pozwalają zmienić pewne jego właściwości. Aby przekonfigurować ustawienia układu za pomocą operacji zapisu do pliku urządzenia (w funkcji *write*) należałoby dodać obsługę dekodowania komunikatów wpisywanych do pliku. Takie rozwiązanie jest czasochłonne i słabo skalowalne. Do celów komunikacji ze sterownikiem za pomocą „komend” binarnych (zamiast ciągów znaków) idealnie nadaje się mechanizm ioctl.

Mechanizm ioctl (Input/Output Control) w sterownikach znakowych pozwala na przesyłanie niestandardowych poleceń pomiędzy przestrzenią użytkownika a sterownikiem jądra. Dzięki temu umożliwia realizację operacji, które wykraczają poza standardowe funkcje wejścia/wyjścia, takie jak odczyt (read) czy zapis (write).

Dodanie obsługi ioctl wymaga zdefiniowania własnych „poleceń”, które będą rozpoznawane przez sterownik, służą do tego podane poniżej makra. [3]

```
#define _IO(type, nr)                   // Operacja bez danych
#define _IOR(type, nr, datatype)        // Odczyt danych
#define _IOW(type, nr, datatype)        // Zapis danych
#define _IOWR(type, nr, datatype)       // Odczyt i zapis danych
```

Argumenty makrodefinicji:

- *type* – tzw. *magic number*, unikalny identyfikator dla poleceń związanych z danym sterownikiem,
- *nr* – numer polecenia w danym sterowniku,
- *datatype* – typ danych przekazywanych w poleceniu.

Za obsługę zdefiniowanych komend odpowiada jedna z funkcji obsługująca wywołanie systemowe ioctl. Jej przykładowa definijca została przedstawiona poniżej.

```
static long mcp4921_ioctl(struct file *filp, unsigned int cmd,
                          unsigned long arg)
{
        struct mcp4921 *mcp4921 = container_of(filp->private_data,
                                               struct mcp4921,
                                               mdev);

        switch(cmd) {
        case MCP4921_RESET:
                dev_info(&mcp4921->spidev->dev,
                         "MCP4921_RESET command\n");

                mcp4921->vref_buf_bit = 0;
                mcp4921->gain_bit = 1;
                mcp4921->enable_bit = 1;
                break;
        /* ... */
}
```

Argumenty funkcji:

- **filp* – wskaźnik na strukturę reprezentującą otwarty plik urządzenia,
- *cmd* – kod polecenia ioctl,
- *arg* - argument przekazywany z przestrzeni użytkownika do jądra.

Funkcja powinna zwracać wartość *0* w przypadku poprawnego wykonania, lub ujemny kod błędu.

Zadanie – dodanie obsługi ioctl

1. Przygotować plik nagłówkowy z definicjami poleceń ioctl.
    - Utworzyć plik *mcp4921_commands.h*.
    - Za pomocą przedstawionych we wstępie makr utworzyć następujące polecenia ioctl:
        - *MCP4921_RESET* – ustawia domyślne wartości bitów konfiguracyjnych (SHDN=1, GA=1, BUF=0),
        - *MCP4921_ENABLE* – ustawia bit SHDN przetwornika na wartość podaną w argumencie,
        - *MCP4921_GAIN* – ustawia bit GA przetwornika na wartość podaną w argumencie,

- *MCP4921_VREF_BUFF* - ustawia bit BUF przetwornika na wartość podaną w argumencie.

Jako *magic number* podać wartość *'k'*, a typ przekazywanych argumentów ustawić na *int*.

2. Zamieścić poniżej definicje poleceń ioctl.

```
#define MCP4921_RESET _IO('k', 0)
#define MCP4921_ENABLE _IOW('k', 1, int)
#define MCP4921_GAIN _IOW('k', 2, int)
#define MCP4921_VREF_BUFF _IOW('k', 3, int)
```

3. Dodać do sterownika *mcp4921* obsługę poleceń ioctl, w tym celu:
   - Utworzyć funkcję *mcp4921_ioctl,* w której obsłużyć zdefiniowane polecenia ioctl, zapisując do bufora diagnostycznego informację o wywołanym poleceniu, zgodnie z poniższym przykładem:

```
case MCP4921_ENABLE:
      dev_info(&mcp4921->spidev->dev,
               "MCP4921_ENABLE command\n");
      break;
```

   - Dodać funkcję *mcp4921_ioctl* do inicjalizacji struktury typu *file_operations* zgodnie z poniższym przykładem:

```
static struct file_operations fops = {
      .owner=THIS_MODULE,
      .write=mcp4921_write,
      .unlocked_ioctl = mcp4921_ioctl
};
```

4. Dodać „dynamiczną" obsługę bitów konfiguracyjnych przetwornika, w tym celu:
   - Do struktury prywatnych danych dodać zmienne reprezentujące bity konfiguracyjne przetwornika C/A:
     - *bool vref_buf_bit*
     - *bool gain_bit*
     - *bool enable_bit*
   - W funkcji *mcp4921_set* wykorzystać utworzone zmienne konfiguracyjne do przygotowania ramki danych dla przetwornika.
   - W funckji *mcp4921*_ioctl:
     - *U*zyskać dostęp do prywatnych danych urządzenia za pomocą makra *container_of.*

- W obsłudze odpowiednich poleceń dodać weryfikację poprawności argumentu *arg* (sprawdzić czy wartość jest właściwa) i ustawić zmienne konfiguracyjne.

5. Utworzyć plik programu działającego w przestrzeni użytkownika o nazwie userspace_mcp4921.c i skopiować do niego poniższy kod.
   Program ten ma za zadanie skomunikować się ze sterownikiem jądra za pomocą interfejsu ioctl i skonfigurować przetwornik C/A.

```
#include <fcntl.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <stdio.h>
#include "mcp4921_commands.h"

int main()
{
        int fd;
        int arg;
        char buff[] = "1000";

        fd = open("/dev/mcp4921", O_RDWR);
        if (fd < 0) {
                printf("Cannot open device file.\n");
                return fd;
        }

        printf("Driver open\n");
        ioctl(fd, MCP4921_RESET, 0);
        arg = 1;
        ioctl(fd, MCP4921_ENABLE, arg);
        ioctl(fd, MCP4921_GAIN, arg);
        arg = 0;
        ioctl(fd, MCP4921_VREF_BUFF, arg);
        write(fd, buff, sizeof(buff));
        printf("Driver close\n");
        close(fd);

        return 0;
}
```

6. Skompilować sterownik *mcp4921* i załadować go do jądra.
7. Skompilować program *userspace_mcp4921.c* za pomocą polecenia *gcc*.
8. Uruchomić program *userspace_mcp4921.c* i odczytać zawartość bufora diagnostycznego. Wynik zamieścić poniżej.

```
pi@pi:~ $ dmesg
[10479.790849] mcp4921 spi0.0: SPI DAC Driver Probed.
[10481.472796] mcp4921 spi0.0: MCP4921_RESET command.
[10481.479788] mcp4921 spi0.0: MCP4921_ENABLE command.
[10481.494650] mcp4921 spi0.0: MCP4921_GAIN command.
[10481.500715] mcp4921 spi0.0: Set GAIN bit to: 1
[10481.510979] mcp4921 spi0.0: MCP4921_VREF_BUFF command.
[10481.517502] mcp4921 spi0.0: Set VREF_BUFF bit to: 0
[10481.533112] mcp4921 spi0.0: Write to device file.
```

9. Zamieścić poniżej kod sterownika *mcp4921*.

```
#include <linux/spi/spi.h>
#include <linux/miscdevice.h>
#include "mcp4921_commands.h"

#define MCP4921_REF_VOLTAGE_mV 3300
#define MCP4921_CFG_BITS (0x03 << 12)
#define MCP4921_MAX_WRITE_SIZE 8
#define BINARY_VAL_FOR_1mV 0x9ee0        /* unsigned Q1.15 -> 1,2412
                                            (1mV to wartość 1,2412
                                            dla przetwornika) */

struct mcp4921 {
        struct spi_device *spidev;
        struct miscdevice mdev;
        int voltage_mV;
        bool vref_buf_bit;
        bool gain_bit;
        bool enable_bit;
};

int mcp4921_set(struct mcp4921 *dev_data,
                unsigned int voltage_12bit)
{
        unsigned int cfg_bits = (dev_data->vref_buf_bit << 14) |
                                (dev_data->gain_bit << 13) |
                                (dev_data->enable_bit << 12);
        unsigned int data = cfg_bits | (voltage_12bit & 0x0fff);
        unsigned char spi_buff[2];

        spi_buff[0] = ( (data & 0xff00) >> 8 );
        spi_buff[1] = ( data & 0x00ff );
        return spi_write(dev_data->spidev, spi_buff, 2);
}

int mcp4921_set_mV(struct mcp4921 *dev_data,
                   unsigned int voltage_mV)
{
        /* Konwersja Q16.15 do Q16.0 */
        unsigned int voltage_12bit = (((unsigned long)voltage_mV *
                        (unsigned long)BINARY_VAL_FOR_1mV) >> 15);
        return mcp4921_set(dev_data, voltage_12bit);
}
```

```
static long mcp4921_ioctl(struct file *filp, unsigned int cmd,
                          unsigned long arg)
{

        struct mcp4921 *mcp4921 = container_of(filp->private_data,
                                               struct mcp4921,
                                               mdev);

        switch(cmd) {
        case MCP4921_RESET:
                dev_info(&mcp4921->spidev->dev,
                         "MCP4921_RESET command.\n");

                mcp4921->vref_buf_bit = 0;
                mcp4921->gain_bit = 1;
                mcp4921->enable_bit = 1;
                break;

        case MCP4921_ENABLE:
                dev_info(&mcp4921->spidev->dev,
                         "MCP4921_ENABLE command.\n");

                if (arg != 0 && arg != 1) {
                        dev_err(&mcp4921->spidev->dev,
                                "ENABLE bit wrong value.\n");
                        return -EINVAL;
                }

                mcp4921->enable_bit = arg;
                break;

        case MCP4921_GAIN:
                dev_info(&mcp4921->spidev->dev,
                         "MCP4921_GAIN command.\n");

                if (arg != 0 && arg != 1) {
                        dev_err(&mcp4921->spidev->dev,
                                "GAIN bit wrong value.\n");
                        return -EINVAL;
                }

                dev_info(&mcp4921->spidev->dev,
                         "Set GAIN bit to: %lu\n", arg);
                mcp4921->gain_bit = arg;
                break;

        case MCP4921_VREF_BUFF:
                dev_info(&mcp4921->spidev->dev,
                         "MCP4921_VREF_BUFF command.\n");

                if (arg != 0 && arg != 1) {
                        dev_err(&mcp4921->spidev->dev,
                                "VREF_BUFF bit wrong value.\n");
                        return -EINVAL;
                }
```

```
                dev_info(&mcp4921->spidev->dev,
                         "Set VREF_BUFF bit to: %lu\n", arg);
                mcp4921->vref_buf_bit = arg;
                break;

        default:
                break;
        }
        return 0;
}

static ssize_t mcp4921_write(struct file *filp, const char *buf,
                             size_t count, loff_t *f_pos)
{
        int err = 0;
        char kspace_buffer[MCP4921_MAX_WRITE_SIZE] = "";
        int voltage_mV;
        struct mcp4921 *mcp4921 = container_of(filp->private_data,
                                               struct mcp4921,
                                               mdev);
        dev_info(&mcp4921->spidev->dev, "Write to device file.\n");

        if (count > MCP4921_MAX_WRITE_SIZE) {
                dev_err(&mcp4921->spidev->dev,
                        "Bad input number.\n");
                return -ERANGE;
        }

        if (copy_from_user(kspace_buffer, buf, count)) {
                dev_err(&mcp4921->spidev->dev,
                        "Getting data from userspace failed.\n");
                return -EFAULT;
        }

        err = kstrtol(kspace_buffer, 0,
                      (unsigned long *) &voltage_mV);
        if (err) {
                dev_err(&mcp4921->spidev->dev,
                        "Bad input value.\n");
                return err;
        }

        if (voltage_mV < 0  ||
            voltage_mV > MCP4921_REF_VOLTAGE_mV) {
                dev_err(&mcp4921->spidev->dev,
                        "Bad voltage value.\n");
                return -EINVAL;
        }

        err = mcp4921_set_mV(mcp4921, voltage_mV);
        if (err) {
                dev_err(&mcp4921->spidev->dev,
                        "SPI communication failed.\n");
                return err;
        }
```

```
        return count;
}


static struct file_operations fops = {
        .owner = THIS_MODULE,
        .write = mcp4921_write,
        .unlocked_ioctl = mcp4921_ioctl
};


static int mcp4921_probe(struct spi_device *dev)
{
        int err = 0;
        struct mcp4921 *mcp4921;

        mcp4921 = devm_kzalloc(&dev->dev, sizeof(struct mcp4921),
                               GFP_KERNEL);
        if (IS_ERR(mcp4921)) {
                dev_err(&dev->dev,
                        "Device data allocation failed.\n");
                err = PTR_ERR(mcp4921);
                goto out_ret_err;
        }

        mcp4921->mdev.minor = MISC_DYNAMIC_MINOR;
        mcp4921->mdev.name = "mcp4921";
        mcp4921->mdev.fops = &fops;
        mcp4921->spidev = dev;
        dev_set_drvdata(&dev->dev, mcp4921);

        err = misc_register(&mcp4921->mdev);
        if (err) {
                dev_err(&dev->dev,
                        "Misc device registration failed.\n");
                goto out_ret_err;
        }

        dev_info(&dev->dev, "SPI DAC Driver Probed.\n");

out_ret_err:
        return err;
}

static void mcp4921_remove(struct spi_device *dev)
{
        struct mcp4921 *mcp4921 = dev_get_drvdata(&dev->dev);
        misc_deregister(&mcp4921->mdev);
        dev_info(&dev->dev, "SPI DAC Driver Removed.\n");
}

static const struct of_device_id mcp4921_of_id[] = {
        { .compatible = "microchip,mcp4921" },
        {},
};
```

```
MODULE_DEVICE_TABLE(of, mcp4921_of_id);

static struct spi_driver mcp4921_driver = {
        .probe = mcp4921_probe,
        .remove = mcp4921_remove,
        .driver = {
                .name = "mcp4921",
                .of_match_table = mcp4921_of_id,
                .owner = THIS_MODULE,
        },
};
module_spi_driver(mcp4921_driver);

MODULE_LICENSE("GPL v2");
```

Komunikacja "sterownika" ze scalonym ekspanderem portów I/O z magistralą SPI z użyciem mechanizmu REGMAP

Mechanizm *regmap* w jądrze Linux to uniwersalny interfejs ułatwiający obsługę mapowania rejestrów urządzeń. Jest szczególnie przydatny dla sterowników obsługujących urządzenia komunikujące się przez magistrale, takie jak I2C czy SPI.

Regmap zapewnia zunifikowane funkcje do obsługi komunikacji, niezależnie od używanej magistrali. Umożliwia to łatwiejszą migrację pomiędzy interfejsami takimi jak np. SPI i I2C.

Dodatkowo mechanizm *regmap* zapewnia obsługę weryfikacji poprawności adresacji rejestrów, które mają być zapisywane lub odczytywane. Służą do tego funkcje, których przykład przedstawiono poniżej.

```
static bool writeable_reg(struct device *dev, unsigned int reg)
{
        /* reg verification */
}

static bool readable_reg(struct device *dev, unsigned int reg)
{
        /* reg verification */
}
```

Argumenty funkcji:

- **dev* – wskaźnik na urządzenie, z którym skojarzony jest regmap,
- *reg* – adres do zweryfikowania.

Funkcje powinny zwrócić wartość *1* w przypadku gdy adres jest poprawny lub wartość *0* jeśli adres jest błędny.

W celu skonfigurowania mechanizmu *regmap* należy odpowiednio zainicjalizować strukturę typu *regmap_config*, której najistotniejsze pola przedstawiono poniżej.

```
struct regmap_config {
      int reg_bits;
      int val_bits;
      bool (*writeable_reg)(struct device *dev,
                            unsigned int reg);
      bool (*readable_reg)(struct device *dev,
                           unsigned int reg);
      unsigned long read_flag_mask;
      unsigned long write_flag_mask;
/* … */
}
```

Opis zawartości struktury:

- *reg_bits* – liczba bitów adresu rejestru,
- *val_bits* – liczba bitów rejestru,
- **writable_reg, *readable_reg* – wskaźniki na funkcje, których zadaniem jest zweryfikować poprawność adresów rejestrów,
- *read_flag_mask, write_flag_mask* – maski bitowe służące do modyfikacji wartości rejestru przez dodanie flag wymaganych przez magistralę lub urządzenie (np. przy I2C – bit RW)

Po przygotowaniu struktury konfiguracyjnej, należy zainicjalizować *regmap* za pomocą makra *devm_regmap_init_spi* (lub *devm_regmap_init_i2c*), które przedstawiono poniżej.

```
#define devm_regmap_init_spi(dev, config) ...
```

Argumenty makrodefinicji:

- *dev* – wskaźnik na urządzenie SPI (zmienna typu *struct spi_device*),
- *config* – wskaźnik na strukturę konfiguracyjną typu *struct regmap_config*.

Makro zwraca adres instancji *regmap* w przypadku poprawnego wykonania, w przeciwnym razie zwraca kod błędu w postaci wskaźnika. Użycie powyższego makra zwalnia z konieczności ręcznego usuwania instancji *regmap*, jądro automatycznie zwolni zasoby w momencie usuwania urządzenia.

Zapis i odczyt rejestru realizowany jest za pomocą poniższych funkcji. [3]

```
int regmap_write(struct regmap *map, unsigned int reg,
                 unsigned int val);

int regmap_read(struct regmap *map, unsigned int reg,
                unsigned int *val);
```

Argumenty funckji:

- **map* – wskaźnik na strukturę reprezentującą regmap (*struct regmap*),
- *reg* – adres rejestru, pod który ma być zapisana wartość,

- *val* – wartość jaka ma zostać zapisana do rejestru,
- **val* – adres zmiennej, do której ma zostać zapisana odczytana wartość rejestru.

Komunikacja "sterownika" z układem, na przykładzie z magistralą SPI i układem scalonym o architekturze rejestrowej

MCP23S09 to 8-bitowy ekspander portów I/O komunikujący się przez magistralę SPI (Serial Peripheral Interface). Układ posiada rejestry konfiguracyjne, które umożliwiają m.in. na ustawienie kierunku pinów, polaryzacji wejść czy skonfigurowanie rezystorów podciągających (pull-up).

Poniżej przedstawiono mapę rejestrów układu MCP23S09. [10]

TABLE 1-4: CONFIGURATION AND CONTROL REGISTERS

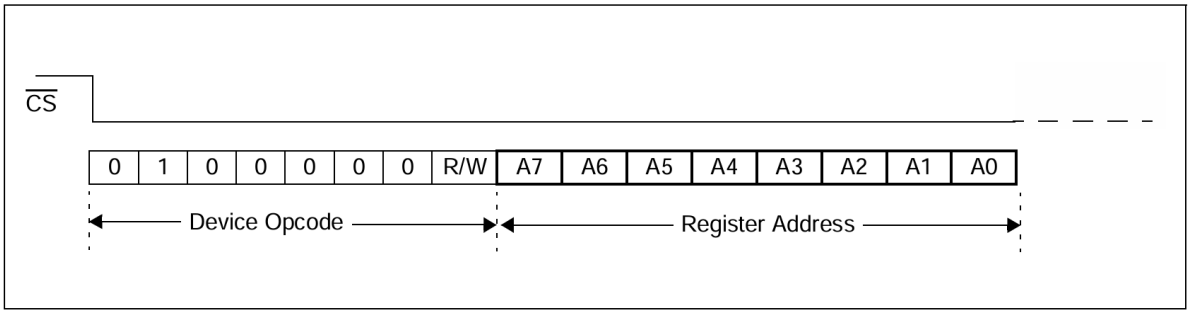
Register Name	Address (hex)	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	POR/RST Value
IODIR	00	IO7	IO6	IO5	IO4	IO3	IO2	IO1	IO0	1111 1111
IPOL	01	IP7	IP6	IP5	IP4	IP3	IP2	IP1	IP0	0000 0000
GPINTEN	02	GPINT7	GPINT6	GPINT5	GPINT4	GPINT3	GPINT2	GPINT1	GPINT0	0000 0000
DEFVAL	03	DEF7	DEF6	DEF5	DEF4	DEF3	DEF2	DEF1	DEF0	0000 0000
INTCON	04	IOC7	IOC6	IOC5	IOC4	IOC3	IOC2	IOC1	IOC0	0000 0000
IOCON	05	—	—	SEQOP	—	—	ODR	INTPOL	INTCC	0000 0000
GPPU	06	PU7	PU6	PU5	PU4	PU3	PU2	PU1	PU0	0000 0000
INTF	07	INT7	INT6	INT5	INT4	INT3	INT2	INT1	INTO	0000 0000
INTCAP	08	ICP7	ICP6	ICP5	ICP4	ICP3	ICP2	ICP1	ICP0	0000 0000
GPIO	09	GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0	0000 0000
OLAT	0A	OL7	OL6	OL5	OL4	OL3	OL2	OL1	OL0	0000 0000

Najważniejsze rejestry to:

- IODIR – rejestr kontrolujący kierunek pinów ekspandera, *(1 – wejście, 0 – wyjście)*,
- GPIO – rejestr kontrolujący stan pinów ekspandera, *(1 – stan wysoki, 0 – stan niski)*.

Sposób adresowania rejestrów przedstawia poniższa grafika. [10]

FIGURE 1-9: SPI ADDRESSING REGISTERS



Ramka adresująca składa się z 8 bitów kodu operacji (w tym 1 bit oznaczający zapis lub odczyt) oraz adresu rejestru, którego dotyczy operacja. Kod operacji dla płytki zestawu do ćwiczeń, przedstawia poniższa definicja.

```
#define MCP23S09_OPCODE 0x40
```

Ustawienie kierunku portu realizowane jest za pomocą poniższej ramki SPI:
[`MCP23S09_OPCODE + 0`] [adres rejestru IODIR] [wartość do zapisania]

Ustawienie stanu portu realizowane jest za pomocą poniższej ramki SPI:
[`MCP23S09_OPCODE + 0`] [adres rejestru GPIO] [wartość do zapisania]

Odczyt wartości portu odbywa się przy pomocy poniższej ramki SPI:
[`MCP23S09_OPCODE + 1`] [adres rejestru GPIO]

Przed odczytaniem stanu portu należy ustawić kierunek portu jako wejście.

Zadanie – obsługa komunikacji SPI za pomocą mechanizmu regmap

1. Utwórz plik nakładki drzewa urządzeń o nazwie *mcp23s09.dts* i skopiuj do niego poniższy kod.

```
/dts-v1/;
/plugin/;

/ {
        compatible = "brcm,bcm2835";
        fragment@0 {
                target = <&spidev0>;
                __overlay__ {
                        status = "disabled";
                };
        };

        fragment@1 {
                target = <&spi0>;
                __overlay__ {
                        status = "okay";
                        #address-cells = <1>;
                        #size-cells = <0>;

                        mcp23s09: mcp23s09@0{
                                compatible = "microchip,mcp23s09";
                                status = "okay";
                                reg = <0>;
                                spi-max-frequency = <100000>;
                                spi-bits-per-word = <8>;
                        };
```

```
                };
        };
};
```

2. Otworzyć plik modułu o nazwie *mcp23s09.c*, który znajduje się w załącznikach.
3. Skonfigurować .mechanizm *regmap*, w tym celu:
    - Stworzyć funkcje o nazwie *writeable_reg* i *readable_reg*, które sprawdzą poprawność podanych adresów. Definicje funkcji podano poniżej.

```
static bool writeable_reg(struct device *dev, unsigned int reg)
{
        switch (reg) {
        case (MCP23S09_OPCODE << 8 | MCP23S09_DIR_REG):
        case (MCP23S09_OPCODE << 8 | MCP23S09_GPIO_REG):
                return true;
        }
        return false;
}

static bool readable_reg(struct device *dev, unsigned int reg)
{
        return (reg == (MCP23S09_OPCODE << 8 | MCP23S09_GPIO_REG));
}
```

- W momencie dodawania urządzenia do systemu (funkcja *probe*) skonfigurować parametry mechanizmu *regmap,* w tym celu:
    - Dodać zmienną typu *struct regmap_config* o nazwie reg_conf.
    - Zainicjalizować zmienną *reg_conf* za pomocą funkcji *memset*, której sposób użycia przedstawiono poniżej. Funkcja wypełnia podaną strukturę zerami.

```
memset(&reg_conf, 0, sizeof(reg_conf));
```

    - Ustawić zmienne konfiguracyjne zanjdujące się wewnątrz struktury reg_conf w następujący sposób:

```
reg_conf.reg_bits = 16;
reg_conf.val_bits = 8;
reg_conf.writeable_reg = writeable_reg;
reg_conf.readable_reg = readable_reg;
reg_conf.read_flag_mask = 0x01;
```

Ponieważ ramka adresująca rejestr układu MCP23S09 ma 2 bajty, wartość *reg_bits* została ustawiona na 16 bitów.
Maska flagi zapisu została ustawiona na *0x01*, co pozwala mechanizmowi *regmap* na automatyczne ustawianie bitu RW w ramce adresującej."

- Zarejestrować *regmap* za pomocą makrodefinicji *devm_regmap_init_spi,* zapisać zwróconą wartość do zmiennej *rmap* (znajdującej się w prywatnych danych urządzenia). Dodać obsługę błędów.

4. Stworzyć funkcje *mcp23s09_set_port*, która ustawi stan portu na podaną w argumencie wartosć. W tym celu:
   - Dodać poniższą definicję funkcji:

```
int mcp23s09_set_port(struct mcp23s09 *mcp23s09,
                      unsigned int port_value)
{
        /* ... */
}
```

   - Ustawić kierunek portu jako wyjście (*0x00*), wysyłając do układu MCP23S09 odpowiednią ramkę danych, za pomocą funkcji *regmap_write* (pamiętać, że adres jest 16-bitowy). Dodać obsługę błędów.
   - Analogicznie jak w poprzednim punkcie, ustawić stan portu, zmieniając adres rejestru i wartość do zapisania.
5. Stworzyć funkcje *mcp23s09_get_port*, która odczyta stan portu i zapisze go pod wskazanym w argumencie adresie. W tym celu:
   - Dodać poniższą definicję funkcji:

```
int mcp23s09_get_port(struct mcp23s09 *mcp23s09,
                      unsigned int *value)
{
        /* ... */
}
```

   - Ustawić kierunek portu jako wejście (*0xff*), wysyłając do układu MCP23S09 odpowiednią ramkę danych, za pomocą funkcji *regmap_write* (pamiętać, że adres jest 16-bitowy). Dodać obsługę błędów.
   - Odczytać wartość rejestru GPIO układu MCP23S09 za pomocą funckji *regmap_read* i zapisać pod adres podany wewskaźniku **value.*

6. Skompilować kod sterownika i nakładki drzewa urządzeń.
7. Załadować nakładkę do jądra.
8. Dodać sterownik do jądra, w tym celu najpierw załadować moduł *regmap-spi*, który jest wymagany do działania mechanizmu *regmap.* Użyć poniższego polecenia.

```
sudo modprobe regmap-spi
```

Polecenie *modprobe* ładuje wskazany moduł i zależności z których on korzysta (inne wymagane moduły).

9. Przetestować funkcjonalność odczytu ustawiając losowe wartości portu za pomocą przełączników dostępnych na płytce zestawu do ćwiczeń i odczytując plik */dev/mcp23s09*.
   Przetestować funkcjonalność ustawiania wartości portu za pomocą wpisywania danych do pliku urządzenia.
10. Zamieścić poniżej odpowiednie wydruki z konsoli i bufora diagnostycznego.

```
pi@pi:~ $ sudo sh -c "echo 0x00 >> /dev/mcp23s09"
pi@pi:~ $ sudo sh -c "echo 0xff >> /dev/mcp23s09"
pi@pi:~ $ sudo head -n1 /dev/mcp23s09
xE2
pi@pi:~ $ sudo head -n1 /dev/mcp23s09
x80
pi@pi:~ $ sudo head -n1 /dev/mcp23s09
x55

pi@pi:~ $ dmesg
[25526.772158] mcp23s09 spi0.0: SPI IO Driver Probed.
[25537.486911] mcp23s09 spi0.0: Write to device file.
[25537.493348] mcp23s09 spi0.0: Set port value.
[25544.254888] mcp23s09 spi0.0: Write to device file.
[25544.261542] mcp23s09 spi0.0: Set port value.
[25554.981272] mcp23s09 spi0.0: Read from device file.
[25554.987786] mcp23s09 spi0.0: Get port value.
[25564.728304] mcp23s09 spi0.0: Read from device file.
[25564.735128] mcp23s09 spi0.0: Get port value.
[25584.902424] mcp23s09 spi0.0: Read from device file.
[25584.909274] mcp23s09 spi0.0: Get port value.
```

11. Zamieścić poniżej kod sterownika.

```
#include <linux/miscdevice.h>
#include <linux/spi/spi.h>
#include <linux/regmap.h>

#define MCP23S09_OPCODE 0x40
#define MCP23S09_DIR_REG 0x00
#define MCP23S09_GPIO_REG 0x09
#define MCP23S09_MAX_WRITE_SIZE 7
#define MCP23S09_MAX_READ_SIZE 7


struct mcp23s09 {
        struct spi_device *spidev;
        struct regmap *rmap;
        struct miscdevice mdev;
        int port_state;
};
```

```
int mcp23s09_set_port(struct mcp23s09 *mcp23s09,
                      unsigned int port_value)
{
        int err = 0;
        dev_info(&mcp23s09->spidev->dev, "Set port value.\n");

        /* Ustawienie kierunku portu jako wyjście */
        err = regmap_write(mcp23s09->rmap, (MCP23S09_OPCODE << 8 |
                           MCP23S09_DIR_REG), 0x00);
        if (err) {
                dev_err(&mcp23s09->spidev->dev,
                        "SPI Communication write error.\n");
                return err;
        }

        /* Ustawienie wartości portu */
        err = regmap_write(mcp23s09->rmap,
                          (MCP23S09_OPCODE << 8 | MCP23S09_GPIO_REG),
                           port_value);
        if (err) {
                dev_err(&mcp23s09->spidev->dev,
                        "SPI Communication write error.\n");
                return err;
        }

        return 0;
}

int mcp23s09_get_port(struct mcp23s09 *mcp23s09,
                      unsigned int *value)
{
        int err = 0;
        dev_info(&mcp23s09->spidev->dev, "Get port value.\n");

        /* Ustawienie kierunku portu jako wejście */
        err = regmap_write(mcp23s09->rmap, (MCP23S09_OPCODE << 8 |
                           MCP23S09_DIR_REG), 0xff);
        if (err) {
                dev_err(&mcp23s09->spidev->dev,
                        "SPI Communication write error.\n");
                return err;
        }

        /* Odczyt wartości portu */
        err = regmap_read(mcp23s09->rmap,
                          (MCP23S09_OPCODE << 8 | MCP23S09_GPIO_REG),
                           value);
        if (err) {
                dev_err(&mcp23s09->spidev->dev,
                        "SPI Communication read error.\n");
                return err;
        }

        return 0;
}
```

```
static ssize_t mcp23s09_write(struct file *filp, const char *buf,
                              size_t count, loff_t *f_pos)
{
        int err = 0;
        struct mcp23s09 *mcp23s09 = container_of(filp->private_data,
                                                 struct mcp23s09,
                                                 mdev);
        char kspace_buffer[MCP23S09_MAX_WRITE_SIZE] = "";
        int port_state;
        dev_info(&mcp23s09->spidev->dev, "Write to device file.\n");

        if (count > MCP23S09_MAX_WRITE_SIZE) {
                dev_err(&mcp23s09->spidev->dev,
                        "Bad input number.\n");
                return -ERANGE;
        }

        if (copy_from_user(kspace_buffer, buf, count)) {
                dev_err(&mcp23s09->spidev->dev,
                        "Getting data from userspace failed.\n");
                return -EFAULT;
        }

        err = kstrtol(kspace_buffer, 0, (long *) &port_state);
        if (err) {
                dev_err(&mcp23s09->spidev->dev,
                        "Bad input number.\n");
                return err;
        }

        if (port_state < 0x00  ||  port_state > 0xff) {
                dev_err(&mcp23s09->spidev->dev,
                        "Bad voltage value\n");
                return -EINVAL;
        }

        err = mcp23s09_set_port(mcp23s09, port_state);
        if (err) {
               dev_err(&mcp23s09->spidev->dev,
                       "Setting port value failed.\n");
               return -EIO;
        }

        return count;
}


static ssize_t mcp23s09_read(struct file *filp, char *buf,
                             size_t count, loff_t *f_pos)
{
        int err = 0;
        unsigned char port_buf[MCP23S09_MAX_READ_SIZE];

        struct mcp23s09 *mcp23s09 = container_of(filp->private_data,
                                                 struct mcp23s09,
                                                 mdev);
```

```
        dev_info(&mcp23s09->spidev->dev,
                 "Read from device file.\n");

        err = mcp23s09_get_port(mcp23s09, &mcp23s09->port_state);
        if (err) {
                dev_err(&mcp23s09->spidev->dev,
                        "Getting port value failed.\n");
                return err;
        }

        sprintf(port_buf, "0x%X\n", mcp23s09->port_state);

        err = copy_to_user(buf, port_buf, strlen(port_buf));
        if (err) {
                dev_err(&mcp23s09->spidev->dev,
                        "Sending data to userspace failed\n");
                return -EIO;
        }

        return count;
}


static struct file_operations fops =
{
        .owner = THIS_MODULE,
        .write = mcp23s09_write,
        .read = mcp23s09_read
};


static bool writeable_reg(struct device *dev, unsigned int reg)
{
        switch (reg) {
        case (MCP23S09_OPCODE << 8 | MCP23S09_DIR_REG):
        case (MCP23S09_OPCODE << 8 | MCP23S09_GPIO_REG):
                return true;
        }
        return false;
}

static bool readable_reg(struct device *dev, unsigned int reg)
{
        return (reg == (MCP23S09_OPCODE << 8 | MCP23S09_GPIO_REG));
}


static int mcp23s09_probe(struct spi_device *dev)
{
        int err = 0;
        struct mcp23s09 *mcp23s09;
        struct regmap_config reg_conf;

        mcp23s09 = devm_kzalloc(&dev->dev, sizeof(struct mcp23s09),
                                GFP_KERNEL);
```

```
        if (IS_ERR(mcp23s09)) {
                dev_err(&dev->dev,
                        "Device data allocation failed.\n");
                err = PTR_ERR(mcp23s09);
                goto out_ret_err;
        }

        mcp23s09->mdev.minor = MISC_DYNAMIC_MINOR;
        mcp23s09->mdev.name = "mcp23s09";
        mcp23s09->mdev.fops = &fops;
        err = misc_register(&mcp23s09->mdev);
        mcp23s09->spidev = dev;
        dev_set_drvdata(&dev->dev, mcp23s09);

        if (err) {
                dev_err(&dev->dev,
                        "Misc device registration failed.\n");
                goto out_ret_err;
        }

        /* regmap config */
        memset(&reg_conf, 0, sizeof(reg_conf));
        reg_conf.reg_bits = 16;
        reg_conf.val_bits = 8;
        reg_conf.writeable_reg = writeable_reg;
        reg_conf.readable_reg = readable_reg;
        reg_conf.read_flag_mask = 0x01;
        /* regmap init */
        mcp23s09->rmap = devm_regmap_init_spi(dev, &reg_conf);
        if (IS_ERR(mcp23s09->rmap)) {
                dev_err(&dev->dev,
                        "Regmap initialization failed.\n");
                err = PTR_ERR(mcp23s09->rmap);
                goto out_deregister_misc;
        }

        dev_info(&dev->dev, "SPI IO Driver Probed.\n");
        goto out_ret_err;

out_deregister_misc:
        misc_deregister(&mcp23s09->mdev);
out_ret_err:
        return err;
}

static void mcp23s09_remove(struct spi_device *dev)
{
        struct mcp23s09 *mcp23s09 = dev_get_drvdata(&dev->dev);
        misc_deregister(&mcp23s09->mdev);
        dev_info(&dev->dev, "SPI IO Driver Removed\n");
}

static const struct of_device_id mcp23s09_of_id[] = {
        { .compatible = "microchip,mcp23s09" },
        {},
};
```

```
MODULE_DEVICE_TABLE(of, mcp23s09_of_id);

static struct spi_driver mcp23s09_driver = {
        .probe = mcp23s09_probe,
        .remove = mcp23s09_remove,
        .driver = {
                .name = "mcp23s09",
                .of_match_table = mcp23s09_of_id,
                .owner = THIS_MODULE,
        },
};
module_spi_driver(mcp23s09_driver);

MODULE_LICENSE("GPL v2");
```

Komunikacja "sterownika" ze scalonym ekspanderem portów I/O z magistralą I2C

Podobnie jak w przypadku magistrali SPI, jądro udostępnia dedykowane funkcje i makra do obsługi magistrali I2C. Poniżej przedstawiono nakładkę drzewa urządzeń, która dodaje do węzła kontrolera *i2c1* urządzenie o nazwie *pcf8574* i adresie 0x20.

```
/dts-v1/;
/plugin/;

/ {
        compatible = "brcm,bcm2835";

        fragment@0 {
                target = <&i2c1>;
                __overlay__ {
                        #address-cells = <1>;
                        #size-cells = <0>;

                        pcf8574: pcf8574@20{
                                compatible = "microchip,pcf8574";
                                reg = <0x20>;
                                status = "okay";
                        };
                };
        };
};
```

Dzięki skalowalności i uniwersalności mechanizmu regmap, przebudowa sterownika bramy IO MCP23S09 na sterownik dla PCF8574 wymaga jedynie kilku kroków.

Ekspander PCF8574 nie posiada rejestrów więc nie obsługuje też ich adresacji. Sterowanie układem odbywa się za pomocą ramek danych które przedstawiono poniżej. [11]

Zapis
[S] [slave adres + write] [**ACK**] [dane do wysłania] [**ACK**] [dane do wysłania] [**ACK**] ... [P]

Odczyt
[S] [slave adres + read] [**ACK**] [odbiór danych] [ACK] [odbiór danych] [NACK] ... [P]

Opis parametrów:
- *S* – bit startu,
- *P* – bit stopu,
- *ACK* – potwierdzenie (master/slave),
- *NACK* – brak potwierdzenia (master/slave),
- *slave adres* – 7-bitowy adres układu slave,
- write – bit RW ustawiony na wartość *0*,
- read – bit RW ustawiony na wartość *1*.

W celu dostosowania sterownika MCP23S09 pracującego z magistralą SPI do obsługi magistrali I2C i układu PCF8574, należy:
- Zmienić strukturę odpowiadającą za konfigurację sterownika *struct spi_driver mcp23s09* na *struct i2c_driver pcf8574*.
- Zmienić makro *module_spi_driver(mcp23s09_driver)* na *module_i2c_driver(pcf8574_driver)*.
- Zmienić zmienną *struct spi_device spidev* na *struct i2c_client *client* (w strukturze z prywatnymi danymi urządzenia)
- Dostosować nazewnictwo funkcji i znmiennych (*"mpc23s09"* zamienić na *"pcf8574"*).
- Odpowiednio skonfigurować *regmap.*
- Dostosować funkcje *_get_port* i *_set_port.*

Zadanie – obsługa komunikacji I2C za pomocą mechanizmu regmap

1. Utworzyć plik nakładki drzewa urządzeń o nazwie *pcf8574.dts* i skopiować do niego kod przedstawiony we wstępie.
2. Skompilować nakładkę i załadować ją do jądra.
3. Otworzyć plik modułu jądra o nazwie *pcf8574.c*, który znajduje się w załącznikach. Zawiera on wstępnie przygotowany sterownik dla układu PCF8572 w którym należy dodać konfigurację *regmap* oraz zawartość funkcji *pcf8574_set_port* i *pcf8574_get_port*.
4. Skonfigurować regmap, w tym celu:
    - Wypełnić pola strutuktury *reg_conf:* 8 bitów adresu, 8 bitów danych, funkcje *writeable_reg* i *readable_reg*.
    - Zainicjalizować regmap za pomocą makra *devm_regmap_init_i2c.*

- W funkcjach weryfikujących poprawność adresu.
  Ponieważ układ PCF8574 nie obsługuje adresowania rejestrów, wartość podana w argumencie *reg,* podczas wysyłania danych zostanie potraktowana jak polecenie ustawienia stanów na porcie. Daje to możliwość wykorzystania adresu jako wartości ustawiającej stan portu przed jego odczytaniem (przed odczytaniem stanu pinu należy ustawić na nim stan wysoki (*1*)).
  W funkcjach *writeable_reg* i *readable_reg* sprawdzić czy adres zapisu jest równy *0x00*, oraz czy adres odczytu jest równy *0xff* (reset portu przed odczytem).

5. Dodać możliwość ustawiania i odczytywania portu, w tym celu:
   - Uzupełnić funkcję *pcf8574_set_port* – wysłać dane do układu za pomocą funkcji *regmap_write* (jak adres rejestru podać wartość *0x00*).
   - Uzupełnić funkcję *pcf8574_get_port* – wysłać dane do układu za pomocą funkcji *regmap_write* (jak adres rejestru podać wartość *0xff*).
6. Skompilować kod sterownika.
7. Załadować do jądra moduł *regmap_i2c* za pomocą polecenia *modprobe*
8. Załadować do jądra sterownik pcf8574 i przetestować jego działanie.
9. Zamieścić poniżej wydruk z konsoli i bufora diagnostycznego potwierdzający działanie sterownika.

```
pi@pi:~ $ sudo sh -c "echo 0xff >> /dev/pcf8574"
pi@pi:~ $ sudo sh -c "echo 0x55 >> /dev/pcf8574"
pi@pi:~ $ sudo head -n1 /dev/pcf8574
0xFF
pi@pi:~ $ sudo head -n1 /dev/pcf8574
0x55
pi@pi:~ $ sudo head -n1 /dev/pcf8574
0x00

pi@pi:~ $ dmesg
[25526.761759] pcf8574 1-0020: probed
[25526.772158] pcf8574 1-0020: I2C IO Driver Probed.
[25537.486911] pcf8574 1-0020: Write to device file.
[25537.493348] pcf8574 1-0020: Set port value.
[25544.254888] pcf8574 1-0020: Write to device file.
[25544.261542] pcf8574 1-0020: Set port value.
[25554.981272] pcf8574 1-0020: Read from device file.
[25554.987786] pcf8574 1-0020: Get port value.
[25564.728304] pcf8574 1-0020: Read from device file.
[25564.735128] pcf8574 1-0020: Get port value.
[25584.902424] pcf8574 1-0020: Read from device file.
[25584.909274] pcf8574 1-0020: Get port value.
```

10. Zamieścić poniżej kod sterownika pcf8574.

```
#include <linux/i2c.h>
#include <linux/miscdevice.h>
#include <linux/regmap.h>


#define PCF8574_WRITE_BUFF_SIZE 6
#define PCF8574_READ_BUFF_SIZE 6
#define PCF8574_PORT_MAX_VALUE 0xff
#define PCF8574_PORT_MIN_VALUE 0x00


struct pcf8574 {
        struct i2c_client *client;
        struct regmap *rmap;
        struct miscdevice mdev;
        int port_state;
};


static int pcf8574_set_port(struct pcf8574 *pcf8574,
                            unsigned int port_value)
{
        int err = 0;

        dev_info(&pcf8574->client->dev, "Set port value.\n");

        err = regmap_write(pcf8574->rmap, PCF8574_PORT_MIN_VALUE,
                           port_value);
        if (err) {
                dev_err(&pcf8574->client->dev,
                        "I2C communication error.\n");
                return err;
        }

        return 0;
}


static int pcf8574_get_port(struct pcf8574 *pcf8574,
                            unsigned int *value)
{
        int err = 0;

        dev_info(&pcf8574->client->dev, "Get port value.\n");

        err = regmap_read(pcf8574->rmap, PCF8574_PORT_MAX_VALUE,
                          value);
        if (err) {
                dev_err(&pcf8574->client->dev,
                        "I2C communication error.\n");
                return err;
        }
        return 0;
```

```
}


static ssize_t pcf8574_write(struct file *filp, const char *buf,
                             size_t count, loff_t *f_pos)
{
        int err = 0;
        int port_state;
        char kbuff[PCF8574_WRITE_BUFF_SIZE] = "";
        struct pcf8574 *pcf8574 = container_of(filp->private_data,
                                               struct pcf8574,
                                               mdev);
        dev_info(&pcf8574->client->dev, "Write to device file.\n");

        if (count > PCF8574_WRITE_BUFF_SIZE) {
                dev_err(&pcf8574->client->dev,
                        "Bad input number.\n");
                return -ERANGE;
        }

        err = copy_from_user(kbuff, buf, count);
        if (err) {
                dev_err(&pcf8574->client->dev,
                        "Can't copy data from user space.\n");
                return -EFAULT;
        }

        err = kstrtol(kbuff, 0, (long *) &port_state);
        if (err) {
                dev_err(&pcf8574->client->dev,
                        "Can't convert data to integer.\n");
                return err;
        }

        if (port_state < PCF8574_PORT_MIN_VALUE  ||
            port_state > PCF8574_PORT_MAX_VALUE) {
                dev_err(&pcf8574->client->dev,
                        "Bad voltage value.\n");
                return -EINVAL;
        }

        pcf8574->port_state = port_state;
        err = pcf8574_set_port(pcf8574, port_state);
        if (err) {
                dev_err(&pcf8574->client->dev,
                        "Could not set port value.\n");
                return err;
        }
        return count;
}

static ssize_t pcf8574_read(struct file *filp, char *buf,
                            size_t count, loff_t *f_pos)
{
        int err = 0;
        unsigned char port_buf[PCF8574_READ_BUFF_SIZE] = "";
```

```
        struct pcf8574 *pcf8574 = container_of(filp->private_data,
                                               struct pcf8574,
                                               mdev);

        dev_info(&pcf8574->client->dev, "Read from device file.\n");

        err = pcf8574_get_port(pcf8574, &pcf8574->port_state);

        sprintf(port_buf, "0x%X\n", pcf8574->port_state);

        err = copy_to_user(buf, port_buf, strlen(port_buf));
        if (err) {
                dev_err(&pcf8574->client->dev,
                        "Can't copy data to user space.\n");
                return -EIO;
        }

        return count;
}

static struct file_operations fops =
{
        .owner = THIS_MODULE,
        .write = pcf8574_write,
        .read = pcf8574_read
};

static bool writeable_reg(struct device *dev, unsigned int reg)
{
        return (reg == PCF8574_PORT_MIN_VALUE);
}

static bool readable_reg(struct device *dev, unsigned int reg)
{
        return (reg == PCF8574_PORT_MAX_VALUE);
}

static int pcf8574_probe(struct i2c_client *client,
                         const struct i2c_device_id *id)
{
        int err = 0;
        struct pcf8574 *pcf8574;
        struct regmap_config reg_conf;

        pcf8574 = devm_kzalloc(&client->dev, sizeof(struct pcf8574),
                               GFP_KERNEL);
        if (IS_ERR(pcf8574)) {
                dev_err(&client->dev, "Memory allocation failed.\n");
                err = PTR_ERR(pcf8574->rmap);
                goto out_ret_err;
        }

        pcf8574->client = client;

        dev_set_drvdata(&client->dev, pcf8574);
```

```
        /* miscdevice config */
        pcf8574->mdev.minor = MISC_DYNAMIC_MINOR;
        pcf8574->mdev.name = "pcf8574";
        pcf8574->mdev.fops = &fops;

        /* miscdevice config */
        err = misc_register(&pcf8574->mdev);
        if (err) {
                dev_err(&client->dev,
                        "Misc device registration failed.\n");
                goto out_ret_err;
        }

        /* regmap config */
        memset(&reg_conf, 0, sizeof(reg_conf));

        reg_conf.reg_bits = 8;
        reg_conf.val_bits = 8;
        reg_conf.writeable_reg = writeable_reg;
        reg_conf.readable_reg = readable_reg;

        /* regmap init */
        pcf8574->rmap = devm_regmap_init_i2c(client, &reg_conf);
        if (IS_ERR(pcf8574->rmap)) {
                dev_err(&client->dev, "Can't init spi regmap.\n");
                err = PTR_ERR(pcf8574->rmap);
                goto out_deregister_misc;
        }

        dev_info(&client->dev, "I2C IO Driver Probed.\n");

        goto out_ret_err;

out_deregister_misc:
        misc_deregister(&pcf8574->mdev);
out_ret_err:
        return err;
}


static void pcf8574_remove(struct i2c_client *client)
{
        struct pcf8574 *pcf8574 = dev_get_drvdata(&client->dev);
        misc_deregister(&pcf8574->mdev);

        dev_info(&client->dev, "I2C IO Driver Removed\n");
}


static const struct of_device_id pcf8574_of_id[] = {
        { .compatible = "microchip,pcf8574" },
        {},
};
MODULE_DEVICE_TABLE(of, pcf8574_of_id);
```

```
static struct i2c_driver pcf8574_driver = {
        .probe = pcf8574_probe,
        .remove = pcf8574_remove,
        .driver = {
                .name = "pcf8574",
                .of_match_table = pcf8574_of_id,
                .owner = THIS_MODULE,
        },
};
module_i2c_driver(pcf8574_driver);

MODULE_LICENSE("GPL v2");
```

Podsumowanie

Praca inżynierska stanowi praktyczne wprowadzenie do tematyki tworzenia sterowników w systemie Linux, której celem było zaznajomienie studenta z podstawowymi mechanizmami oraz technikami niezbędnymi do skutecznej implementacji sterowników w tym systemie operacyjnym. Przyjęta struktura instrukcji, łącząca teorię z zadaniami praktycznymi, pozwala na stopniowe przyswajanie wiedzy oraz bezpośrednią aplikację omawianych zagadnień. W ramach pracy, student miał możliwość zapoznania się z poszczególnymi aspektami implementacji sterowników, począwszy od podstawowych koncepcji, aż po bardziej zaawansowane techniki programowania i interakcji z systemem Linux.

Liniowy rozwój treści, w którym każde nowe zagadnienie jest oparte na wcześniej zdobytej wiedzy, stanowi kluczowy element strukturalny instrukcji. Takie podejście umożliwia studentom stopniowe wprowadzanie się w coraz bardziej złożone mechanizmy, co przyczynia się do efektywnego przyswajania materiału oraz głębszego zrozumienia funkcjonowania systemu Linux w kontekście implementacji sterowników.

Celem pracy było nie tylko przekazanie teoretycznej wiedzy, ale także rozwój praktycznych umiejętności niezbędnych do samodzielnej implementacji sterowników, co zostało osiągnięte dzięki starannemu doborowi zadań praktycznych ilustrujących omawiane zagadnienia. Każde z nich jest ściśle powiązane z przedstawioną wcześniej teorią, co pozwala studentowi na pełniejsze zrozumienie omawianych mechanizmów oraz ich zastosowanie w praktyce.

Podsumowując, praca stanowi solidne wprowadzenie do tematyki tworzenia sterowników w systemie Linux, oferując zarówno teoretyczne podstawy, jak i praktyczne doświadczenie w pracy z tym systemem. Instrukcja zaprojektowana w tej formie jest efektywnym narzędziem w nauce implementacji sterowników.

Bibliografia

[1] J. Corbet, A. Rubini, G. Kroah-Hartman, „Linux Device Drivers, Third Edition”, 2005
[2] GNU, General Public License, version 2,
[3] Kod źródłowy jądra Linux v6.1.21,
[4] S. French, The Linux Kernel documentation,
[5] Piyu Dhaker, Introduction to spi interface, Analog Devices, 2018
[6] Sal Afzal, I2C Primer: What is I2C? Analog Devices, 2016
[7] Microchip Technology Inc., MCP4921/4922 12-Bit DAC with SPI Interfaceauthor, 2007
[8] Simon Martin, Schemat platformy Raspberry Pi Zero W, 2021
[9] Dr. Steve Arar, Multiplication Examples Using the Fixed-Point Representation, 2017
[10] Microchip Technology Inc, 8-Bit I/O Expander with Open-Drain Outputs, 2014
[11] NXP Semiconductors, Remote 8-bit I/O expander for I2C-bus with interrupt, 2013